



GENOA UNIVERSITY
COMPUTER ENGINEERING

ADVANCED TECHNIQUES FOR PROPOSITIONAL SATISFIABILITY

TECNICHE INNOVATIVE PER LA SODDISFACIBILITÀ PROPOSIZIONALE

Supervisor

Prof. Enrico GIUNCHIGLIA

Co-supervisor

Dr. Paolo MARIN

Authors

Alessandro CARREGA
Raimondo BELLENO LOI

*“Multa non quia difficilia
sunt non audemus,
se quia non audemus
sunt difficilia.”*

Seneca

*“Gli uomini comuni guardano
le cose nuove con occhio vecchio.
L’uomo creativo guarda le cose
vecchie con occhio nuovo.”*

G.P. Bona

Sommario

Il problema di soddisfacibilità proposizionale (SAT) consiste nella risoluzione di una formula espressa in logica proposizionale. L’obiettivo è la risposta alla seguente domanda: esiste un assegnamento delle variabili che possa soddisfare tale formula? Ugualmente importante è individuare se tale assegnamento non esiste, arrivando a concludere che la funzione espressa dalla formula è identicamente “*falsa*” per tutti i possibili assegnamenti delle sue variabili. In questo ultimo caso si dice che la funzione è insoddisfacibile, altrimenti è soddisfacibile.

Il problema SAT appartiene alla classe di complessità NP-completa. I problemi NP-completi sono i più difficili della classe NP (non deterministici a tempo polinomiale). Hanno grande importanza sia teorica che pratica; se si trovasse un algoritmo in grado di risolvere in tempo polinomiale un qualsiasi problema NP-completo allora sarebbe possibile utilizzarlo per risolvere qualunque problema NP. La classe NP si riferisce a quei problemi che possono essere risolti in tempo polinomiale attraverso un algoritmo non deterministico. Nella pratica, nel caso peggiore, sono quindi risolti in tempo esponenziale.

Il problema SAT è di fondamentale importanza in un ampio spettro di settori dell’informatica, che vanno dall’informatica teorica [1] agli algoritmi [2], dall’intelligenza artificiale [3] [4], alla progettazione hardware [5] [6] fino all’automazione e verifica di progettazione di circuiti elettronici [7] [8].

Sulla base di ciò molti solver sono stati sviluppati negli ultimi anni allo scopo di allargare il portafoglio dei problemi risolvibili. Di questi ne sono stati esaminati alcuni tra quelli allo stato dell'arte quali *zChaff*, *MiniSat*, *Siege*, *RSat*, *BerkMin* e *MiniMarch*.

L'obiettivo di questo lavoro è lo sviluppo e l'analisi sperimentale di alcune tecniche innovative per il *solving*. Per questo scopo è stato progettato un solver, chiamato *StaGe*, che è stato quindi implementato da zero e non come estensione di un altro preesistente. Questa scelta è dovuta principalmente al fatto che nessun solver allo stato dell'arte presentava una struttura dati idonea allo sviluppo delle diverse tecniche. Per garantire la massima efficienza possibile del codice è stata utilizzato il linguaggio di programmazione C++.

In *StaGe* sono state implementate diverse tecniche già esistenti in solver allo stato dell'arte, come ad esempio la *Score Initialization* [9] delle euristiche e il *Progress Saving* [10], ed altre nuove, come la propagazione dei letterali puri (MLF, Monotone Literal Fixing) [11] e il metodo degli Implicanti.

La tecnica MLF consiste nel semplificare la formula assegnando un letterale a “vero” qualora il suo negato non occorra all'interno della stessa.

Il metodo degli Implicanti ha lo scopo di assegnare il maggior numero possibile di variabili per ogni assegnamento determinato dall euristica. In questo modo è possibile aumentare le probabilità di trovare un conflitto nelle clausole binarie e di conseguenza ridurre il numero di operazioni di BCP nelle clausole *n*-arie.

Per comparare l'efficacia di queste tecniche sono state utilizzate classi di istanze provenienti dalle ultime SAT-Competition e SAT-Race. Queste ultime, di rilevanza a livello internazionale come strumento di valutazione dei SAT-solver, hanno permesso di osservare le variazioni delle performance con classi di problemi provenienti da diversi domini. Una stima del tempo di macchina utilizzato per questo scopo conta approssimativamente 100 ore.

Nell'ultima parte sono analizzate le sopracitate tecniche preesistenti in combinazione a quelle degli Implicanti e dell'MLF.

Dai risultati si evince che non tutte le tecniche implementate riescono a migliorare le prestazioni del nostro solver. I migliori si hanno con la tecnica del *Progress Saving* in combinazione con la *Score Initialization* in quanto si

riscontra un aumento del numero di problemi risolti unitamente alla diminuzione del tempo totale di esecuzione. Per contro, le tecniche degli Implicanti e MLF non hanno dato i risultati sperati non portando a miglioramenti sulla maggior parte delle formule considerate.

Contents

1	Introduction	1
1.1	Objectives	2
1.2	Thesis Structure	3
2	State of the art	6
2.1	The DP and DL implementation	7
2.1.1	The Davis Putnam Algorithm	7
2.1.2	The Davis-Logemann-Loveland Algorithm	8
2.2	Benchmarks	9
2.3	zChaff	10
2.3.1	Benchmarking	12
2.4	MiniSat	14
2.4.1	Benchmarking	15
2.4.2	Minisat with expensive conflict minimization	16
2.5	Siege	20
2.5.1	Benchmarking	21
2.6	RSat	24
2.6.1	Benchmarking	24
2.7	BerkMin	27
2.7.1	Benchmarking	29
2.8	MiniMarch	32
2.8.1	Benchmarking	35
2.9	Solver Comparison	38
3	Solver	39
3.1	Database	45
3.1.1	Parser	54
3.2	Heuristics	58

3.3	BCP	61
3.3.1	Two Watching Literal Algorithm	62
3.3.2	Breath-First BCP	63
3.3.3	Unit	66
3.4	Backjumping	71
3.4.1	Implication Graph	71
3.4.2	Conflict Clause Learning	72
3.4.3	First-UIP	73
3.4.4	Backjumping Implementation	74
4	Advanced Techniques	77
4.1	StaGe benchmarking	77
4.2	Implicates	81
4.2.1	Different implementations	83
4.2.2	Benchmarking	89
4.3	Monotone Literal Fixing	95
4.3.1	Techniques description	95
4.3.2	Implementation	96
4.3.3	Benchmarking	101
4.4	Score Initialization	106
4.4.1	Benchmarking	106
4.5	Expensive Conflict Clause Minimization	110
4.5.1	Implementation	110
4.5.2	Benchmarking	113
4.6	Progress Saving	116
4.6.1	Implementations	116
4.6.2	Benchmarking	117
4.7	Look-ahead Technique	120
4.7.1	FLD	120
4.7.2	Implementation	121
4.7.3	LHT benchmarking	123
4.7.4	FLD benchmarking	125
4.8	Priority Propagation	130
4.8.1	Implementation	130
4.8.2	Benchmarking	131

5	Conclusions	134
5.1	Technique combination	134
5.2	Future work	145

Chapter 1

Introduction

The Boolean Satisfiability problem (SAT) is a decision problem of the complexity theory, which instance is a Boolean expression written using only AND, OR, NOT, variables and parentheses. The question is: given an expression, is there some assignment of FALSE or TRUE values to the variables that can make the entire expression true? Equally important is to determine whether no such assignments exist, which imply that the function expressed by the formula is identically FALSE for all possible variable assignments. In this latter case, the function is unsatisfiable.

A formula of propositional logic is said to be satisfiable if logical values can be assigned to its variables in a way that makes the formula TRUE. The propositional satisfiability problem (PSAT, [12]), which decides whether a given propositional formula is satisfiable, is of central importance in various areas of computer science, including theoretical computer science, artificial intelligence [2] [1] [3] [4], hardware design, electronic design automation and verification [5] [7] [6] [8]. These are a small pool of hypothetical applications of this kind of problems. From complexity theory we know that the boolean satisfiability problem is NP-complete [12] [13] [14]. If P is a proper subset of NP , there exists no algorithm g such that for some polynomial function, f , g decides any given SAT instance in time bounded above by f applied to the instance length.

The problem can be significantly restricted while still remaining NP-complete. By applying De Morgan's laws, we can assume that NOT operators are only applied directly to literals and not to expressions; we refer to either a variable or its negation as a literal. For example, both x_1 and $\neg x_2$ are literals, the first is a *positive* literal and the second a *negative* literal. If we

OR together a group of literals, we get a clause, such as $\{x_1 \vee \neg x_2\}$. Finally, let us consider formulas that are a conjunction (AND) of clauses. This is the conjunctive normal form (CNF).

By sufficiently restricting an intractable problem, one can always obtain a tractable problem. Some SAT instance classes are solvable in polynomial time. For example, instances containing no clause with more than two different literals can be solved in linear time [15]. Instances containing no clause with more than one positive literal are solvable in linear time [15]. Algorithms that take exponential time for some classes may take polynomial time for others. We are interested to test different new techniques in order to increase the efficiency to solve many instances generated through the application of SAT or practical problems arising in contemporary industry.

1.1 Objectives

As above described, the problem of boolean satisfiability has central importance in various areas of computer science, including algorithm languages, artificial intelligence, and hardware design. Efficient and scalable algorithms for SAT that were developed over the last decade have contributed to dramatic advances in our ability to automatically solve problem instances involving tens of thousands of variables and millions of clauses. Examples of such problems in Electronic Design Automation (EDA) include combinational equivalence checking, model checking, formal verification of pipe-lined microprocessors, automatic test pattern generation, routing of FPGA (field-programmable gate array), and so on. In fact, a SAT-solving engine is now considered to be an essential component in the EDA toolbox and all EDA vendors provide such a capability. In recent years industries such as IBM, Intel and Microsoft have begun to use with great success the Formal Verification Methods to validate combinatoric and sequential electronic circuits [5] [7] [6] [8], and software systems [2] [1] [3] [4].

For these reasons, different works have been developed in the last years. The results are, sometimes, not very good. The development of a new SAT-solver refers to the latest and best techniques in literature in this field of research. The differences between the solvers do not exclusively consist in the different introduced features but also in how these features are implemented. In the next chapter are described and analyzed several modern state-of-art solvers used in our experimental analysis as basis for comparison. Based on

these considerations, we have developed a new SAT-solver, called *StaGe*. In this solver, we have introduced new techniques and inserted the best features extracted from the state-of-art SAT-solvers. Our work has been focused on the experimental analysis of the different techniques. With these analysis it is possible to individuate which combinations of features can obtain the best results.

Our solver *StaGe* has been developed from scratch using the base techniques from *MiniSat* [16] and *RSat*[17], the two SAT-solvers with the best results in last SAT-Competition [18] and SAT-Race [19]. Other solvers have been used as basis for development. One of these ones is *Siege*[20], a solver developed taking into account every possible aspect of the use of memory. To this purpose, in *StaGe* binary clauses are stored in Database similarly to *Siege*, in order to improve performance during data access.

1.2 Thesis Structure

In Chapter 2 are explained the modern SAT-solvers as *MiniSat*, *RSat*, *Siege*, *zChaff*[21], and *MiniMarch*[22]. It is described the implementation of the different features used as Progress Saving [10], Expensive conflict clause minimization [16], and Look-ahead technique [22]. In particular are analyzed the performances of these solvers and the relative techniques with the benchmark instances used in the SAT-Race 2006¹.

In Chapter 3, it is described our solver *StaGe*, that has been developed to test the innovative techniques implemented. The core part of *StaGe* is the *Database* structure, which includes every information necessarily for every part of the solver as the formula and the learned clauses.

The Parser section describes the adopted solution to read the input formula from the file and to store these information into the Database. After this section, Chapter 3 reports the Heuristics part. As in the Database and Parser part, the solution is a specific adjustment of the version used in *MiniSat*, and *RSat*.

For the Boolean Constraint Propagation (BCP), it is traced the concept behind the two literal watching (TLW) scheme introduced in *Chaff*[21]. The

¹SAT-Race 2006 is a competitive event for solvers of the Boolean Satisfiability (SAT) problem. For further information see <http://fmv.jku.at/sat-race-2006/>

BCP operations is computed in two different strategies: one with a breadth search in each clause of the formula and the other one with two separated breadth searches. The first search visits the binary clauses and the second one the N -ary clauses. The second strategy tries to find the conflict in the binary clauses in order to reduce the size of the learned conflict clauses.

The Backtracking procedure is developed with the implementation of the so-called First Unique Implication Point (First-UIP or 1-UIP) introduced in [23] from the authors of *Chaff*. At the moment, this is the best method to implement the backtracking considering all the most important aspects: size of conflict clauses, memory occupation, and execution time [23].

Chapter 4 describes the new techniques developed and their combined use. At first, we analyze the Implicate technique. This feature allows to assign as many variables as possible for each assignment suggested by the heuristic. In this way, during the BCP procedure, it is easier to find a conflict in a binary clause and then reduce the size of the conflict clause.

The Monotone Literal Fixing finds the pure literals in the formula [11]. In our solver, this feature is introduced with a specific implementation derived from the work in [11].

Score Initialization is derived from that one introduced in *MiniMarch* and *Actin*[9]. With the initialization of variables (or literals) it is possible to make more efficient the search.

The expensive conflict clause minimization is based on the work in *MiniSat* and in *RSat*[16]. The other considered solvers do not include this characteristic. The advantage to reduce the size of the learned clauses may negatively affect the computational time.

The most innovative feature introduced in *RSat* is Progress Saving (PS) described in [10]. For each variable, it is stored the value of its last assignment. When the heuristic selects a variable, it uses this information to choose its sign and then is used to reduce the amount of work repetition that is inherent in backtracking SAT-solvers.

Another feature describes in Chapter 4 is the Flip Last Decision (FLD) technique that consists in a modified version of Look-ahead procedure introduced in *MiniMarch*. In our solver, *StaGe*, it is possible to flip (i.e. change the sign of the variable) the assignment of the last variable if its value does not affect the implication graph.

In Chapter 5 the results of our experiments and techniques are summarized. The conclusions about our work report that in this area there is not a perfect solution. The continuous improvements are possible with the use of new features or with a more efficient logic implementation. Our work opens more question about the use or not of some characteristics.

The last section of our thesis is dedicated to the future work. For example, it could be possible to integrate other important techniques used in *RSat* as the heuristic for ordering the implications. Even more interesting, it could be the introduction of the Reassignments feature that could improve the performances of a SAT-Solver.

Chapter 2

State of the art

Boolean satisfiability is well studied, and SAT-solvers have a long history. The *Davis Putnam* (DP) algorithm [24], published in 1960, is typically known as the first entry in the solver time-line. It is very inefficient in finding a satisfying assignment to solve real world instances and, moreover, the memory consumption is explosive in practice. Because of that, the new *Davis-Logemann-Loveland* (DLL) algorithm [25] was developed.

Over the past decade, several powerful DLL solvers has been introduced, including *Posit* [26], *Satz* [27], and *Kcnfs* [28]. These solvers are different from the original DLL algorithm in the fact that they have better decision heuristics. Such solvers are still at the state of art for proving random instances unsatisfiable (incomplete local-search programs, such as *UnitWalk* [29], are more successful on satisfiable random instances).

Conflict-driven clause learning was first used in the solvers *Grasp* [30] and *RelSat* [31]. By appending implicates to the formula and using these implicates to perform non-chronological backtracking, these solvers outperform the best DLL solvers on non-random instances. These techniques pose the basis of all the most successful modern solvers for instances coming from industry.

The solver *Chaff* made *Grasp* and *RelSat* obsolete. The most important innovation of *Chaff* was a decision strategy that allows it taking better advantage in conflict-driven clause learning. *Chaff* also introduced a variation of two-pointer BCP algorithm published several years earlier as part of *Sato* solver [32]. Because of *Chaff* was designed to be applied in industrial setting, heavy emphasis was placed on an efficient implementation.

2.1 The DP and DL implementation

Most competitive modern SAT solvers, are, in most cases, extensions of Davis-Logemann-Loveland (DLL) algorithm [25]. The DLL algorithm is a variation of the Davis-Putnam (DP) algorithm [24].

2.1.1 The Davis Putnam Algorithm

The input formula is F .

1. If the input contains an *empty clause*, return UNSATISFIABLE.
2. Remove from F all clauses that contain at once both literals x and $\neg x$ of some variable x . If F is now empty, return SATISFIABLE.
3. If F contains both the clauses $[x]$ and $[\neg x]$, return UNSATISFIABLE.
4. If there is a unit clause $[x]$ in F , delete from F all clauses containing x , and remove the negation of x from all clauses where it appears. If F is now empty, return SATISFIABLE.
5. If F contains a unit clause go back to 4.
6. While there is a pure literal in F , delete all clauses where it appears. If F is now empty, return SATISFIABLE.
7. Select a variable, x , that has a literal in one of the shortest clauses in F . Replace F with $F \wedge [x]$. Go back to step 2.

Step 2 removes the clauses that are trivially satisfied. Steps 3 and 4 constitute the *boolean constraint propagation* (BCP). In step 3, the formula is checked for contradictory unit clauses. The presence of both clauses x and $\neg x$ makes the formula trivially unsatisfiable. In step 4 all instances of x are removed through *unit-subsumption*. If x is FALSE then the unit clause $[x]$ is unsatisfiable. Thus x being TRUE is a necessary consequence of any truth assignment that satisfies all clauses in F . The clauses containing x are satisfiable and it is possible to remove them. If the negation of x appears, it cannot contribute to satisfying a clause where it occurs. Therefore it may be removed from such clauses that will necessarily be satisfied through other assignments. Step 6 purges every *pure literal* from the formula. A literal is pure in F if its negation does not occurs in the formula. It is safe to make these ones TRUE because the

assignment with FALSE cannot contribute to satisfying any clause. Most SAT solvers do not actually implement this procedure, because it is expensive to monitor if a literal occurs in a satisfied clause. This rule is developed in the *Monotone Literal Fixing* (MLF) strategy [11]. Whenever the execution reaches the step 7, all previous strategies can not go on simplifying the formula and then a variable has to be selected. The policy of selection, or *decision strategy*, plays an important role in determining the overall performance of the search.

2.1.2 The Davis-Logemann-Loveland Algorithm

The Davis-Putnam algorithm may make the formula growing up exponentially. Since the addition space used by DLL is only linear in the number of variables, it is preferred to the DP algorithm.

The input formula is F .

1. Let $[x]$ be a unit clause in F . Remove from F all clauses that contain the literal x . Remove the negation of x from all clauses where it appears.
2. If F contains a unit clause go back to 1.
3. If F contains an empty clause, return UNSATISFIABLE.
4. While there is a pure literal in F , delete all clauses where a pure literal occurs. If F is now empty, return SATISFIABLE.
5. Select variable x that occurs in F .
6. Call the formula $F \wedge [x]$. If the call returns SATISFIABLE then returns SATISFIABLE.
7. Call the formula $F \wedge [\neg x]$. If the call returns SATISFIABLE then returns SATISFIABLE, else returns UNSATISFIABLE.

Steps 1 and 2 implement boolean constraint propagation. In step 5 a decision strategy is employed to choose x . As for the DP algorithm, the policy of choosing variables for removal can have a great impact on overall performance. In step 6, $F \wedge [x]$ forces x to be TRUE and in step 7, $F \wedge [\neg x]$ forces x to be FALSE. F is satisfiable if and only if either it has a satisfying assignment under which x is TRUE, or it has one under which x is FALSE.

2.2 Benchmarks

The benchmarks used to test the performances of various state-of-art solvers and in experimental analysis have been selected by SAT-Race 2006 and SATLIB.

As mentioned in Chapter 1, SAT-Race is a competitive event for solvers of the Boolean Satisfiability (SAT) problem. It contains benchmarks from the industry and it is strongly updated.

SATLIB is a collection of benchmark problems, solvers, and tools regarding the Boolean Satisfiability (SAT) problem. The main objective of SATLIB is to provide a uniform test-bed for SAT-solvers as well as a site for collecting SAT problem instances, algorithms, and empirical characterisations of the algorithms' performance ¹. This collection is not really up to date, and many benchmarks, are solvable from every modern SAT-solvers in just a few seconds. However, there are still some benchmarks that are not solved by any state of art SAT-solvers.

Inst. family(#,Sat,Unsat)	V	C	$\frac{C}{V}$	Bin	$\frac{\text{Lits}}{N\text{-Clauses}}$
aloul (1, 0, 1)	260.00	1586.00	6.10	98.36%	10.00
een (3, 0, 3)	83468.00	251647.00	3.01	59.98%	3.05
goldb (4, 0, 4)	15271.50	95211.00	6.23	22.98%	2.84
grieu (3, 3, 0)	643.33	80498.67	125.13	19.64%	3.02
hoons (4, 0, 4)	18287.00	54661.50	2.99	66.65%	3.00
manol-pipe (35, 0, 35)	93858.91	278301.31	2.97	66.67%	3.00
mizh (3, 2, 0)	60395.00	221424.33	3.67	44.25%	3.62
narain (3, 1, 2)	1741952.00	6808991.33	3.91	76.19%	4.04
schup (3, 0, 3)	338220.00	1069494.00	3.16	48.00%	2.07
simon (4, 3, 1)	9113.50	32073.50	3.52	48.42%	4.37
stric-bmc (1, 1, 0)	39598.00	194641.00	4.92	73.92%	4.51
vange (1, 1, 0)	117426.00	432787.00	3.69	8.04%	3.14
velev (34, 18, 16)	112890.65	3753762.56	33.25	80.79%	9.71

Table 2.1: Industrial benchmarks selected from former SAT-Race 2006. Each row represents a different instance family, with the following information: “V”, average number of variables; “C”, average number of clauses; “C / V”, average ratio between number of clauses and number of variables; and “Lits / N-Clauses”, average number of literals for nary clauses.

¹For further information see <http://www.satlib.org/>

In our tests, for every solver was given 15 minutes to decide each instance on an *1.73 GHz Pentium Mobile M with 1 gigabyte of RAM*. The industrial benchmarks from pipe-lined machine verification (*manol-pipe*, *velev*) derive from the encoding of electronic design automation problems.

Inst. family(#,Sat,Unsat)	V	C	$\frac{C}{V}$	Bin	$\frac{\text{Lits}}{\text{N-Clauses}}$
ais (4, 4, 0)	155.00	2729.50	17.61	49.13%	3.16
BF (4, 0, 4)	1697.50	5037.25	2.97	57.16%	3.20
blocksworld (7, 7, 0)	1644.29	29877.86	18.17	75.10%	3.05
BMC (13, 13, 0)	30077.46	160334.38	5.33	78.60%	5.25
flat (100, 100, 0)	90.00	300.00	3.33	90.00%	3.00
GCP (4, 4, 0)	3843.75	206255.50	53.66	99.91%	19.75
hanoi (2, 2, 0)	1324.50	9693.00	7.32	29.18%	3.06
ind-inference (41, 41, 0)	655.90	7994.73	12.19	92.65%	18.32
logistics (4, 4, 0)	1881.25	11682.25	6.21	70.19%	3.78
parity (30, 20, 0)	1044.47	3542.40	3.39	22.79%	3.00
phole (5, 0, 5)	74.00	322.00	4.35	97.20%	8.00
pret (8, 0, 8)	105.00	281.00	2.68	0.00%	3.00
QG (22, 10, 12)	1043.18	29078.36	27.87	42.24%	3.38
SSA (8, 4, 4)	2351.00	6674.38	2.84	60.53%	3.18
SW100 (100, 100, 0)	500.00	3100.00	6.20	96.77%	5.00

Table 2.2: Benchmarks selected from SATLIB.

The benchmarks selected from SATLIB are shown in Table 2.2. Are included the following instance families: BF (circuit fault analysis: Bridge Fault), blocksworld and logistics (planning), BMC (SAT-encoded bounded model checking), flat (“Flat” Graph Colouring), GCP (large SAT-encoded Graph Colouring Problems), hanoi (SAT-encoding of Towers of Hanoi), ind-inference (from a problem in inductive inference), parity (from problem in learning the parity function), phole (Pigeon hole problem), pret (encoded 2-colouring), QG (SAT-encoded Quasigroup), SSA (Circuit fault analysis: Single-Stuck-At fault), and SW100 (“Morphed” Graph Colouring).

2.3 zChaff

zChaff is a SAT-solver that targets the industrial category and hopes to be reasonably successful in the handmade category. It implements the well known *Chaff* algorithm [33] which includes the VSIDS decision strategy. The principal feature introduced in *zChaff* are:

- Decision Strategy - VSIDS and *BerkMin* Type

- Boolean Constraint Propagation - Two literal watching
- Non-chronological Backtracking - Learning the First-UIP Conflict Clause
- Conflict Clause Based Assignment Stack Shrinking

Decision Strategy - Vsids and *BerkMin* Type

The use of a heuristic called *Variable State Independent Decaying Sum* is the most important contribution of *Chaff*. It is a literal count heuristic. It allows *Chaff* (and *zChaff*) to solve difficult industrial SAT problems much faster, with far fewer decisions than solvers like *RelSat*, *Grasp*, and *Sato*.

VSIDS is realized in *zChaff* as follows. Each literal l has a score $s(l)$, and an occurrence count $r(l)$. When a decision is necessary, a free literal with the highest score is set TRUE. Initially for each literal, l , $s(l) = r(l) = 0$. Before search begins, $s(l)$ is incremented for each occurrence of a literal, l , in the input formula. When a clause c is learned during search, $r(l)$ is incremented for each literal $l \in c$. Every 255 decisions, the scores are updated: for each literal l $s(l)$ becomes $r(l) + s(l)/2$, and $r(l)$ becomes zero. *zChaff* uses this decision strategy with another one: *BerkMin Type Decision Strategy*. This latter decision strategy is used with the most recent unsatisfied conflict clauses. The main ideas of this approach are described by the author of *BerkMin* [34].

Boolean Constraint Propagation - Two literal watching

zChaff uses the two literal watching scheme [33] for BCP as proposed by *Chaff* [33]. A key benefit of the two literal watching scheme is that at the time of backtracking, there is no need to modify the watched literals in the clause database. This reduces the total number of memory access. This scheme is discussed in Chapter 3.

Non-chronological Backtracking - Learning the First-UIP Conflict Clause

Conflict driven clause learning along with non-chronological backtracking were first incorporated into a SAT-solver in *Grasp* and *RelSat*. These techniques are essential for efficient solving of structured problems. This technique is described in Chapter 3.

Conflict Clause Based Assignment Stack Shrinking

This is related to one of the techniques used by the *JeruSat* solver [35]. When new learned First-UIP clause exceeds a certain length L , it participates in the decision strategy.

2.3.1 Benchmarking

The results of *zChaff*, compared to the other solvers (*MiniSat*, *RSat*, *BerkMin*, *MiniMarch* and *StaGe*) are not so relevant. It is normal that the performances are not comparable to other solvers because *MiniSat*, *RSat*, *BerkMin*, and *MiniMarch* are more recent.

Inst. family (#,S)	CPU-Time (s)	Decisions ($\times 10^3$)	Conflicts ($\times 10^3$)
aloul (1/1)	10.7	31.2	30.0
een (3/3)	102.6	225.2	14.3
goldb (2/4)	2074.4	1632.6	105.5
grieu (1/3)	2239.6	264.7	216.4
hoons (2/4)	2874.4	717.1	259.1
manol-pipe (28/34)	10736.0	1051.0	56.5
mizh (0/3)	2700.0	-	-
narain (2/3)	2101.9	32.2	5.6
schup (0/3)	2700.0	-	-
simon (1/4)	3595.7	6284.5	286.1
stric-bmc (1/1)	27.7	77.2	14.4
vange (0/1)	900.0	-	-
velev (19/34)	17248.7	1240.6	54.7
TOTAL (60/98)	47311.7	1085.0	65.9

Table 2.3: *zChaff* on SAT-Race benchmarks. Each row represents a different instance family, with the number of solved/total instances. The “CPU-Time” field contains the total time (in seconds) required to solve the instances. The “Decisions” and “Conflicts” fields represent respectively the average number of decisions and conflicts. If an instance is not solved, the CPU-Time used is the timeout parameter (900 seconds). Obviously, the average number of decisions and conflicts are evaluated considering only the instances solved.

In Table 2.3 are shown the results of *zChaff* on industrial benchmarks selected from SAT-Race 2006. The best performances are with the manol instance family.

Inst. family (#,S)	CPU-Time (s)	Decisions ($\times 10^3$)	Conflicts ($\times 10^3$)
AIS (4/4)	1.2	2.9	2.7
BF (4/4)	0.0	0.3	0.1
blocksworld (7/7)	0.8	0.9	0.3
BMC (13/13)	60.7	23.0	4.2
flat (100/100)	0.0	0.0	0.0
GCP (0/4)	3600.0	-	-
hanoi (2/2)	96.8	243.6	33.4
ind-inference (41/41)	1.3	0.6	0.1
logistics (4/4)	0.1	5.4	0.3
parity (20/30)	9005.4	2.2	2.1
phole (5/5)	8.9	7.8	7.5
pret (8/8)	0.3	2.1	1.2
QG (22/22)	144.7	13.9	9.8
SSA (8/8)	0.0	0.2	0.0
SW100 (100/100)	0.0	0.1	0.0
TOTAL (338/352)	12920.5	3.8	1.3

Table 2.4: *zChaff* on SATLIB benchmarks.

The other benchmarks selected from our experiments are shown in Table 2.4. The problems are with the GCP and parity instance families. The other instances are solved without problems by *zChaff*.

In general, *zChaff* has been tested in order to obtain a starting point. In this way it is possible to compare the subsequent solvers which we are going to analyze.

2.4 MiniSat

MiniSat is a minimalistic implementation of a *Chaff*-Like SAT solver. This is based on two literal watching scheme for fast BCP [33]. The version of *MiniSat* used with our benchmarks is 2.0, which includes variable elimination style simplification natively.

The most important aspects regard:

- Variable Order
- Binary Clauses
- Clause Deletion
- Conflict Clause Minimization

Variable Order

The decision heuristic of *MiniSat* is an improved VSIDS order, where variable activities are decayed 5% after each conflict. Important to the heuristic is increasing the activity of all variables occurring in some clauses used in the conflict analysis, not just variable of the final conflict clause (as in the first version of *MiniSat*). To keep the variable sorted on activity at all times, a heap is used. *StaGe* uses a version derived from this heuristic and it is discussed in Chapter 3.

Binary Clauses

Binary clauses are implemented by storing the literal to be propagated directly in the watcher list. For the author, storing all binary clauses in a separated set of vectors and propagating all binary clauses before or after the bigger ones affects the implication graph and seems to produce worse conflict clauses. In our solver, *StaGe*, the binary clauses are implemented, for each literal, with dedicated vectors (like in *Siege*) and they are propagated before the other bigger clauses. There is not a perfect solution about this problem. Every approach has the positive and negative effects. In Chapter 3 the different implementations are summarized.

Clause Deletion

MiniSat aggressively deletes learned clauses based on an activity heuristic similar to the one for variables. The limit on how many learned clauses are allowed is increased after each restart. Keeping the number of clauses low seems to be particular important for some small but hard problems. Some other solvers use this strategy to delete the learned clauses with different parameters. For example, *RSat* uses a modified version of this procedure.

Conflict Clauses Minimization

This is the best improvement of *MiniSat* 2.0 respect to original version. There are two different approaches for the conflict clause minimization: one soft and one more expensive in terms of computational time. This feature is used in our solver *StaGe* and it is described in Chapters 3 and 4. In practice, the conflict clause minimization allows to reduce the size of conflict clauses learned and this one can increase the efficiency of the resolution.

2.4.1 Benchmarking

The results of *MiniSat* in our experiments, with the same instances used with *zChaff*, explain that this is one of the best modern SAT solvers for different kind of problems.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	ZC	MS	ZC	MS	ZC	MS
aloul (1/1)	10.7	173.2	31.2	3140.1	30.0	2663.4
een (3/3)	102.6	9.9	225.2	43.4	14.3	8.7
goldb (3/4)	2074.4	1967.6	1632.6	2758.8	105.5	587.1
grieu (2/3)	2239.6	916.9	264.7	57.9	216.4	38.4
hoons (3/4)	2874.4	1415.3	717.1	219.1	259.1	177.7
manol-pipe (30/34)	10736.0	5930.8	1051.0	483.6	56.5	115.3
mizh (2/3)	2700.0	1118.2	-	1624.7	-	294.1
narain (3/3)	2101.9	1741.8	32.2	260.7	5.6	105.4
schup (2/3)	2700.0	1422.8	-	337.7	-	243.1
simon (3/4)	3595.7	2269.8	6284.5	1544.9	286.1	998.7
stric-bmc (1/1)	27.7	7.6	77.2	30.3	14.4	10.5
vange (1/1)	900.0	15.5	-	449.5	-	102.2
velev (20/34)	17248.7	13486.4	1240.6	332.6	54.7	34.2
TOTAL (74/98)	47311.7	30475.9	1085.0	585.1	65.9	185.2

Table 2.5: *MiniSat* (MS) on SAT-Race benchmarks compared to *zChaff* (ZC).

The results in Table 2.5 show that *MiniSat* works better than *zChaff* with these types of benchmarks. It solves more instances and the CPU-time spent is lower.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	ZC	MS	ZC	MS	ZC	MS
AIS (4/4)	1.2	0.0	2.9	0.1	2.7	0.0
BF (4/4)	0.0	0.0	0.3	0.4	0.1	0.1
blocksworld (7/7)	0.8	0.5	0.9	1.1	0.3	0.4
BMC (13/13)	60.7	16.2	23.0	11.3	4.2	2.9
flat (100/100)	0.0	0.1	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	96.8	2.7	243.6	45.8	33.4	24.1
ind-inference (41/41)	1.3	1.2	0.6	0.6	0.1	0.3
logistics (4/4)	0.1	0.1	5.4	2.7	0.3	0.3
parity (20/30)	9005.4	9002.1	2.2	2.0	2.1	1.8
phole (5/5)	8.9	40.9	7.8	338.4	7.5	276.5
pret (8/8)	0.3	0.0	2.1	0.0	1.2	0.0
QG (22/22)	144.7	24.6	13.9	8.1	9.8	6.7
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.3	0.1	0.2	0.0	0.0
TOTAL (338/352)	12920.5	12688.9	3.8	6.6	1.3	4.9

Table 2.6: *MiniSat* (MS) on SATLIB benchmarks compared to *zChaff* (ZC).

In Table 2.6, the results are the same of *zChaff* as number of instances solved. However, they show a significant reduction in the time required to solve the instances.

In general, *MiniSat* outperforms the results of *zChaff*. This conclusion derives from the fact that *MiniSat* extends the core of *Chaff* with new features that *zChaff* does not include.

2.4.2 Minisat with expensive conflict minimization

In our experiments, we tested *MiniSat* with the standard implementation for conflict minimization. In order to analyze the conflict minimization procedure we tested the expensive conflict clause minimization in *MiniSat*. We have called this version of *MiniSat* as “*MiniSat-Simp*”.

In industrial benchmarks in Table 2.7, *MiniSat-Simp* solves more instances than *MiniSat*. In addition to a greater number of solved instances also the performances are better than the standard version of *MiniSat*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	MSimp	MS	MSimp	MS	MSimp
aloul (1/1)	173.2	112.7	3140.1	2216.8	2663.4	1885.3
een (3/3)	9.9	8.8	43.4	33.9	8.7	11.3
goldb (3/4)	1967.6	1978.0	2758.8	2790.7	587.1	557.7
grieu (3/3)	916.9	858.2	57.9	596.2	38.4	387.0
hoons (3/4)	1415.3	1068.8	219.1	80.3	177.7	61.5
manol-pipe (32/34)	5930.8	5087.1	483.6	905.1	115.3	263.4
mizh (2/3)	1118.2	1893.2	1624.7	2579.7	294.1	515.9
narain (2/3)	1741.8	1769.2	260.7	13.4	105.4	5.0
schup (2/3)	1422.8	1039.4	337.7	272.5	243.1	190.2
simon (3/4)	2269.8	1643.8	1544.9	1008.9	998.7	724.8
stric-bmc (1/1)	7.6	3.2	30.3	12.0	10.5	4.2
vange (1/1)	15.5	81.3	449.5	1167.4	102.2	310.6
velev (21/34)	13486.4	12929.9	332.6	345.2	34.2	41.4
TOTAL (77/98)	30475.9	28473.4	585.1	764.5	185.2	235.7

Table 2.7: *MiniSat-Simp* (MSimp) on SAT-Race benchmarks compared to *MiniSat* (MS).

It is simple to realize that this technique improves the efficiency of the resolution. This is not what happens with benchmarks selected from SATLIB in Table 2.8 where the number of instances solved is the same of the other solvers but there is a slight worsening in terms of performances.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	MSimp	MS	MSimp	MS	MSimp
AIS (4/4)	0.0	0.0	0.1	0.1	0.0	0.0
BF (4/4)	0.0	0.1	0.4	0.2	0.1	0.1
blocksworld (7/7)	0.5	1.3	1.1	1.4	0.4	0.6
BMC (13/13)	16.2	21.5	11.3	7.5	2.9	3.5
flat (100/100)	0.1	0.2	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	2.7	2.5	45.8	46.1	24.1	24.1
ind-inference (41/41)	1.2	1.9	0.6	0.4	0.3	0.1
logistics (4/4)	0.1	0.2	2.7	1.1	0.3	0.1
parity (20/30)	9002.1	9001.4	2.0	1.8	1.8	1.7
phole (5/5)	40.9	73.9	338.4	542.5	276.5	454.9
pret (8/8)	0.0	0.0	0.0	0.0	0.0	0.0
QG (22/22)	24.6	18.9	8.1	6.1	6.7	4.9
SSA (8/8)	0.0	0.1	0.2	0.1	0.0	0.0
SW100 (100/100)	0.3	1.5	0.2	0.1	0.0	0.0
TOTAL (338/352)	12688.9	12723.6	6.6	9.2	4.9	7.5

Table 2.8: *MiniSat-Simp* (MSimp) on SATLIB benchmarks compared to *MiniSat* (MS).

Comparison

In Figures 2.1 and 2.2 are shown the differences between the standard version of *MiniSat* and the one with expensive conflict minimization (*MiniSat-Simp*). These graphs confirm the improvement of *MiniSat-Simp* than the standard version.

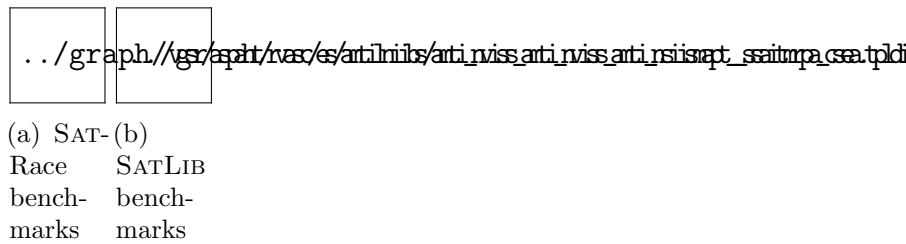


Figure 2.1: Comparison between *MiniSat* and *MiniSat-Simp* on (a) SAT-Race and (b) SATLIB benchmarks.

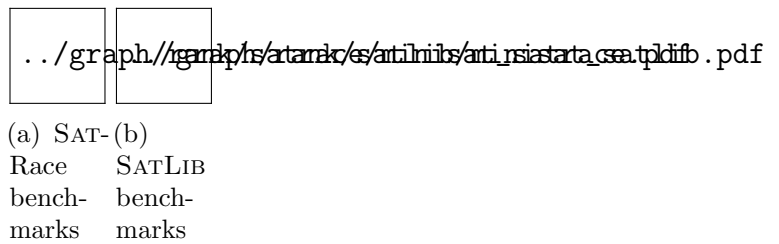


Figure 2.2: Comparison between *MiniSat* and *MiniSat-Simp* on (a) SAT-Race and (b) SATLIB benchmarks.

2.5 Siege

Siege is a modern SAT solver which is focused on hardware optimization [20]. For this reason the author, Lawrence Ryan, designs a solver considering the kind of problems that it is supposed to solve. His work is concerned primarily with SAT instances that have generated as a means of solving real-world electronic design automation problems. The backtracking/conflict clause learning technique used is the same of *zChaff* and *Chaff*. The BCP is performed with a classic two Literal watching scheme.

The differences between these solvers and other ones are on different implementation of database. *Siege* spits the clauses in binaries, ternaries and N -aries. Every kind of clauses is implemented in a specific way in order to reduce the access time to the memory. In addition, Ryan thinks that it is important to avoid (or reduce) the missing cache and, for this reason, for each literal is allocated 20 bits, one bit for the index and 19 bits for the variable index. In this way the solver supports no more than 524,288 variables. This restriction reduces the set of instances to which the solver is applicable.

Siege uses the classic two literal watching scheme with 21 bits for literal (20 + 1 for watched flag). Therefore, it is possible to store three 21-bit literal instances into 64 bits of memory. This is important because *Siege* uses a specific structure for ternary clauses. For every literal there is an array containing the other literals that are in the same ternary clauses, the first two literals for the first clause, the second two for the second one and so on. The same considerations apply to the binary clauses. For each literal there is a vector containing the literals that occur in the same binary clauses. Therefore, *Siege* is a SAT-solver that tries to improve the boolean constraint propagation because about 90 percent of the run-time of a clause-learning DLL solver is spent performing this procedure.

The decision heuristic used is called VMTF, *variable move-to-front decision strategy*. It is simple and extremely inexpensive to compute. An occurrence count $r(l)$ is kept for each literal l . Initially, for every l , $r(l) = 0$. Before search begins $r(l)$ is incremented for each occurrence of a literal, l , in the input formula. An ordered list of the formula, W , is also maintained. Once the counts have been determined for the input formula, W is arranged so that, for all variables v_0, v_1 : if $r(v_0) + r(\neg v_0) > r(v_1) + r(\neg v_1)$, then v_0 precedes v_1 . The more often a variable occurs in the input formula, the closer it begins to the front of the list. When a clause, c , is learned during search, $r(l)$ is

incremented for each literal $l \in c$. Then, some of the variables in c are moved to the front of W . The number of variables moved is a small constant, m , e.g., 8. If c contains only $n < m$ literals, n variables are moved. The moved variables are positioned at the front of the list in an arbitrary order. When a decision is necessary, the free variable, v , that is nearest the front of W is set TRUE if $r(v) > r(\neg v)$, FALSE if $r(\neg v) > r(v)$, and randomly otherwise. Clearly then, VMTF does not simply choose variables from the most recently derived clause.

The other parts of *Siege* are the same seen in most of modern SAT-solvers with, obviously, a different implementation. It is impossible to describe the specific implementation because *Siege* is not an open source SAT-solver.

2.5.1 Benchmarking

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	SI	MS	SI	MS	SI
aloul (1/1)	173.2	234.0	3140.1	217.2	2663.4	211.6
een (3/3)	9.9	38.4	43.4	141.5	8.7	17.5
goldb (2/4)	1967.6	2154.6	2758.8	1861.7	587.1	364.6
grieu (1/3)	916.9	1819.4	57.9	47.1	38.4	42.0
hoons (2/4)	1415.3	2033.9	219.1	255.0	177.7	193.1
manol-pipe (30/34)	5930.8	7495.8	483.6	836.1	115.3	136.2
mizh (0/3)	1118.2	2700.0	1624.7	-	294.1	-
narain (0/3)	1741.8	2700.0	260.7	-	105.4	-
schup (2/3)	1422.8	1459.9	337.7	346.7	243.1	267.0
simon (0/4)	2269.8	3600.0	1544.9	-	998.7	-
stric-bmc (1/1)	7.6	7.6	30.3	38.8	10.5	11.5
vange (1/1)	15.5	27.5	449.5	1312.1	102.2	31.0
velev (30/34)	13486.4	9274.1	332.6	1345.3	34.2	104.8
TOTAL (73/98)	30475.9	33545.1	585.1	991.9	185.2	126.4

Table 2.9: *Siege* (SI) on SAT-Race benchmarks compared to *MiniSat* (MS).

Siege is specialized to problems of real-world electronic design automation, like pipe-lined machine verification. Indeed, *Siege* has the best results with Industrial benchmarks, especially velev instance family (Table 2.9). In some cases (like narain instance family) it is impossible to solve the instances because the number of variables is bigger then the limit above described. The number of instances solved is higher than *zChaff* case but lower than *MiniSat* one. In addition, the CPU-time required is higher than *MiniSat* case but lower than *zChaff* case.

With the SATLIB benchmarks in Table 2.10, the number of instances solved is the same solved by *MiniSat*. However, there is a substantial worsening about the CPU-time spent.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	SI	MS	SI	MS	SI
AIS (4/4)	0.0	0.4	0.1	2.0	0.0	1.8
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	0.5	0.6	1.1	2.0	0.4	0.6
BMC (13/13)	16.2	28.6	11.3	23.9	2.9	5.7
flat (100/100)	0.1	1.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	2.7	34.9	45.8	97.5	24.1	72.1
ind-inference (41/41)	1.2	1.2	0.6	0.8	0.3	0.2
logistics (4/4)	0.1	0.1	2.7	3.7	0.3	0.3
parity (20/30)	9002.1	9003.4	2.0	2.5	1.8	2.3
phole (5/5)	40.9	91.3	338.4	32.8	276.5	32.0
pret (8/8)	0.0	0.1	0.0	1.9	0.0	0.5
QG (22/22)	24.6	42.5	8.1	7.8	6.7	6.6
SSA (8/8)	0.0	0.1	0.2	0.3	0.0	0.0
SW100 (100/100)	0.3	1.0	0.2	0.1	0.0	0.0
TOTAL (338/352)	12688.9	12805.1	6.6	2.9	4.9	1.8

Table 2.10: *Siege* (SI) on SATLIB benchmarks compared to *MiniSat* (MS).

In conclusion, *Siege* is focused on resolution of specific problems. The experiments show that, it is immediately after *MiniSat* in terms of number of instanced solved and execution time.

Comparison

In Figures 2.3 and 2.4 are shown the results about the comparison between *Siege* and *MiniSat*. The consideration made previously are confirmed by these graphics.

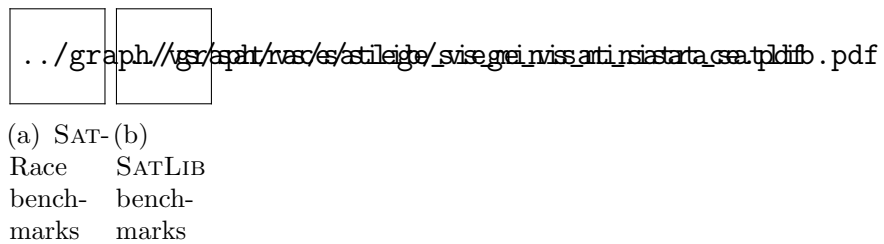


Figure 2.3: Comparision between *Siege* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

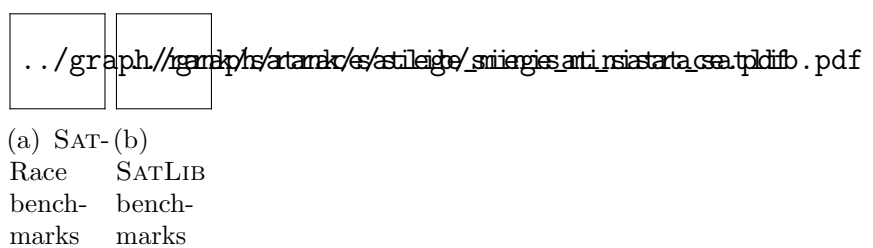


Figure 2.4: Comparision between *Siege* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

2.6 RSat

RSat is one of the last DLL-based complete SAT-solver and introduces some new techniques that we tested in our experiments with solver *StaGe*. The version used is the last: 2.0.

It is used the classic two literal watching scheme. Boolean Constraint Propagations is boosted with a heuristic for ordering implications. This mechanism helps reduce the amount of work the solver has to do to detect each conflict [36]. The heuristic is implemented with an heap structure ordered with an activity literal which is similar to that used for the decision strategy.

It utilizes conflict clause learning with conflict clause minimization technique [37] that helps reduce the size of learned clauses. These learned clauses are deleted based on their usefulness in recent resolution proofs. Learned clauses deletion is performed once the number of clauses reaches a slowly increasing limit.

RSat utilizes a lightweight component caching scheme called progress saving [10]. This component caching scheme helps reduce the amount of work repetition that is inherent in backtracking SAT-solver. Progress Saving is occasionally disabled in order to prevent caching scheme from getting stuck in a cycle of a bad assignments.

The policy for the restart used in *RSat* is largely motivated by the work in [38]. According to the policy, the limit on the number of conflicts experienced by the solver before it has to restart is set respect to the Luby's series [39].

RSat is the most recent SAT-solver that we tested. It includes some features that we have incorporated in our solver *StaGe* with some specific modifications.

2.6.1 Benchmarking

With industrial benchmarks in Table 2.11, *RSat* is the best solver. It solves 79 instances and there is a significant improvements about the CPU-time required.

The results with SATLIB benchmarks in Table 2.12 follow the results of other solvers with these benchmarks. Compared to *MiniSat*, there is a slight worsening in terms of performances, especially about the CPU-time required.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	RS	MS	RS	MS	RS
aloul (0/1)	173.2	900.0	3140.1	-	2663.4	-
een (3/3)	9.9	11.3	43.4	51.7	8.7	6.8
goldb (3/4)	1967.6	1895.4	2758.8	2489.8	587.1	256.0
grieu (3/3)	916.9	175.6	57.9	275.9	38.4	168.9
hoons (3/4)	1415.3	1307.1	219.1	74.0	177.7	57.2
manol-pipe (31/34)	5930.8	4875.9	483.6	380.1	115.3	84.6
mizh (2/3)	1118.2	1554.3	1624.7	2936.9	294.1	489.4
narain (3/3)	1741.8	1144.4	260.7	120.5	105.4	35.0
schup (2/3)	1422.8	1239.3	337.7	275.0	243.1	161.8
simon (2/4)	2269.8	2611.9	1544.9	1114.2	998.7	435.4
stric-bmc (1/1)	7.6	12.0	30.3	46.5	10.5	11.8
vange (1/1)	15.5	8.0	449.5	292.4	102.2	16.9
velev (25/34)	13486.4	10525.1	332.6	658.2	34.2	65.6
TOTAL (79/98)	30475.9	26260.5	585.1	585.6	185.2	101.7

Table 2.11: *RSat* (RS) on SAT-Race benchmarks compared to *MiniSat* (MS).

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	RS	MS	RS	MS	RS
AIS (4/4)	0.0	0.0	0.1	0.2	0.0	0.0
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	0.5	0.9	1.1	1.2	0.4	0.5
BMC (13/13)	16.2	30.2	11.3	14.3	2.9	3.9
flat (100/100)	0.1	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	2.7	1.9	45.8	32.1	24.1	17.9
ind-inference (41/41)	1.2	4.1	0.6	1.0	0.3	0.4
logistics (4/4)	0.1	0.1	2.7	1.7	0.3	0.2
parity (20/30)	9002.1	9002.9	2.0	2.4	1.8	2.3
phole (5/5)	40.9	314.4	338.4	76.9	276.5	67.4
pret (8/8)	0.0	0.1	0.0	1.9	0.0	1.3
QG (22/22)	24.6	38.8	8.1	10.6	6.7	9.0
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.0
SW100 (100/100)	0.3	0.3	0.2	0.2	0.0	0.0
TOTAL (338/352)	12688.9	12993.9	6.6	3.0	4.9	2.1

Table 2.12: *RSat* (RS) on SATLIB benchmarks compared to *MiniSat* (MS).

Comparison

In Figures 2.5 and 2.6 are shown the graphics about the comparison between *RSat* and *MiniSat*. In industrial benchmarks the CPU-time required is lower in *RSat* than in *MiniSat*. In the other benchmarks the performances are comparable.

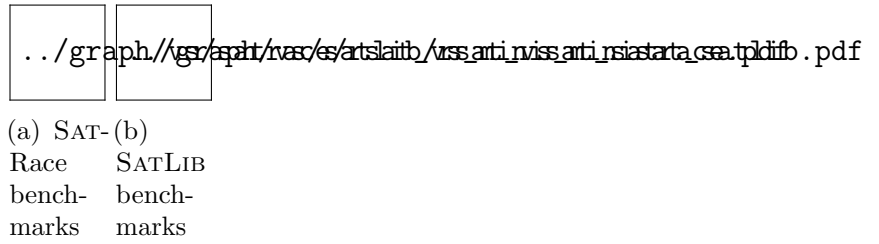


Figure 2.5: Comparison between *RSat* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

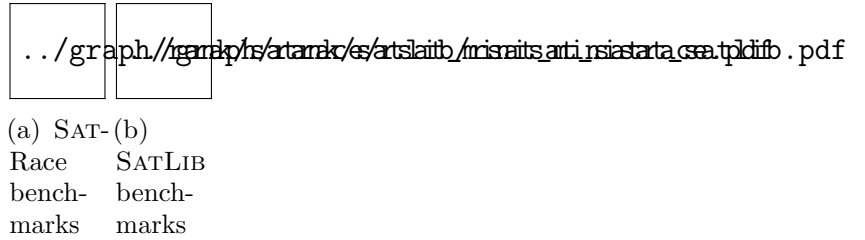


Figure 2.6: Comparison between *RSat* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

In general, in our experiments, *RSat* is the best solver in both the number of instances resolved and the execution time required.

2.7 BerkMin

His name derives from the cities where the authors live, Berkeley and Minsk. It is a modern SAT-solver, but it is less recent than *MiniSat*, *Siege*, and *RSat*. It inherits such features of *Grasp*, *Sato*, and *Chaff* as clause recording, fast BCP, restart, and conflict clause “aging”. The main techniques used in *BerkMin* are a new decision making procedure and a new method of clause database management. These features are, obviously, used in the new SAT-solver as *MiniSat*, *Siege*, and *RSat*.

The most important feature introduced are:

- New Decision Strategy
- Clause Database Management

New Decision Strategy

Each variable is assigned a counter (denote it by $ac(v)$, with v an arbitrary variable) that stores the number of clauses, responsible for at least one conflict, that have a literal of v . The value of $ac(v)$ is updated during the reverse BCP procedure. As soon as a new clause is responsible for the current conflict is encountered, the counters of the variables, whose literals are in this clause, are incremented by 1. The values of all counters are periodically divided by a small constant greater than 1. In this way the influence of “aged” clauses is decreased and the preference is given to recently deduced clauses.

Resuming, *BerkMin* has two decision making procedures. The first procedure is to pick the variables v with the maximum value of $ac(v)$ as the next branching variable. However, it is *BerkMin*’s second choice. The main decision making procedure, that was used in all our experiments, is based on chronological conflict clause ordering.

In *BerkMin*, the clause database is organized as a stack. The clauses of the initial CNF are located at the bottom of the stack and each new conflict clause is added at the top of this stack. In the process of assigning values some clauses of the stack get satisfied. There is always an unsatisfied clause that is the closest to the top of the stack: the *current top clause*.

Let C to be the current top clause. The idea of *BerkMin* is to choose the next branching variable among FREE variables of C , if it is a conflict clause. Among the variables of C , the variable v with the largest value of $ac(v)$ is selected.

In order to select the new branching variable there is, for each literal l , a cost function $lit_ac(l)$ that gives the number of conflict clauses generated so far that contain the literal l .

Initially, $lit_ac(l)$ is equally to 0. As soon as a conflict clause C is added to the database, for each literal l of C , $lit_ac(l)$ is incremented by 1. The value of $lit_ac(l)$ is not divided by a constant. If v is the next branching variable, the literal l , $l \in \{v, \neg v\}$, with the largest value of $lit_ac(l)$ is selected. If $lit_ac(v) = lit_ac(\neg v)$, literal l is selected at random.

If the current top clause is not a conflict clause, there is, for each literal l , a cost function $nb_two(l)$ that approximated the number of binary clauses in the “neighborhood” of literal l . Function $nb_two(l)$ is computed as follows.

First, the number of all binary clauses containing literal l is calculated. Then for each binary clause C containing literal l , the number of binary clauses containing literal $\neg x$, is computed where x is the other literal of C . The sum of all computed numbers gives the value of $nb_two(l)$. Given the next branch variable v , the literal l , $l \in \{v, \neg v\}$, with the greatest value of $nb_two(l)$ is selected. If $nb_two(l) = nb_two(\neg l)$, literal l is chosen at random. To reduce the amount of time spent on computing $nb_two(l)$, it is used a threshold value (default value: 100).

Clause Database Management

Before starting a new iteration, some clauses are removed from the database in order to reduce memory used for database allocation. A fraction of clauses is removed “*automatically*” due to retaining some value assignments deduced in the last iteration.

The rest of the clauses to be removed are selected with a specific heuristic. The set of conflict clauses is a queue. New conflict clause are added to the tail of the queue while clauses to be considered for removal are located in its head. The clauses are removed regarding a specific activity. This one is measured, for an arbitrary clause C , by the number of conflicts for which C is responsible so far. In the head part of the queue only clauses with activity greater then 60 are considered active. In the rest of the queue active clauses are the ones whose activity is greater than 7. Every 1024 nodes (i.e. conflicts) the threshold value of the activity for the clauses of the head part of the queue is increased. *BerkMin* uses a specific restart policy: the current search tree is abandoned after generating 550 conflict clauses.

2.7.1 Benchmarking

In our experiments, the performances of *BerkMin* are almost comparable to *MiniSat* without expensive conflict clause minimization for the number of instances solved. From the point of view of CPU-time required, *BerkMin* more time than *MiniSat*. In addition, it outperforms the performances of *zChaff*. Indeed, *BerkMin* is developed with the structure of *Chaff* as starting point.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	BM	MS	BM	MS	BM
aloul (0/1)	173.2	900.0	3140.1	-	2663.4	-
een (3/3)	9.9	60.6	43.4	171.6	8.7	10.1
goldb (2/4)	1967.6	2453.0	2758.8	2169.1	587.1	471.4
grieu (2/3)	916.9	1040.3	57.9	171.9	38.4	167.6
hoons (3/4)	1415.3	1792.4	219.1	271.2	177.7	188.5
manol-pipe (30/34)	5930.8	7476.9	483.6	785.3	115.3	105.3
mizh (0/3)	1118.2	2700.0	1624.7	-	294.1	-
narain (2/3)	1741.8	1285.9	260.7	14.3	105.4	1.9
schup (2/3)	1422.8	1658.6	337.7	532.9	243.1	399.3
simon (0/4)	2269.8	3600.0	1544.9	-	998.7	-
stric-bmc (1/1)	7.6	7.6	30.3	31.1	10.5	11.2
vange (0/1)	15.5	900.0	449.5	-	102.2	-
velev (26/34)	13486.4	12693.3	332.6	1287.2	34.2	122.0
TOTAL (71/98)	30475.9	36568.4	585.1	903.7	185.2	127.0

Table 2.13: *BerkMin* (BM) on SAT-Race benchmarks compared to *MiniSat* (MS).

In Table 2.13, the results of *BerkMin* are better than *zChaff* and slightly lower the standard version *MiniSat*.

With the SATLIB benchmarks in Table 2.14, the results are the same obtained with the other solvers and there are no substantial differences.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	BM	MS	BM	MS	BM
AIS (4/4)	0.0	0.0	0.1	0.0	0.0	0.0
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	0.5	0.8	1.1	1.1	0.4	0.6
BMC (13/13)	16.2	32.6	11.3	32.7	2.9	4.2
flat (100/100)	0.1	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	2.7	5.6	45.8	67.9	24.1	32.0
ind-inference (41/41)	1.2	0.4	0.6	0.4	0.3	0.2
logistics (4/4)	0.1	0.1	2.7	0.8	0.3	0.2
parity (20/30)	9002.1	9003.0	2.0	2.5	1.8	2.2
phole (5/5)	40.9	0.5	338.4	4.2	276.5	1.5
pret (8/8)	0.0	0.1	0.0	1.8	0.0	0.5
QG (22/22)	24.6	23.3	8.1	8.1	6.7	7.1
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.3	0.0	0.2	0.1	0.0	0.0
TOTAL (338/352)	12688.9	12666.3	6.6	2.6	4.9	1.0

Table 2.14: *BerkMin* (BM) on SATLIB benchmarks compared to *MiniSat* (MS).

Comparison

BerkMin was developed from *zChaff* SAT-solvers. Compared to *MiniSat* it solves fewer instances requiring more CPU-time.

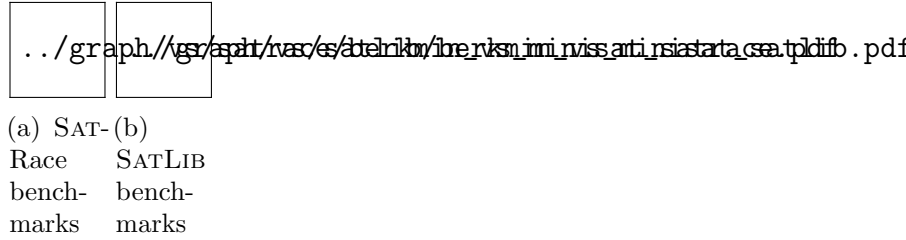


Figure 2.7: Comparison between *BerkMin* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

From Figure 2.7, it is evident that the performances of *BerkMin* are on SAT-Race, and SATLIB Instances worse than the those ones of *zChaff*.

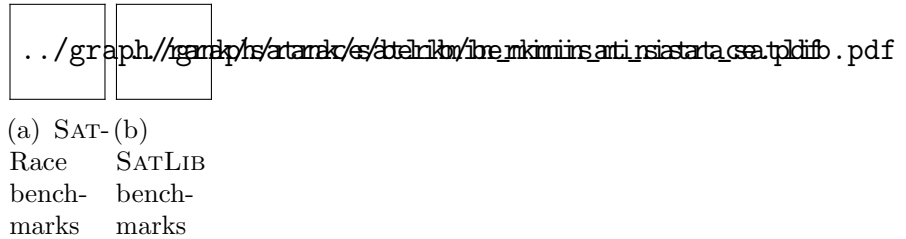


Figure 2.8: Comparison between *BerkMin* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

Instead, from Figure 2.8, *MiniSat* outperforms the performances of *BerkMin*, especially in industrial benchmarks (Figure 2.8(a)).

In conclusion, *BerkMin* is a good SAT-solver. It does not reach the performance of *MiniSat* but we must take into account that it is older than solvers like *MiniSat*, *RSat*, and *Siege*. Some techniques are modified and evolved in other modern solvers. In *StaGe*, there are some features that are the modified version of some characteristics found in *BerkMin*.

2.8 MiniMarch

The goal of this solver is to develop an alternative version of *MiniSat* less sensitive to the shuffling of the input formula. *MiniMarch* extends *MiniSat* with the following techniques:

Clause sorting In order to find conflict clauses imposing strong constraints faster and facilitate resistance against the shuffling of clauses in the input formula.

Branching heuristic *MiniMarch* combines dynamic look-ahead branching heuristic for balanced variables with static occurrence based heuristic for unbalanced variables.

Activity initialization . The activities are initialized, to make the initial variable decisions taken less sensitive to variable index shuffling.

Clause Sorting

Clauses are sorted, the first sorting criterion is the shortest size. Amongst clauses of the same length the clauses with the highest weight w_{clause} are ordered.

$$w_{clause} = \sum_{l \in c} w_{literal}(l) \quad (2.1)$$

$$w_{literal}(l) = \sum_{c(l)} \frac{1}{2^{|c|-1}} \quad c(l): \text{ all clauses of } l \quad (2.2)$$

Simplifier

The simplifier in *MiniSat* 2.0 attempts to eliminate variables to the formula. In an attempt to eliminate variable v the solver does a merge on all pair clauses with literal v occurring in one and literal $\neg v$ in the other. The result of a merge is the clause that consists of all literals in both clauses minus v and $\neg v$.

The simplifier used to call merge on the clauses from the list using the scheme in Algorithm 2.1.

Algorithm 2.1 *MiniMarch*, Simplifier procedure.

Require: variable v to eliminate

Require: Sorted set C_v of all clauses containing v

```

1: for  $i = 0$  to  $|C_v| - 1$  do
2:   for  $j = i + 1$  to  $|C_v|$  do
3:     if polarity of  $v$  in  $C_v[i] \neq$  polarity of  $v$  in  $C_v[j]$  then
4:       MERGE( $C_v[i], C_v[j]$ )
5:     end if
6:   end for
7: end for

```

Heuristic

MiniMarch combines dynamic look-ahead branching heuristic for balanced variables with static occurrence based heuristic for unbalanced variables. The balance of a variable is defined as:

$$bal(v) = \frac{w_{literal}(v) + 1}{w_{literal}(\neg v) + 1} + \frac{w_{literal}(\neg v) + 1}{w_{literal}(v) + 1} - 2 \quad (2.3)$$

A variable is regarded as balanced if it's balance is smaller then the balance of the whole formula which is defined by:

$$w_{poslit} = \sum_{l \in Vars} w_{literal}(l) \quad (2.4)$$

$$w_{neglit} = \sum_{\neg l \in Vars} w_{literal}(\neg l) \quad (2.5)$$

$$balance = \frac{w_{poslit} + 1}{w_{neglit} + 1} + \frac{w_{neglit} + 1}{w_{poslit} + 1} - 2 \quad (2.6)$$

For unbalanced variables ($bal(v) > balance$) the branching direction $d(v)$ is determined as the direction of the literal with the lowest literal weight $w_{literal}$. For balanced variables this same value $d(v)$ is determined as a first estimate to the best branching direction. It might be undefined if $w_{literal}(\neg v) = w_{literal}(v)$. Whenever the activity based branching heuristic used by *MiniSat* picks up a balanced variable the look-ahead procedure is invoked: The literal l , which the look-ahead procedure starts with, is the opposite of the literal determined by $d(v)$. This is because look-ahead has to propagate both literals and decide on the best choice. If the literal propagated second has the highest chance of being the best the chance of having to revert to the literal propagated first

Algorithm 2.2 *MiniMarch*, Look-ahead procedure.

```

1: oldLevel  $\leftarrow$  backtrackLevel
2: propQ1  $\leftarrow$  PROPAGATE(l)
3: if propQ1 = CONFLICT then
4:   return CONFLICT
5: else
6:   BACKTRACK(oldLevel)
7:   propQ2  $\leftarrow$  PROPAGATE( $\neg l$ )
8:   if propQ2  $\neq$  CONFLICT and propQ1 > propQ2 then
9:     BACKTRACK(oldLevel)
10:    PROPAGATE(l)
11:   end if
12: end if

```

goes down. If $d(v)$ has not been defined it is selected as the literal with the highest number of clause watches or if that is equal the negation of the literal of v that occurred first in the sorted set of clauses.

The variables *propQ1* and *propQ2* return a measurement indicating the “quality of the propagation”. They can be either a natural number or conflict. If no conflict was found the value is defined as:

$$\text{propQ}(v) = |1\text{UP}(v)| * \sum_{r \in 1\text{UP}(v)} m(r) \quad (2.7)$$

Where $1\text{UP}(v)$ is the set of all variables of which the value became fixed as a result of iterative unit propagation resulting of the propagation of v (including v itself). The value of $m(r)$ is the sum of the distances the watch pointers moved to the next undefined variable in a clause that was watching r .

Initialization

MiniMarch uses activity initialization, a technique that was proposed in [9]. The initial value of activity differs from the one proposed in that earlier work:

$$\text{activity}_{\text{initial}}(v) = \frac{\#_{\text{clauses}} - i_v}{\#_{\text{clauses}}} \quad (2.8)$$

With i_v the index of the clause where v occurs for the first time in the sorted set of clauses.

2.8.1 Benchmarking

In this thesis, we presented *MiniMarch* for some interesting features. In the experiments, it works fine but has some troubles with specific benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	MM	MS	MM	MS	MM
aloul (1/1)	173.2	254.4	3140.1	3360.8	2663.4	3292.6
een (3/3)	9.9	9.8	43.4	24.1	8.7	8.4
goldb (3/4)	1967.6	1622.8	2758.8	1189.0	587.1	222.3
grieu (3/3)	916.9	689.9	57.9	417.1	38.4	267.3
hoons (4/4)	1415.3	678.6	219.1	367.8	177.7	326.3
manol-pipe (31/34)	5930.8	4961.1	483.6	591.8	115.3	160.5
mizh (3/3)	1118.2	1393.3	1624.7	1974.0	294.1	548.3
narain (1/3)	1741.8	2123.2	260.7	336.3	105.4	179.3
schup (2/3)	1422.8	1077.8	337.7	344.1	243.1	220.0
simon (3/4)	2269.8	1918.7	1544.9	1352.3	998.7	1009.0
stric-bmc (1/1)	7.6	4.7	30.3	15.4	10.5	5.6
vange (1/1)	15.5	46.8	449.5	406.2	102.2	138.2
velev (19/34)	13486.4	18021.5	332.6	475.8	34.2	88.4
TOTAL (75/98)	30475.9	32802.5	585.1	647.1	185.2	242.4

Table 2.15: *MiniMarch* (MM) on SAT-Race benchmarks compared to *MiniSat* (MS).

It is reasonable to compare *MiniMarch* with the basic version of *MiniSat* expensive without conflict clause minimization. In Table 2.15, it is solved a greater number of instances but this has a negative impact on the total time required for execution.

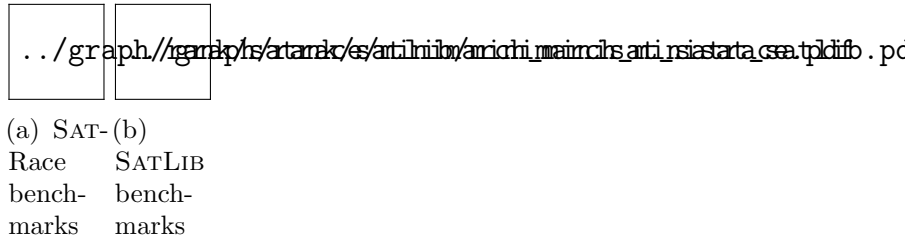
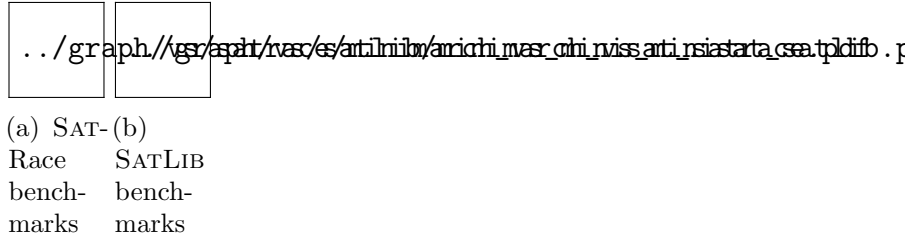
The results with SATLIB benchmarks show in Table 2.16 indicate that although it solves the same number of instances, the statistics are slightly worse than *MiniSat* particularly as regards the CPU-time required.

Comparison

In general, *MiniMarch* has the same performance than *MiniSat* without conflict clause minimization. In Figures 2.9 and 2.10 is shown the comparison between these two solvers. The CPU-time required by *MiniMarch* is higher than the one required by *MiniSat*. This means that though *MiniMarch* solves more instances, it does not increase the performances of *MiniSat*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	MM	MS	MM	MS	MM
AIS (4/4)	0.0	0.4	0.1	2.2	0.0	1.6
BF (4/4)	0.0	0.1	0.4	0.2	0.1	0.1
blocksworld (7/7)	0.5	1.6	1.1	0.5	0.4	0.4
BMC (13/13)	16.2	27.0	11.3	8.1	2.9	3.3
flat (100/100)	0.1	0.2	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	2.7	11.9	45.8	96.1	24.1	73.2
ind-inference (41/41)	1.2	22.7	0.6	6.2	0.3	2.2
logistics (4/4)	0.1	0.4	2.7	1.3	0.3	0.3
parity (20/30)	9002.1	9001.7	2.0	2.1	1.8	2.0
phole (5/5)	40.9	89.8	338.4	475.2	276.5	470.1
pret (8/8)	0.0	0.0	0.0	0.0	0.0	0.0
QG (22/22)	24.6	22.5	8.1	5.3	6.7	5.0
SSA (8/8)	0.0	0.1	0.2	0.1	0.0	0.0
SW100 (100/100)	0.3	2.1	0.2	0.1	0.0	0.0
TOTAL (338/352)	12688.9	12780.4	6.6	9.2	4.9	8.3

Table 2.16: *MiniMarch* (MM) on SATLIB benchmarks compared to *MiniSat* (MS).



From the features introduced in *MiniMarch*, we have been developed in our solver *StaGe* the look-ahead technique with a specific implementation. It also inspired the development of feature Flip Last Decision. In Chapter 4 these techniques are described in detail.

2.9 Solver Comparison

In this chapter we have described different state-of-art SAT-solvers: *zChaff*, *MiniSat*, *Siege*, *RSat*, *BerkMin*, and *MiniMarch*. Each solver has been compared with *MiniSat*. The best results, as shown in Figure 2.11, are obtained with *RSat* solver. *zChaff* is the worst SAT-solver between these ones considered.

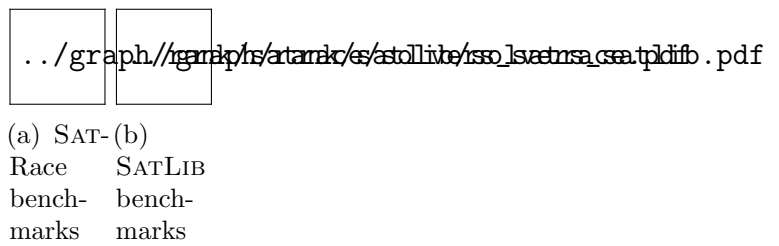


Figure 2.11: Comparison between the various solvers tested in our experiments on (a) SAT-Race and (b) SATLIB benchmarks.

Chapter 3

Solver

This chapter describes the core development of our solver *StaGe*. The focus is on the database structure, passing for to the BCP procedure to the Backtracking operation.

StaGe was developed in C++ with the object-oriented approach. Every important aspect of the solver is realized with a class. The principal part of our solver is the structure *Solver* that represents the center of the program. Each class has a method called *run()* which is used to execute the procedure

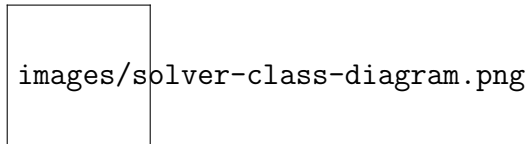


Figure 3.1: Diagram of Solver class.

related to the class itself. Accordingly, the method *run()* of *Solver* is the application entry point. In the function *main()*, an instance of the object *Solver* is allocated and then it is executed the *run()* method. The class *Solver* contains an instance of the *BCP*, *Unit*, *Heuristics* and *Backjumping* classes.

Another important aspect in this class is the *unit_run()* procedure. This function does the same operation of the *run()* method but it is performed only for the operations at decision level 0. The *run()* method is developed as an infinite loop which represents the core of the Solver. The various policies, i.e. MLF (Monotone Literal Fixing), Implicates, PS (Progress Saving), and PP (Priority Propagation), are used for initialization, setting and integration of the different features.

Algorithm 3.1 Solver, run() method.

```

1: init LHT Policy
2: init MLF Policy
3: UNIT_RUN()
4: IMPLICATES()
5: loop
6:   if is necessary flipping last decision then
7:     flip last decision
8:   else
9:     select a Free literal from HEURISTICS
10:  end if
11:  BCP()
12:  if CONFLICT then
13:    BACKJUMPING()
14:    add the new CONFLICT CLAUSE to DATABASE
15:    if SIZE of new CONFLICT CLAUSE = 1 then
16:      UNIT_RUN()
17:    end if
18:  end if
19:  update variable and clause activity
20:  RESTART Policy
21:  REMOVE LEARNED CLAUSE Policy
22:  IMPLICATE Policy
23:  MLF Policy
24:  FLD Policy
25:  PS Policy
26:  PP Policy
27: end loop

```

As previously mentioned, the first method called is *unit_run()* executed only at decision level 0, when the stack of the assignments is empty. The reason for two different implementations for the propagation, BCP and *Unit*, is that the assignments at decision level 0 are guaranteed and it is possible to remove some clauses and literals permanently from the formula in order to reduce the used memory and make the access time to read the formula itself faster. So we define these assignments as “*permanent*”. The procedure

Algorithm 3.2 Solver, code of *unit_run()* method.

```

1: if MLF enabled then
2:   repeat
3:     unit.RUN()
4:     mlf.RUN_4_UNIT()
5:   until there is a another variable to assign
6: else
7:   unit.RUN()
8: end if

```

unit_run() in Algorithm 3.2 calls the entry point method *run()* of the instance *unit*. It manages only a different approach when MLF is enabled or when it is not active. After this one, it is executed the method to make the implicate graph for the binary clauses. Considering this aspect, it was a good choice to use a separate structure to contain only the binary clauses (like in *Siege*). The structure for the binary clauses is described in the following Database section.

The program ends if it is found a solution or if the problem is unsatisfiable. When there is or not a solution the method *result()* of Database class is called. This function launches the *exit()* standard method. In this way each class can ends the program without the need to return a specific value that requires the management by the parent classes.

The first conditional if in the loop manages the FLD feature in order to choose the next FREE variable. To select a FREE literal is used the *get()* method stored in the Heuristics class. Now, it is necessary to propagate the chosen literal and, for this reason, it is called the BCP operation. In our solver *StaGe* the clauses are separated in two structures: *binaries* and *N-aries*. The BCP class manages the search with two different approaches that represent the *Priority Propagation* (PP) feature. This is discussed in the BCP section. It is important to know that the BCP procedure returns if there is a *conflict*

or not. If there is an empty clause it is necessary to call the *Backjumping* procedure. *StaGe* uses the classic Conflict-driven clause learning introduced in [23]. The learned conflict clause is added to the Database. If the conflict clause is a permanent assignment (i.e. *unit clause*, a clause with only 1 literal) it is necessary to call the unit propagation with the *run()* method of the Unit class. The management of the conflict clause ends with the update of the variable and clause activities used for the Remove Clause policy.

The last part of the loop contains the management of the different strategies which are realized with a common specific structure, reported in Algorithm 3.3. The Restart policy is performed as an alternative to the Remove Learned

Algorithm 3.3 Solver, common structure for Policy management.

```

1: if condition for policy management then
2:   manage policy
3: end if

```

Clause, Implicate and MLF policies. These last policies lose their effectiveness if they are executed immediately after a restart with the assignment stack empty. The restart procedure is executed when the number of conflicts from

Algorithm 3.4 Solver, Restart Policy.

```

1: if  $db.num\_conflicts\_from\_restart \geq db.restart\_limit$  then
2:    $db.num\_conflicts\_from\_restart \leftarrow 0$ 
3:    $INC(db.num\_restarts)$ 
4:    $db.restart\_limit \leftarrow db.restart\_limit \times db.restart\_inc$ 
5:    $db.learnts\_limit \leftarrow db.learnts\_limit \times db.learnts\_inc$ 
6:    $db.decision\_level \leftarrow 0$ 
7:    $db.CANCEL()$ 
8:    $db.lht\_decide \leftarrow \text{false}$ 
9: else
10:  MLF Policy
11:  IMPLICATE Policy
12:  REMOVE LEARNED CLAUSE Policy
13: end if

```

the last restart is greater or equal to the chosen limit. The initial value of $db.restart_limit$ is 100 and the increment $db.restart_inc$ is 1.5. Important is that also the limit for the learned clauses is incremented ($db.learnts_limit$ initially equals to $N\text{-ary clauses}/3$ and $db.learnts_inc$ equals to 1.1). This

approach follows the idea of *MiniSat* and *RSat*. When a restart is executed, a new search tree, called iteration, is started. Iteration after iteration, the limits for the number of conflicts and the learned clauses are increased. In this way it is possible to increase the probability of finding the solution in the current search tree.

In Algorithm 3.5 the Remove Learned Clauses, Implicate, and MLF policies are shown. The Remove Learned Clauses procedure is the method introduced in *MiniSat*. The condition is that the number of the learned clauses is greater than the specific limit. If this condition is TRUE the *db.reduce_learnt()* method is executed. The Implicate and MLF policies have the same structure with

Algorithm 3.5 Solver, Remove Learned Clause, Implicate, and MLF Policies.

```

1: REMOVE LEARNED CLAUSE Policy
2: if db.learnts.SIZE()  $\geq$  db.learnts_limit then
3:   db.REDUCE_LEARNT()
4: end if
5: IMPLICATE Policy
6: if IMPLICATE enabled and
   db.num_conflicts_from_implicate  $\geq$  db.implicate_limit then
7:   implicate.RUN()
8:   db.num_conflicts_from_implicate  $\leftarrow$  0
9:   db.implicate_limit  $\leftarrow$  db.implicate_limit  $\times$  db.implicate_inc
10: end if
11: MLF Policy
12: if MLF enabled and db.num_conflicts_from_mlf  $\geq$  db.mlf_limit then
13:   mlf.RUN()
14:   db.num_conflicts_from_mlf  $\leftarrow$  0
15:   db.mlf_limit  $\leftarrow$  db.mlf_limit  $\times$  db.implicate_inc
16: end if

```

a similar condition but with different parameters. These procedures follow the same concept of the Restart policy. The unit of measurement used is the number of conflicts. The class member representing the number of conflicts is increased progressively in order to reduce the computational burden for the continuing calls to these complex procedures. The last policies, shown in Algorithm 3.7, has the same construct characterized by a condition with the classic limit and a change of the status. Each policy changes the enable option. One time the feature is enable, one time is disabled, and so on. The used limits differ on the basis of the enable status. For this feature the unit

Algorithm 3.6 Solver, last Policies.

```

1: if FLD enabled and  $\text{db.num\_conflicts\_from\_fld} \geq \text{db.fld\_limit}$  then
2:   if  $\text{db.fld\_on}$  then
3:      $\text{db.fld\_limit} \leftarrow \text{db.fld\_limit\_on}$ 
4:   else
5:      $\text{db.fld\_limit} \leftarrow \text{db.fld\_limit\_off}$ 
6:   end if
7:    $\text{db.fld\_on} \leftarrow \neg \text{db.fld\_on}$ 
8:    $\text{db.num\_conflicts\_from\_fld} \leftarrow 0$ 
9: end if
10: if PS enabled and  $\text{db.num\_conflicts\_from\_ps} \geq \text{db.ps\_limit}$  then
11:   if  $\text{db.ps\_on}$  then
12:      $\text{db.ps\_limit} \leftarrow \text{db.ps\_limit\_on}$ 
13:   else
14:      $\text{db.ps\_limit} \leftarrow \text{db.ps\_limit\_off}$ 
15:   end if
16:    $\text{db.ps\_on} \leftarrow \neg \text{db.ps\_on}$ 
17:    $\text{db.num\_conflicts\_from\_ps} \leftarrow 0$ 
18: end if
19: if PP enabled and  $\text{db.num\_conflicts\_from\_pp} \geq \text{db.pp\_limit}$  then
20:   if  $\text{db.pp\_on}$  then
21:      $\text{db.pp\_limit} \leftarrow \text{db.pp\_limit\_on}$ 
22:   else
23:      $\text{db.pp\_limit} \leftarrow \text{db.pp\_limit\_off}$ 
24:   end if
25:    $\text{db.pp\_on} \leftarrow \neg \text{db.pp\_on}$ 
26:    $\text{db.num\_conflicts\_from\_pp} \leftarrow 0$ 
27: end if

```

of measurement is the number of conflicts too. As shown in the algorithms,

Algorithm 3.7 Solver, principal policy parameters.

```

1: {MLF Parameters}
2:  $mlf\_limit \leftarrow 100$ 
3:  $mlf\_inc \leftarrow 1.1$ 
4: {IMPLICATE Parameters}
5:  $implicate\_limit \leftarrow 150$ 
6:  $implicate\_inc \leftarrow 1.75$ 
7: {FLD Parameters}
8:  $fld\_limit\_off \leftarrow 200$ 
9:  $fld\_limit\_on \leftarrow 100$ 
10: {PS Parameters}
11:  $ps\_limit\_off \leftarrow 400$ 
12:  $ps\_limit\_on \leftarrow 100$ 
13: {PP Parameters}
14:  $pp\_limit\_off \leftarrow 250$ 
15:  $pp\_limit\_on \leftarrow 150$ 

```

in each policy it is uses the *db* object that represents the global instance of the Database class. It is accessible from all classes, including Solver itself.

3.1 Database

Now, it is described the structure of Database that stores all the information used by all classes of the solver *StaGe*. All the attributes and methods are *public*; in this way they are accessible from every external class. To this purpose it is allocated the global object *db*, already described at the end of the previous section.

In Figure 3.2, are shown the main characteristics of Database. First of all, it is important to remember that the type *vector* is our implementation inspired by the *MiniSat* version. The vector class gives the access like a

images/database-class-diagram.png

Figure 3.2: Diagram of Database class.

standard C++ array (with square parentheses, []). The reason for own development is that the STL¹ vector is less compact and fast. Indeed it is included in the STL core with different levels of abstraction. The most important methods of our vector release are *erase()* and *reserve()*. The classic *push_back()* method is invoked to add a new element in the last position of the vector.

Algorithm 3.8 Vector, *erase()* method.

Require: *pos*, vector index to erase

- 1: $m_data[pos] \leftarrow m_data[m_size]$
 - 2: $DEC(m_size)$
-

Algorithm 3.9 Vector, *reserve()* method.

Require: new vector capacity *new_cap*

- 1: **if** *new_cap* > *m_capacity* **then**
 - 2: *m_capacity* $\leftarrow$ *new_cap*
 - 3: reallocation of *m_data* with the new capacity
 - 4: **end if**
-

With the *erase()* method it is possible to remove an element in position *pos* with only 1 access. It does not run the shift of all elements following that one to remove. Instead it moves the last element in the position *pos* and it decreases of 1 the vector size. The memory is not released and the capacity² of the vector remains the same one.

The *reserve()* method lets to allocate the memory space necessary for the vector. This is useful for structures of which are known the size. In this way only one time is allocated the space required. In this function the capacity property is updated.

The Database class contains the structures used to store the formula. For each literal there is a vector of literals which represents the other ones

¹The Standard Template Library (STL) is a software library partially included in the C++ Standard Library. It provides containers, iterators, algorithms, and functors. More specifically, the C++ Standard Library is based on the STL published by SGI (<http://www.sgi.com/>). Both include some features not found in the other. The STL of SGI is rigidly specified as a set of headers, while ISO C++ does not specify header content, and allows implementation either in the headers, or in a true library.

²The capacity is a property of the vector which represents the amount of memory allocated for the vector itself.

presented in the binary clauses. For example, if there are these three binary clauses: $[A, B]$, $[B, C]$, $[\neg C, D]$ the vectors stored in Database are:

A	(B)
B	(A, C)
C	(B)
$\neg C$	(D)
D	($\neg C$)

Table 3.1: Database, stored vectors.

So, for each binary clause, there are two representations, one for literal. This structure, presented in Database, is called *binaries*. It consists of a vector of *unsigned int*. Furthermore, the literal is represented with a positive number in this way: an even number is the positive literal, the following odd one is his negation; for example, 0 is the positive literal and 1 is his corresponding negation. It is simple to know the negation of a literal: it is enough to make a XOR bit-wise operation between the literal and the number 1.

Instead the N -ary clauses are stored according different considerations. It is used a vector of type *Clause* called *clauses*. The Clause class is implemented as an extension from the version introduced in *MiniSat*. This structure greatly simplifies the programming solver because it allows a high level of abstraction. It is possible to access each literal of a clause as a normal C++ vector. It incorporates the necessary information related to the specific clause as if the clause is learned or the clause activity (used for the *Remove Learned Clauses* strategy). The structure *learnt* stores the learned clauses. For this type of clauses it is used a specific structure in order to reduce the problems arising from the consumption of memory.

Summing up, all the structures are shown in Table 3.2.

The status of each literal is represented with three macros: *VAR_FREE*, *VAR_TRUE*, and *VAR_FALSE* defined in the header *Definitions.h*. A *permanent literal* is defined as a variable that was assigned at *decision level* 0 with the *Unit* propagation. The *literal_watch* vector is described in the section of the TLW scheme. This vector contains the pointers to the clauses that

vector<uint> binaries	structure to store binary clauses.
char assign	status of literal (FREE, TRUE, FALSE).
char * unit_assign	status of permanent literal (FREE, TRUE, FALSE).
vector<Clause> literal_watch	used in TLW scheme.

Table 3.2: Database, structure index by literal.

the literal “*watches*”. So, during the BCP, for each literal it is not necessary scanning every clause but only those ones presented in this vector.

Instead the structures index by variable are shown in Table 3.3. A variable is computed from a literal with the macro defined in Definitions.h: $\text{VAR}(lit) = lit/2$. The class *Reason* is a wrapper of the Clause class. It

Reason reason	gets the reason for the assigned variable. It is NULL if the variable is FREE.
uint level	indicates the decision level of the assigned variable.
bool seen	implements the <i>Backjumping</i> procedure.
double var_score	score variable.

Table 3.3: Database, structure index by variable.

contains a pointer to the clause that represents the reason for a specific variable assignment. This class is implemented in this way because of a problem with the binary clauses, where the reason is only one literal (*integer* type) and not a clause class. To this purpose in the *Reason* class it is allocated a special clause, *memory_4_binary*, that represents the binary reason.

Algorithm 3.10 Reason, unset() method.

1: *reason* $\leftarrow$ NULL

The *Reason* structure is used in the *Backjumping* feature. The member *level* represents the current decision level of the assigned variable. When a variable is FREE, this information indicates the last decision level where the variable was assigned. When a variable is released its decision level is not updated.

Algorithm 3.11 Reason, set() method.

Require: c , pointer to clause

1: $reason \leftarrow c$

Algorithm 3.12 Reason, set() method for binary clauses.

Require: literal l to set

Require: reason r of the assignment

1: $memory_4_binary[0] \leftarrow l$

2: $memory_4_binary[1] \leftarrow r$

3: $reason \leftarrow memory_4_binary$

In Database there are a lot of members used for statistics. In Table 3.4 there is a summary of these informations. In addition to these members Database uses other different ones to solve the instances. First of all, the most important one is the stack implemented with the vector class. There are two different stacks. The first one, standard, is used for the assigned variables, and the second one is for the unit assignments at decision level 0. The *has_conflict* data member is used in the Solver class as result for the BCP operation. The *variable_order* member is used by the Heuristics class and it is discussed in the relative section. It is implemented with a *Heap* based on the version developed by *RSat*. It allows to balance two conflicting needs: the necessity to easily obtain the new chosen variable, trying to reduce the computational time required to manage the operations performed on the heap itself. The other variables are connected to the specific feature and they are described in the relative sections. In addition, Database incorporates important public methods used by different classes. The *read()* method parses the input formula and saves it in Database structure. The *Score Initialization* method is called by this function.

There are four methods to set the variables: *set_unit()*, *set_decision()*, *set_mlf()*, and *set()*. The last one is the general method, used by other functions to perform their operations. It adds the chosen literal, from Heuristics, in the tail of *stack_assign* structure. Then it assigns the positive and negative literals and the decision level of the selected variable. The *set_decision()* method uses this *set()* method and, in addition, it performs some operations, as setting *has_conflict*, and *has_propagated* variables to FALSE. Instead, as shown in Algorithm 3.14, the *set_unit()* method uses a different approach. It sets as “seen” the literals and adds these ones in the relative stack. The use

uint num_variables	number of variables.
uint num_unit_variables	number of permanent variables.
uint num_literals	number of literals.
uint num_clauses	number of clauses.
uint num_unary_clauses	number of unit clauses.
uint num_binary_clauses	number of binary clauses.
uint lits_nary_clauses	number of literals in nary clauses.
uint num_learnt_clauses	number of learned clauses.
uint num_learnt_unary_clauses	number of unit learned clauses.
uint num_learnt_binary_clauses	number of binary learned clauses.
uint lits_learnt_nary_clauses	number of N -ary learned clauses.
uint num_decisions	number of decisions.
uint num_conflicts	number of conflicts.
uint num_conflicts_from_restart	number of conflicts from last restart.
uint num_binary_conflicts	number of binary conflicts.
uint num_nary_conflicts	number of N -ary conflicts.
uint num_learnt_conflicts	number of learned conflicts.
uint num_backjumps	number of backjumps. The unit used for the measurement is the number of decision levels.
uint num_ccmin	number of conflict clause minimizations. The unit of measurements used is the number of literals deleted from the conflict clauses.
uint num_restarts	number of restarts.
uint num_BCPs	number of BCP propagations in terms of number of assigned variables.

Table 3.4: Database, statistic variables.

bool has_conflict	indicates if it was found a conflict.
vector<uint> stack_assign	contains the assigned variables.
vector<uint> stack_unit	contains the permanent assigned variables.
Heap variable_order	used by <i>Heuristics</i> to select the next FREE variable.

Table 3.5: Database, other data members.

read (<i>file, input, _output</i>)	reads formula input file.
set_index ()	performs BCP operation.
si ()	Score Initialization method.
set_unit (<i>lit</i>)	assigns the literal TRUE in the unit propagation.
set_decision (<i>lit</i>)	sets the decision literal TRUE.
set_mlf (<i>lit, is_with_unit</i>)	used by MLF to assign the pure literal TRUE.
set (<i>lit</i>)	sets the literal TRUE.
cancel ()	cancels the assigned variables with a decision level greater than the current one.
add_binary_clause ()	adds the new binary conflict clause to Database.
add_nary_clauses ()	adds the new N -ary conflict clause to Database.
var_update (<i>clause</i>)	updates the variable score. Executed in HEURISTICS.
clause_update (<i>clause</i>)	updates the clause activity Used for Remove Learned Clause strategy.
remove_clause (<i>clause</i>)	removes from the memory the clauses that are satisfied in unit propagation.
locked (<i>clause</i>)	indicates if a clause is a reason of some variable. Used for Remove Learned Clauses strategy.
reduce_learnt ()	Removes Learned Clauses strategy.
decay ()	updates the increments of scores for variables and clauses.
inc_conflict ()	increases number of conflicts variable
fld ()	runs FLD feature.
result (<i>result_type</i>)	ends the program.
print_stats ()	prints all stats.

Table 3.6: Database, public methods.

Algorithm 3.13 Database, `set()` method.

Require: literal l to assign

- 1: $stack_assign \leftarrow l$
 - 2: $assign[l] \leftarrow \text{VAR_TRUE}$
 - 3: $assign[\neg l] \leftarrow \text{VAR_FALSE}$
 - 4: $level[\text{VAR}(l)] \leftarrow decision_level$
-

Algorithm 3.14 Database, `set_unit()` method.

Require: literal l to assign as unit

- 1: $assign[l] \leftarrow \text{VAR_TRUE}$
 - 2: $assign[\neg l] \leftarrow \text{VAR_FALSE}$
 - 3: $seen[\text{VAR}(l)] \leftarrow \text{true}$
 - 4: $stack_unit \leftarrow l$
 - 5: $\text{INC}(num_unit_variables)$
-

of this structure is described in the *Unit* section.

To unset a variable it is used the *cancel()* method. It does not unset a specific variable but all those ones with a decision level greater than the current one. It returns the last literal removed from the stack. When it is

Algorithm 3.15 Database, `cancel()` method.

Require: $index_stack$, index last assigned variable in stack

- 1: $l \leftarrow stack_assign[index_stack]$
 - 2: **while** $stack$ is not empty and decision level of $l >$ current decision level **do**
 - 3: $stack_assign \Rightarrow_{index_stack} l$
 - 4: $assign[l] \leftarrow \text{VAR_FREE}$
 - 5: $assign[\neg l] \leftarrow \text{VAR_FREE}$
 - 6: ...
 - 7: $variable_order.PUSH(\text{VAR}(l))$
 - 8: ...
 - 9: $DEC(index_stack)$
 - 10: **end while**
 - 11: $stack_assign.RESIZE(index_stack + 1)$
-

performed the Backjumping procedure a new conflict clause is created and it is necessary to add it in the Database. To this purpose two complementary methods are implemented: *add_binary_clause()* and *add_nary_clause()*.

Each function uses the data member *conflict_clause* that is a Clause object

Algorithm 3.16 Database, add_binary_clause method.

- 1: $binaries[conflict_clause[0]] \leftarrow conflict_clause[1]$
 - 2: $binaries[conflict_clause[1]] \leftarrow conflict_clause[0]$
 - 3: $INC(num_learnt_binary_clauses)$
-

Algorithm 3.17 Database, add_nary_clause method.

- 1: call the static method $ADD()$ of $CLAUSE$ to create a new N -ary clause c
 - 2: $literal_watch[conflict_clause[0]] \leftarrow c$
 - 3: $literal_watch[conflict_clause[1]] \leftarrow c$
 - 4: $learnts \leftarrow c$
 - 5: $CLAUSE_UPDATE(c)$
 - 6: $lits_learnt_nary_clauses \leftarrow$
 $lits_learnt_nary_clauses + conflict_clause.SIZE()$
 - 7: **return** c
-

stored in the Database class. It is used to save the new conflict clause found. If the new learned clause is binary then it is added to the binary structure. In this way the learned binary clauses are not removed. Only the learned N -ary clauses are checked by the remove clauses strategy. The reason of this choice is that the number of the learned binary clauses are not so high and are not memory expansive.

The methods $var_update()$ and $clause_update()$ making current, respectively, the variable score and the clause activity. The first procedure orders the $variable_order$ heap and the second one is used to select the learned clauses to remove.

The $remove_clause()$ method deletes from Database and from the memory the clauses stored. It removes the learned N -ary clauses and the ones that are satisfied in the unit propagation (decision level 0). It is an important operation useful to monitor the used memory especially for problems with a high number of clauses.

To remove the learned clauses is used the $reduce_learnt()$ method. His logic follows the ones used in *MiniSat*, and in *RSat*. The learned clauses are sorted with the $reduce_learnt_lt$ struct: the clauses with the highest activities are in the tail of the vector. The limit variable is computed as in *MiniSat* with the $clause_inc$ member periodically updated in the $decay()$ method. For the first half of the learned clauses (i.e. the clauses with the lowest activities) is watched if the clause is locked or not. It consists to check if the learned clause is the reason of an assignment. If the clause is not the reason it is

possible to remove it. For the second half of the learned clauses there is another condition in addition to the one just mentioned: the activity of the learned clause has to be lower than the limit, above described. Considering

Algorithm 3.18 Database, `reduce_learnt()` method.

```

1: limit  $\leftarrow$  clause_inc / number of learned clauses
2: sort the learned clauses
3: for all first half of learned clauses lc do
4:   if  $\neg$  LOCKED(lc) then
5:     REMOVE_CLAUSE(lc)
6:   end if
7: end for
8: for all second half of learned clauses lc do
9:   if  $\neg$  LOCKED(lc) and activity of lc < limit then
10:    REMOVE_CLAUSE(lc)
11:   end if
12: end for

```

that the limit is periodically increased, the procedure to remove the learned clauses, iteration by iteration, deletes more and more learned clauses. This implementation tries to reduce the memory leak due to the fact that the size of the learned clauses is generally high (in our implementation for each conflict only one clause is learned).

The last important method is `result()` that ends the program. His parameter is an integer number that represents the solution (*RESULT_SAT*: 10; *RESULT_UNSAT*: 20, by the rules of the *SAT-Race* and *SAT-Competition*). It calls the `print_stats()` method and writes to file the solution with the correct assignments.

3.1.1 Parser

In this section are described the procedures required to read the formula from input file. In addition it includes the techniques introduced in *MiniSat*, and *RSat* about variable elimination style implementation.

The `read()` function is a method of Database class and it uses some structures developed in *MiniSat*. These ones are the *StreamBuffer* class and the functions defined in file *Loader.h*. The *StreamBuffer* structure is the same developed in *MiniSat* and, for this reason, it not so interesting to analyse. It provides an abstraction that is useful to read the formula. In *Loader.h* there

are some functions that are subsequently described.

The input instance respects the following rules. It begins with the line:

p cnf <number of variables> <number of clauses>

Then after this header, the clauses are represented with a list of numbers. They finish with the character 0. So a positive literal is represented with a positive number and a negative one with a negative number. A comment line starts with the letter “c”. The number of clauses is not read from the input because it is not necessary. Indeed when a new clause is read it is added to the specific vector (*binaries* or *clauses*) that allocates automatically the memory.

The *skipWhiteSpace()* function, defined in *Loader.h*, ignores the white space in the file. Instead the *readClause()* function maps the literals from the file pattern to the *StaGe* one. Indeed in our solver the literals start from 0 and are positive. So the literal 1 is mapped with 0, -1 with 1, and so on³.

The *skipLine()* method ignores a comment line that starts with the letter “c”. In addition, this method ignores any other line post header that begins with “p”.

The most important procedure is implemented to remove the duplicate variables from the clause or ignoring the ones that are trivially satisfied. First of all the clause is sorted. If a clause contains duplicate literals then they are placed close. For each literal it is checked if the literal is the negation of the previous one. If it is true then the clause is trivially satisfied and it is possible to ignore it. There is another case when it is possible to ignore the clause: a literal is assigned to TRUE. This is possible because when it is read a unit clause it is executed the corresponding unit propagation. The literal is deleted from the clause when it is assigned to FALSE or it is equal to the previous one instead. From the viewpoint of the computational time it is not a problem to call the sort procedure for each clause. Indeed these calls are executed before the search only one time and the size of the clauses are not so relevant. To sort a clause is used the standard *quicksort* function⁴. The *si()*, *Score Initialization*, function is called when the reading is finished. It follows the idea of the *MiniMarch* solver and it is described in the specific section. At the end of the function, are printed some stats about number of

³This method to decode the literals is based on the one introduced in *Qube* (<http://www.star.dist.unige.it/~qube/>).

⁴For a short description of this algorithm see <http://en.wikipedia.org/wiki/Quicksort>.

Algorithm 3.19 Database, read() method.

Require: STREAMBUFFER *buf*

Require: vector of literal *lits*

```

1: lits.RESERVE(VECTOR_CAPACITY)
2: loop
3:   SKIPWHITESPACE(buf)
4:   if *buf = EOF then {if it is at the end of file}
5:     exit from loop
6:   else if *buf = 'p' then
7:     if MATCH(buf, "p cnf") then
8:       set number or variables, literals and ignore the number of clauses.
9:       memory allocation of structures for literals, variables, clauses, and
       other specific features.
10:    else
11:      print error and term program
12:    end if
13:  else if *buf = 'c' or *buf = 'p' then
14:    SKIPLINE(buf)
15:  else
16:    READCLAUSE(buf, lits)
17:    check for duplicate variables or useless clause c.
18:    if  $\neg$  ignore_clause then
19:      if SIZE of clause c > 1 then
20:        add c to the database
21:      else
22:        launch unit propagation.
23:      end if
24:    end if
25:  end if
26: end loop
27: runs SCORE INITIALIZATION
28: prints formula stats

```

Algorithm 3.20 read() function, check redundancy in the formula.

```

1: sort read clause
2: check if first literal of clause  $c$  if Free
3: for all literal  $l$  in  $c$  do
4:   if  $l$  is true or  $l$  = negation of previous then
5:     ignore  $c$ 
6:   else if  $l$  is false or  $l$  = previous literal of  $c$  then
7:     delete  $l$  from  $c$ 
8:   end if
9: end for

```

variables, literals, clauses, parsing time, and used memory. To compute this last statistic is used the function developed in *MiniSat*.

StaGe is a command line program with some options which let to customize the resolution with different features. The possible chooses are:

-t, -timeout <num>	sets the timeout, in seconds.
-p, -print <output-file>	prints the solution found and the correct assignments.
-i, -implicate	enables the IMPLICATE feature.
-m, -mlf <always — online>	enables the MONOTONE LITERAL FIXING feature.
-f, -fld <conf — prop>	enables the FLIP LAST DECISION feature.
-s, -ps	enables the PROGRESS SAVING feature.
-a, -si	enables the SCORE INITIALIZATION feature.
-c, -ec	enables the EXPENSIVE CONFLICT CLAUSE MINIMIZATION.
-o, -lht	enables the LOOKAHEADS technique.
-v, -stats	prints detailed statistics.

Table 3.7: *StaGe*, command line options.

In this way it is possible to test easily the features with different interaction in order to understand the techniques that outperform the performance of the solver and what combinations of these ones are the best solutions. Obviously, these considerations are empirical because they derive from the experiments done and there is not a mathematical or logic demonstration.

3.2 Heuristics

The decision heuristic is a crucial factor influencing the performances of the modern complete SAT-solvers based on the DPLL algorithm. This heuristic decides which variable to choose at each decision point during the search and what value to assign it first.

In recent years, the field has been a witness to breakthrough in the design of decision heuristics that are empirically successful on real-world industrial SAT instances. The key observation was that the decision heuristic must be dynamic, that is, it must re-focus the search on recently learned conflict clauses.

In Chapter 2 are described different heuristics of the most important SAT-solvers. The first dynamic heuristic is VSIDS [33], that maintains a score for each literal. As it is described in the *zChaff* section, this score is incremented when the literal appears in a conflict clause. Periodically, the scores are halved. This strategy ensures that the prover picks literals that were involved in the derivation of recent conflict clauses. The other important discussed heuristic is developed in the *BerkMin* solver [34]. For the authors, VSIDS is not sufficiently dynamic, in the sense that it may still pick literals that are irrelevant to the currently explored branch. Instead they proposed organizing all conflict clauses in a list and picking the next decision literal from the top-most unsatisfied clause on the list. Our implementation was

images/heuristics-class-diagram.png

Figure 3.3: Diagram of Heuristics class.

derived from the work of *MiniSat* and *RSat*. *MiniSat* extends the classic VSIDS technique, described above. The variables are ordering based on their activity. Unlike the original VSIDS, it does not imposes an order on literals, it make no distinction between p and $\neg p$. Each variable has an *activity* attached to it. Every time a variable occurs in a recorded conflict clause, its activity is increased. This operation is know as *bumping*. After recording the conflict, the activity of all the variables in the system are multiplied by a constant less than 1, thus *decaying* the activity of the variables over time. Recent increments count more than old. The current sum determines the activity of

the variable.

RSat uses the same concept of *MiniSat* but it is implemented in a different way. In addition, it uses a Progress Saving feature, that is natively integrated in the program.

StaGe uses the implementation adopted in *RSat* with a priority heap and the concept introduced in *MiniSat*. This choice is derived from the reason that the implementation of *RSat* is more compact and optimized to be run faster. In the *read()* Database method it is called the *init()* function of the

Heap variable_order	ordered list of variables.
double * var_score	activity variable.
double var_score_inc	increases the activity value.
double var_score_decay	decreases the activity value.

Table 3.8: Database, data members used by Heuristics.

Heap class. The number of variables is used to set the size and capacity of

Algorithm 3.21 Heap, *init()* method.

- 1: set SIZE, CAPACITY, SCORE
 - 2: memory allocation for private data members of HEURISTICS
 - 3: insert variables in HEAP
-

the heap. The variables are inserted in the heap ordered by index and not by activity. With the *Score Initialization* feature it is possible to sort the variables in the heap considering their activities.

The main method of the Heuristics class is the *get()* procedure. It gets the variable from the top of the heap. This extracted variable is always a FREE variable. Indeed the method to pop an element from the heap replaces the chosen element with the one with the highest activity. If the heap is empty it means that the SAT instance is satisfiable because all the variables are assigned and there is not a conflict. Periodically, a random selection is made.

The literal is chosen randomly. This approach is used in *MiniSat*. In *RSat* the selected literal is always the positive one. We use the *MiniSat* choice because we think that there is no reason to select always the positive (or negative) literal. The variable activities are updated in the *Backjumping* procedure with the method *var_update()* stored in *Database*. The activity variable is incremented with the *var_score_inc* member. If the variable to

Algorithm 3.22 Heuristics, get() method.

```

1: INC(number of decisions)
2: if random selection then
3:   select a random variable
4: end if
5: while chosen variable is not FREE do
6:   if HEAP is empty then
7:     db.RESULT(RESULT_SAT)
8:   end if
9:   db.variable_order.POP()
10: end while
11: if PS enabled then
12:   PS selection literal
13: else
14:   select random literal
15: end if
16: if IMPLICATE enabled then
17:   IMPLICATE selection literal
18: end if

```

Algorithm 3.23 Database, var_update method.

Require: variable, v

```

1: increment  $var\_score[v]$  with  $var\_score\_inc$ 
2: if  $var\_score[v] < 1^{100}$  then
3:   for all variable  $s$  do
4:      $var\_score[s] \leftarrow var\_score[v] / 1^{-100}$ 
5:   end for
6:   update the position of  $v$  in HEAP
7: end if

```

update has an activity greater then 1^{100} each variable activity is multiplied by 1^{-100} . Then the position of the variable in the heap is updated.

The value of *var_score_inc* is initially 1. It is periodically incremented with the *var_score_decay* constant equal to $1/0.95$. This operation is executed in the *decay()* Database method which was called in the *run()* method of the Solver class. The *decay()* method is the last procedure called when a conflict is found.

3.3 BCP

As first emphasised in the *Chaff* literature, typically about of 90 percent of the run-time of a clause-learning DLL-solver is spent performing BCP [33]. The BCP procedure is executed frequently, every time a variable is assigned a truth value.

The modern SAT-solvers perform this procedure in different ways. *Siege* is oriented to reduce the memory accesses. For his author BCP operates both broadly and non-sequentially over a data structure that is normally many times larger than the L2 cache of the host computer. So this diffuse access pattern means data cache is frequently missed. Resuming, the extraordinary cost of BCP stems from a memory latency problem. The data structure is divided in binary, ternary and N -ary clauses.

Instead *MiniSat* is implemented with a different approach. The clauses are stored in the same structure and they are not divided. Indeed the binary clauses are represented by storing the literals to be propagated in the watcher list. For the author of *MiniSat* the propagation of all binary clauses before or after the bigger clauses affects the implication graph and seems producing worse conflict clauses. At the reality, as described in Chapter 2, our experiments show that *Siege* works fine with some kind of benchmarks. It means that propagate the binary clauses separately do not produce always worse conflict clauses.

In *StaGe* the structure for the clauses is inspired by the version used in *Siege*. There is a specific structure for the binary and N -ary clauses. There is not a structure for the ternary clause because in our experiments seems do not increase the performances. Indeed about of 80 percent of the clauses in the formula are binaries while the ternary ones represent a very small percentage. In addition our implementation is not focused on reducing the missing cache and in general the memory accesses.

Every modern SAT-solvers perform an evolution of the basic two literal watching scheme introduced in *Chaff*. It is a simplification of the *head/tail lists* (HTL) algorithm [40] introduced in *Sato* [32].

3.3.1 Two Watching Literal Algorithm

To restate the semantics of the BCP operation: given a formula and a set of assignments with DLs, deduce any necessary assignments and their DLs, and continue this process transitively by adding the necessary assignments to the initial set. Necessary assignments are determined exclusively by repeated applications of the *unit clause rule*, when a clause has only 1 FREE variable. It stops when no more assignments can be deduced, or when a conflict is identified.

A clause is implied if, and only if, all but one of its literals is assigned to FALSE. So, to implement BCP efficiently, it is necessary to find a way to quickly visit all clauses that become newly implied by a simple addition to a set of assignments.

The most intuitive way to do this is to simply look at every clause in the database clauses that contains a literal that the current assignment sets to FALSE. In effect, it is possible to keep a counter for each clause of how many FALSE literals are in the clause, and modify the counter every time a literal in the clause is set to FALSE. However, if the clause has N literals, there is really no reason that it necessary to visit it when 1, 2, 3, 4, $\dots$, $N - 1$ literals are set zero. It is better to visit it when “the number of literals” counter goes from $N - 2$ to $N - 1$.

As first approximation, it is possible to pick any two literals not assigned to FALSE in each clause to watch at any given time. Thus, it is guaranteed that until one of those two literals is assigned to FALSE, there cannot be more than $N - 2$ literals in the clause assigned to FALSE, that is, the clause is not implied. Now, it is necessary to visit only each clause when one of its two *watched literals* is FALSE. During the visit of a clause, one of the condition must hold:

1. The clause is not implied, and thus at least 2 literals are not assigned to FALSE, including the other currently watched literal. This means that at least one non-watched literal is not assigned to FALSE. This literal is chosen to replace the one just assigned to FALSE. So it is maintained the property that the two watched literals are not assigned to FALSE.

2. The clause is implied. It follows the procedure for visiting an implied clause (usually, this will generate a new implication, unless the clause is already satisfied). One should take note that the implied variable must always be the other watched literal, since, by definition, the clause only has one literal not assigned to FALSE, and one of the two watched literals is now assigned to FALSE.

It is invariant that in any state where a clause can become newly implied, both watched literals are not assigned to FALSE. A key benefit of two literal watching scheme is that at the time of backtracking, there is no need to modify the watched literals in the clause database. Therefore unassigning a variable can be done in a constant time. Further, reassigning a variable that has been recently assigned and unassigned will tend to be faster than the first time it was assigned. This is true because the variable may only be watched in a small subset of the clauses in which was previously watched. This significantly reduces the total number of memory access, which, exacerbated by the high data cache miss rate is a bottleneck for most SAT-implementations. Figure

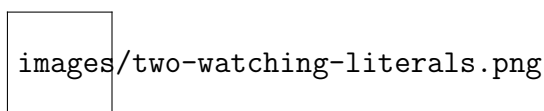


Figure 3.4: BCP using two literal watching.

3.4 shows this technique. It describes how the watched literals for a single clause change under a series of assignments and unassignments. Note that the initial choice of the watched literals is arbitrary, and that for the purpose of this example, the exact details of how the sequence of assignments and unassignments is being generated is irrelevant.

3.3.2 Breath-First BCP

In our solver *StaGe*, the BCP procedure is performed as *breath-first* visit. The breath-first is better than depth-first for the point of view of programming and computational time. Every modern SAT-solvers use this method in order to perform the BCP procedure.

The clauses are divided in two structures: *binaries*, and *N-aries*. For the binary clauses it is not necessary to use the two watching literal scheme. A

binary clause is a logic implication that it is solved easily. For each literal l there is a link to the clauses where it occurs. So, for the BCP, only the binary clauses where $\neg l$ is present are considered.

For the N -ary clauses, it is implemented the TLW algorithm. The data member *literal_watch*, stored in Database, is a vector, indexed by literal, containing the clauses that are watched by the literal itself. The BCP provides

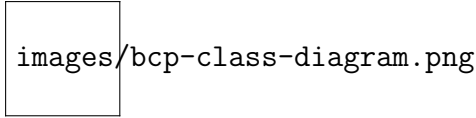


Figure 3.5: Diagram of BCP class.

the *run()* method to execute the propagation from the last decision variable assigned. It is used an index variable: *index_4_bcp*, stored in Database. This member indexes the position in the stack (*stack_assign*, in Database) of the variables to propagate. Unlike *Siege* and other SAT-solvers, there is not a queue where are saved temporary the literals to propagate. In this way there are less operations to compute. The propagation ends when the index is at the top of the stack. It is important to remember that our stack is implemented as a vector. So when a new variable is assigned, it is added in the tail of stack with the *push_back()* method. Therefore, *index_4_bcp* is incremented until to reach the end of this vector.

The binary clauses are visited before the N -ary ones. Each of these types of clauses are visited with two different function in BCP: *binaries()* and *propagate()*.

To perform the BCP procedure are used two different approaches. The strategies differ on the type of the search computed. The BCP mechanism is developed with the Priority Propagation (PP) feature. Normally, this option is not enabled, and the propagation is a breath-first search in the binary clauses and, after, in the N -ary ones. When PP is enabled, there are two different breath-first visits: one for binaries, and one for N -aries instead. In practise, it is done the propagation only considering the binary clauses. If there is not a conflict, the propagation is done for the other N -ary clauses. Now, it is possible that some variables are assigned during the BCP operation in the N -ary clauses. In this case the propagation continues repeating the procedure for the variables just assigned. The literal selected by index is

Algorithm 3.24 BCP, run() method.

```

1: if PP enabled then
2:   perform priority propagation (PP) technique
3: else
4:   while index_4_bcp  $\neq$  EOF stack do
5:     increment number of BCP
6:      $l \leftarrow \neg \text{stack}[\text{index\_4\_bcp}]$ 
7:     INC(index_4_bcp)
8:     BINARIES(l)
9:     PROPAGATE(l)
10:    if CONFLICT in BINARIES(l) or in PROPAGATE(l) then
11:      return CONFLICT
12:    end if
13:  end while
14: end if
15: return NO_CONFLICT

```

not removed from the stack because it is not necessary. So a queue to store the literals to propagate is useless (as in *RSat* and *MiniSat*); only the stack structure is necessary. It is used the negation of the literal in the stack because it is necessary to compute the BCP procedure in the clauses where the literal is FALSE. In Algorithm 3.24, where PP is disabled, are executed the *binaries()* and *propagate()* functions for the literal *l*. If a conflict is found in *binaries()* or *propagated()* functions the execution is stopped and the *run()* method returns that there is a conflict. In our implementation, the type used to return if there is a conflict is boolean. So CONFLICT equals to FALSE, and NO_CONFLICT to TRUE. In the *binaries()* method all the binary clauses are visited. We consider *l* as the literal to propagate. For a specific binary clause it is important to watch the assignment of the other literal. As *l* is obviously FALSE, the other literal indicates if there is a conflict or not. We call this other literal *lit*. If *lit* is TRUE it does nothing because the clause is satisfied. Instead if *lit* is FALSE it means that this binary clause is empty, i.e. there is a conflict. Then the empty clause found is stored in the specific *binary_empty_clause* structure. The *empty_clause* Database member points to this variable. If *lit* is FREE a new assignment in the stack is executed and the reason of this assignment is set.

The TLW scheme is developed as follow. The W_1 and W_2 literals are the first two ones of the clauses. For each clause the literal to propagate *l* has to

Algorithm 3.25 BCP, `binaries()` method.

Require: literal l to propagate

```

1: for all binaries  $b$  where  $l$  appears do
2:    $lit \leftarrow$  other literal of binary clause
3:   if  $lit$  is false then {a conflict is found}
4:     copy the empty clause in binary_empty_clause
5:      $db.empty\_clause \leftarrow binary\_empty\_clause$ 
6:     increase the number of conflicts
7:     return CONFLICT
8:   else if  $lit$  is Free then
9:     update  $db.has\_propagated$ 
10:     $db.SET(other\_lit)$ 
11:     $db.reason[VAR(lit)].SET(lit, l)$ 
12:   end if
13: end for
14: return NO_CONFLICT

```

be in the W_2 position. So W_2 is always a FALSE literal. If W_1 is TRUE the clause is satisfied and there is nothing to do. Instead if W_1 is FALSE a new conflict is found.

With the MLF feature disabled the clause is scanned along until a non-FALSE literal s is found. If s is found we move the clause from the literal watch of l to the *literal_watch* of s . The non-FALSE literal is placed in the W_1 position. If there is not a non-FALSE literal and W_1 is FALSE there is a conflict. So the *empty_clause* Database member and the number of conflicts are updated. Instead if W_1 is not FALSE (i.e. it is obviously FREE) it is assigned with the *set()* method of the Database class.

In the section about Priority Propagation it is analysed the propagation when this feature is enabled (i.e. the code not described here). Obviously, the *binaries()* and *propagated()* methods are used in this case too.

3.3.3 Unit

Unlike the other modern SAT-solvers, *StaGe* splits the BCP procedure in two different parts. The common part is described in the previous section. The *Unit* class is developed to perform the unit propagation at decision level 0. This means that the assignments made in this class are permanent, and it is possible to remove the satisfied clauses and the useless literals. The

Algorithm 3.26 BCP, propagate() method.

Require: literal l to propagate

```

1: for all clause  $c$  in  $\text{literal\_watch}[l]$  do
2:   if  $W_1 = l$  then
3:      $\text{SWAP}(W_1, W_2)$ 
4:   end if
5:   if  $W_1$  is  $\neg$  false then
6:     if MLF enabled then
7:       MLF management
8:     else
9:        $\text{has\_mlf\_lit} \leftarrow \text{false}$ 
10:      search  $\neg$  false literal  $s$  in  $c$ 
11:    end if
12:    if  $s$  is found then
13:      remove  $c$  from  $\text{literal\_watch}[l]$ 
14:       $\text{literal\_watch}[s] \leftarrow c$ 
15:       $\text{SWAP}(W_2, s)$ 
16:    else if  $W_1$  is false then
17:      if  $\neg \text{has\_mlf\_lit}$  then {there is a conflict}
18:         $\text{db.empty\_clause} \leftarrow c$ 
19:        update the number of conflicts
20:        return CONFLICT
21:      else {ignored clause with MLF enabled}
22:         $\text{INC}(\text{db.num\_ignored\_empty\_clauses\_4\_mlf})$ 
23:      end if
24:    else
25:      update  $\text{db.has\_propagated}$ 
26:       $\text{db.SET}(W_1)$ 
27:       $\text{db.reason}[\text{VAR}(W_1)].\text{SET}(c)$ 
28:    end if
29:  end if
30: end for
31: return NO_CONFLICT

```

images/unit-class-diagram.png

Figure 3.6: Diagram of Unit class.

scheme is similar to the BCP class. There are some differences about the implementation. The index to select the literal to propagate is not a Database member. It is a private member of the Unit class because it is not used by another class.

In Database there is an important method that is used by the Unit class. It is the procedure to assign a variable at decision level 0: *set_unit()*, already described in the Database section. When a variable is assigned, it is added to the current stack (*stack_assign*) and to another one: *stack_unit*. The status of a variable is saved in the *assign* Database member. The *run()* method is

Algorithm 3.27 Unit, run() method.

```

1: index  $\leftarrow$  0
2: repeat
3:   db.stack_unit  $\Rightarrow_{index}$  l
4:   INC(index)
5:   SIMPLIFY(l)
6:   BINARIES( $\neg l$ )
7:   PROPAGATE( $\neg l$ )
8: until index is not at the end of stack_unit
9: REMOVE(db.clauses, db.num_clauses, db.lits_nary_clauses)
10: REMOVE(db.learnts, db.num_learnt_clauses,
        db.lits_learnt_nary_clauses)
11: db.variable_order.FILTER(VAR_FILTER())

```

more linear than the one in the BCP procedure. The loop is the same used in BCP's *run()* but with a different index. For each literal *l* the functions *simplify()*, *binaries()*, and *propagate()* are called. After the loop it is called the *remove()* method in order to remove useless clauses and literals in the *N*-ary and learned clauses. The last method invoked is for the heap structure. The *filter()* (with the *var_filter()* parameter) method removes the non-FREE variables from the heap and orders the variables by their activities. The *simplify()* method removes from the memory the clauses not useful because they are satisfied. For each binary clause where the literal *l* occurs it is removed the correspondent reference from the binary clauses pointed by other literal of the binary clause itself. Then all the binary clauses of *l* are deleted from the memory with the *dealloc()* method. The same things are done for the clauses presented in *literal_watch* of *l*. They are removed from the *literal_watch* vector of other watched literal. Then the *literal_watch* vector

Algorithm 3.28 Unit, simplify() method.

Require: literal l to simplify

```

1: for all binary clause  $c$  where  $l$  occurs do
2:    $lit \leftarrow$  other literal of  $c$ 
3:    $db.binaries[lit].REMOVE(l)$ 
4: end for
5: free from memory  $db.binaries[l]$ 
6: for all clauses  $c$  in  $db.literal\_watch[l]$  do
7:   if  $l = W_2$  then
8:      $db.literal\_watch[W_1].REMOVE(c)$ 
9:   else
10:     $db.literal\_watch[W_2].REMOVE(c)$ 
11:   end if
12: end for
13: free from memory  $db.literal\_watch[l]$ 

```

of l is deleted from the memory. Remember that *literal_watch* contains the pointers to the clauses. So when it is removed the clauses related are in the memory yet. To remove them is called the *remove()* procedure. The

Algorithm 3.29 Unit, binaries() method.

Require: literal l to propagate

```

1: for all binary clauses  $c$  where  $l$  occurs do
2:    $lit \leftarrow$  other literal of  $c$ 
3:   if  $lit = \mathbf{Free}$  then
4:      $db.SET\_UNIT(lit)$ 
5:   else if  $lit = \mathbf{true}$  then
6:      $db.binaries[lit].REMOVE(l)$ 
7:   else
8:      $db.RESULT(RESULT\_UNSAT)$ 
9:   end if
10: end for
11: free from memory  $db.binaries[l]$ 

```

binaries() method is similar to the one implemented in the BCP procedure. The differences are that if *lit* is FREE it is assigned with *set_unit()* and not with *set()*. If it is TRUE the clause is satisfied and it is removed from the memory. Instead if it is FALSE a contradiction is found and the SAT-instance is unsatisfiable. After the loop the *binaries* vector of l is deleted from the

memory. With *propagate()* it is done the TLW scheme in the N -ary clauses.

Algorithm 3.30 Unit, *propagate()* method.

Require: l , literal to propagate

```

1: for all clauses  $c$  in  $\text{db.literal\_watch}[l]$  do
2:   if  $W_1 = l$  then
3:      $\text{SWAP}(W_1, W_2)$ 
4:   end if
5:   if  $W_1$  is  $\neg$  true then
6:     search a  $\neg$  false literal  $s$  in  $c$ 
7:     if  $s$  is found then
8:       if  $s$  is Free then
9:         remove  $W_2$  from  $c$ 
10:         $W_2 \leftarrow s$ 
11:         $\text{db.literal\_watch}[W_2] \leftarrow c$ 
12:        update statistics
13:      else
14:         $\text{db.literal\_watch}[W_1].\text{REMOVE}(c)$ 
15:      end if
16:    else if  $W_1$  is false then
17:       $\text{db.RESULT}(\text{RESULT\_UNSAT})$ 
18:    else
19:       $\text{db.SET\_UNIT}(W_1)$ 
20:       $\text{db.literal\_watch}[W_1].\text{REMOVE}(c)$ 
21:    end if
22:  else
23:     $\text{db.literal\_watch}[W_1].\text{REMOVE}(\&c)$ 
24:  end if
25: end for
26: free from memory  $\text{db.literal\_watch}[l]$ 

```

The differences from the version developed in the BCP class are that if a clause is satisfied then it is removed (line 14). It is not necessary setting the reason for the variables assigned with the specific *set_unit()* method. When a contradiction is found it is called the *result()* Database method and the program ends because the SAT instance is unsatisfiable. The *propagate()* method does not remove the clauses from the memory, but only the pointers in *literal_watch*. The *remove()* method is developed to delete these objects from the memory. If a clause is satisfied by W_1 or W_2 then it is directly removed because the *literal_watch* is already freed. If the clause is satisfied by

Algorithm 3.31 Unit, remove() method.

```

1: for all  $N$ -ary clause  $c$  do
2:   if  $W_1$  or  $W_2$  are true then
3:     update statistics
4:     delete  $c$  from memory
5:   else
6:     for all other literal  $l$  in  $c$  do
7:       if  $l$  is true then
8:         if  $W_2$  is Free then
9:            $\text{db.literal\_watch}[W_1].\text{REMOVE}(c)$ 
10:           $\text{db.literal\_watch}[W_2].\text{REMOVE}(c)$ 
11:         end if
12:         update statistics
13:         delete  $c$  from memory
14:       end if
15:     end for
16:   end if
17: end for

```

another literal, it is necessary to remove the clause pointer from the respective literal_watch structures.

3.4 Backjumping

This class implements the procedures to perform the *conflict-driven clause learning* (CDCL). It represents a fundamental improvement over DLL, and all substantial recent advances in SAT-solver technology are closely related to it. This is the most important characteristic of a SAT-solver. Without the backtracking procedure and the relative conflict clause learning, it is impossible to solve the SAT-instances in a reasonable period of time.

StoGe implements the First-UIP technique derived from the work described on [23] where it is introduced the concept of Implication Graph to represent the implication relationship.

3.4.1 Implication Graph

During the search process, the implication relationship of variable assignments can be expressed as an implication graph. A typical implication graph is

represented in Figure 3.7. The implication graph is a directed acyclic graph (DAG)⁵. Each vertex represents a literal assignment. The incident edges of each vertex are the reason that lead to the assignment. The vertices that have directed edges to certain vertex are called as its antecedent vertices. A decision vertex has no incident edge. Each decision variable has a decision level associated with it (the number within the parentheses). In implication graphs with no conflict there is at most one vertex for variable (the positive or negative literal). When there is a conflict, a variable has two vertex (one for the positive literal, and one for the negative one). In Figure 3.7, the variable $V[18]$ is the conflicting variable (i.e. the variable that has generated the conflict). In an implication graph, vertex a is said to dominate vertex b

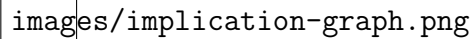


Figure 3.7: A typical implication graph.

if, and only if, any path from the decision variable of the decision level of a to b needs to go through a . A Unique Implication Point (UIP) is a vertex at the current decision level that dominates both vertices corresponding to the conflicting variable. In Figure 3.7, in the sub-graph of current decision level 5, V_{10} dominates both vertices V_{18} and $-V_{18}$, therefore, it is a UIP. The decision variable is always a UIP. There may be more than one UIP for a certain conflict. In the example, the UIPs are: V_{11} , V_2 , and V_{10} . The UIP are ordered starting from the conflict. So V_{10} is the First-UIP.

3.4.2 Conflict Clause Learning

Conflict analysis is the procedure that finds the reason for a conflict and tries to resolve it. It tells the SAT-solver that there exists no solution for the problem in a certain search space, and indicates a new search space to continue the search.

The conflict clause is generated by a bipartisan of the implication graph. The partition has all the decision variables on one side (*reason side*), and the conflicting variable in the other side (*conflict side*). All the vertices on

⁵For a short description see http://en.wikipedia.org/wiki/Directed_acyclic_graph.

the reason side that have at least one edge to the conflict side comprise the reason to the conflict. This bi-partition is called *cut*.

The UIP is extended to decision levels other than the current level. A vertex a at decision level dl is a UIP if, and only if, any path from the decision variable of dl to the conflicting variable needs to either go through a , or go through a vertex of decision level higher than dl that is on the reason side of the partition.

As mentioned before, each non-decision variable has an antecedent clause that represents the reason for the assignment. The antecedent clause is represented in the implication graph as the incoming edges of a vertex. For example, in Figure 3.7, $\neg V_2$ has two incident edges, from $\neg V_{12}$ and V_{16} respectively. Therefore, the antecedent clause for V_2 is $[\neg V_{16} \vee V_{12} \vee \neg V_2]$. This clause is obviously presents in the clause database. It is possible to construct additional clauses that are consistent with the clause database by replacing certain literals in the antecedent clause with all of their antecedent literals. In this way it is also possible to create a clause from a cut not involving conflicts. This means that, for every implication graph, there are a big number of clauses that can be learned. At the reality, in the modern SAT-solvers, for every conflict only one clause is learned. Indeed the number of learned clauses grows dramatically and, for this reason, there is a specific procedure to remove the learned clauses.

3.4.3 First-UIP

As described in [23], the First-UIP is the best choice from the possible cut to use. In *StoGe*, it is implemented the Backjumping procedure with this technique.

The first cut consents to create the new conflict clause as an asserting clause. An asserting clause is a clause that has only one literal free. With the first cut the UIP is in the current decision level. In Figure 3.8, with the First

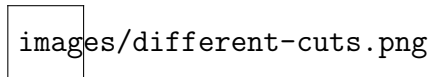


Figure 3.8: Different cuts.

UIP Cut, the conflict clause is: $[V_{10} \vee \neg V_{19} \wedge \neg V_8 \wedge V_{17}]$. The only variable

at the current decision level is V_{10} . So with the backtracking procedure it is only necessary cancelling the assignments of the highest decision level in the conflict clause, with the exception of the current one. The conflict clause found is an assertion clause and it is necessary performing the propagation of the variables at the current decision level. In the example, the assignments at decision level 5 are cancelled and it is assigned the literal V_{10} in the decision level 3. The experiments done in [23] show that the scheme based on the first unique implication point of the implication graph has proved to be quite robust and effective to solve SAT problems in comparison with other schemes.

3.4.4 Backjumping Implementation

StaGe performs the described techniques with the Backjumping class. The entry point is the *run()* method called by the Solver class. The *run()* method

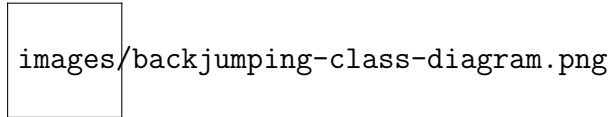


Figure 3.9: Diagram of Backjumping class.

executes a loop to find the First-UIP. To do this, there is a counter called *lits_current*. Initially this counter is zero. A variable is extracted from the stack. It is used the same strategy adopted in the BCP procedure. The variable is not removed from the stack. It is used an index to get the specific literal. In each iteration, the index (*index_stack*) is decreased. Then it is got the reason clause related to the literal. For each element of this clause, it is monitored if the variable is already checked (the attribute *seen*, stored in Database). If it is not already checked and it was not assigned at the current decision level then it is added to the conflict clause. Instead if it is on the current decision level the *lits_current* counter is increased. After the loop, the literal l is the First-UIP. This literal will be the next assignment. So it is added to the conflict clause in position 0 (W_1). If the conflict clause is not an unary clause, is performed the simplification strategy. The expensive conflict clause minimization is described in Chapter 4. The standard simplification strategy consists to check if a literal has the reason with each variable “seen”. If it is true the variable is removed from the conflict clause. As example,

Algorithm 3.32 Backjumping, run() method.

Require: clause c , reason of literal in position $index$ in the stack

```

1: db.CLAUSE_UPDATE( $c$ ) {only for learned clauses}
2: repeat
3:   for all literal  $l$  in  $c$  do
4:     if VAR[ $l$ ] is  $\neg$  SEEN then
5:       db.VAR_UPDATE(VAR( $l$ ))
6:       set as SEEN VAR( $l$ )
7:       if db.level[VAR( $l$ )] = db.decision_level then
8:         INC( $lits\_current$ )
9:       else
10:        CONFLICT CLAUSE  $\Leftarrow l$ 
11:      end if
12:    end if
13:  end for
14:  if MLF enabled then
15:    MLF Backtracking strategy
16:  else
17:    while VAR( $l$ ) is not SEEN do
18:       $stack\_assign \Rightarrow_{index\_stack} l$ 
19:      DEC( $index\_stack$ )
20:    end while
21:    set as SEEN VAR( $l$ )
22:    DEC( $lits\_current$ )
23:  end if
24:   $reason \leftarrow db.reason[VAR(l)].get$ 
25: until  $lits\_current > 0$ 
26: CONFLICT CLAUSE  $\Leftarrow_{w_1} \neg l$ 
27: EXPENSIVE CCCMIN or STANDARD SIMPLIFICATION strategy
28: if CONFLICT CLAUSE SIZE  $> 1$  then
29:   CONFLICT CLAUSE  $\Leftarrow_{w_2}$  literal  $l \in$  CONFLICT CLAUSE with max
   decision level  $<$  current decision level
30: end if
31: set the SEEN attribute to false  $\forall$  variables in CONFLICT CLAUSE
32: update statistics

```

we consider the following conflict clause: $[D, A, B, C]$ and the following reasons:

A: $[B, C]$
 B: no reason
 C: $[E, H]$

This is an asserting clause. D is the only FREE literal. Every other literals are FALSE. The assignment of A is implied by the assignments of B and C . Therefore, the reason of A is included in the conflict clause. That means that the literal A is redundant and it is possible to remove them from the conflict clause. These aspects are described more precisely in the expensive conflict clause minimization section in Chapter 4. After the simplification method

Algorithm 3.33 Backjumping, standard simplification strategy.

```

1: for all literal  $l$  in CONFLICT CLAUSE do
2:    $r \leftarrow$  get reason of  $l$ 
3:   if  $r \neq \text{NULL}$  then
4:      $not\_remove \leftarrow \text{true}$ 
5:     for all literal  $l$  in  $r$  do
6:       if  $\neg db.seen[VAR(l)]$  then
7:          $not\_remove \leftarrow \text{true}$ 
8:       end if
9:     end for
10:    if  $not\_remove = \text{true}$  then
11:      delete  $l$  from CONFLICT CLAUSE
12:    end if
13:  end if
14: end for

```

the conflict clause is found and it is possible to add this one in the Database structure. This procedure is called in the *run()* method of the *Solver* class.

Chapter 4

Advanced Techniques

StaGe is developed to improve the performances of the modern SAT-Solvers as *RSat*, *MiniSat*, *Siege*, etc... Obviously, our experiments show that there is not a perfect solver. In some cases a solver works faster, but in another ones the results are not satisfactory.

Particular attention was dedicated to introduce new advanced techniques in order to test the impact on the resolution. In addition, we have implemented some features inspired by the versions made in other state-of-art SAT-solvers. It is important to test if a feature has good performances with other features, and if it is true with which ones it performs the better results. Like this, it is empirically explored the performances of each feature and their potential combinations.

4.1 StaGe benchmarking

Our solver *StaGe* is tested with the benchmarks described in Chapter 2. The implementation involves the features described in the previous chapter. The results are comparable to *BerkMin* and *MiniSat* (standard, without expensive conflict clause minimization).

In the SAT-Race benchmarks of Table 4.1, *StaGe* solves more instances than *MiniSat*. However, this improvement cause a deterioration in terms of the execution time. Still referring to *MiniSat*, *StaGe* gets the best results with the *velev* instance family (24 instances resolved against 20 ones of *MiniSat*, see Table 2.5 in Chapter 2). From the various experiments done with the

different SAT solvers, it is obvious that the industrial instances selected by SAT-Race 2006 are more complex to solve than those ones selected by SATLIB.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	<i>StaGe</i>	MS	<i>StaGe</i>	MS	<i>StaGe</i>
aloul (1/1)	173.2	65.0	3140.1	8154.8	2663.4	7083.3
een (3/3)	9.9	14.1	43.4	49.2	8.7	12.2
goldb (3/4)	1967.6	1788.3	2758.8	1467.1	587.1	269.5
grieu (3/3)	916.9	577.9	57.9	445.9	38.4	327.4
hoons (3/4)	1415.3	1354.6	219.1	63.0	177.7	49.8
manol-pipe (29/34)	5930.8	7747.7	483.6	552.2	115.3	152.6
mizh (1/3)	1118.2	2695.8	1624.7	2063.0	294.1	723.3
narain (3/3)	1741.8	1297.3	260.7	139.9	105.4	79.4
schup (2/3)	1422.8	1678.3	337.7	529.5	243.1	368.5
simon (3/4)	2269.8	1714.2	1544.9	1668.3	998.7	1174.2
stric-bmc (1/1)	7.6	6.7	30.3	26.2	10.5	10.8
vange (1/1)	15.5	11.4	449.5	367.6	102.2	60.4
velev (24/34)	13486.4	13857.3	332.6	665.2	34.2	110.7
TOTAL (77/98)	30475.9	32808.5	585.1	716.2	185.2	278.4

Table 4.1: *StaGe* on SAT-Race benchmarks compared with *MiniSat* (MS). Each row represents a different instance family, with the following information: “CPU-Time”, total execution time required; “Decisions”, average number of decisions; and “Conflicts”, average number of conflicts.

The results of *StaGe* on benchmarks extracted from SATLIB in Table 4.2 confirm this trend. *StaGe* solves the same number of instances of other SAT solvers tested in our experiments. Only *MiniSat* is able to resolve these instances with a lower CPU-time.

Comparison

StaGe was developed using as a reference point *MiniSat*. So it is appropriate to evaluate the performance of *StaGe* respect to *MiniSat* through crossed analysis. As previously noted, *StaGe* solves more instances than *MiniSat* but with a higher CPU-time.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	MS	<i>StaGe</i>	MS	<i>StaGe</i>	MS	<i>StaGe</i>
AIS (4/4)	0.0	0.1	0.1	1.6	0.0	1.2
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	0.5	1.0	1.1	1.4	0.4	0.9
BMC (13/13)	16.2	18.5	11.3	11.7	2.9	4.0
flat (100/100)	0.1	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	2.7	5.3	45.8	63.1	24.1	42.6
ind-inference (41/41)	1.2	0.3	0.6	0.5	0.3	0.1
logistics (4/4)	0.1	0.1	2.7	1.5	0.3	0.3
parity (20/30)	9002.1	9003.1	2.0	5.5	1.8	5.1
phole (5/5)	40.9	84.3	338.4	2648.3	276.5	2250.5
pret (8/8)	0.0	0.0	0.0	1.5	0.0	0.6
QG (22/22)	24.6	11.7	8.1	6.0	6.7	5.2
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.3	0.0	0.2	0.1	0.0	0.0
TOTAL (338/352)	12688.9	12724.6	6.6	40.9	4.9	34.4

Table 4.2: *StaGe* on SATLIB benchmarks compared with *MiniSat* (MS).

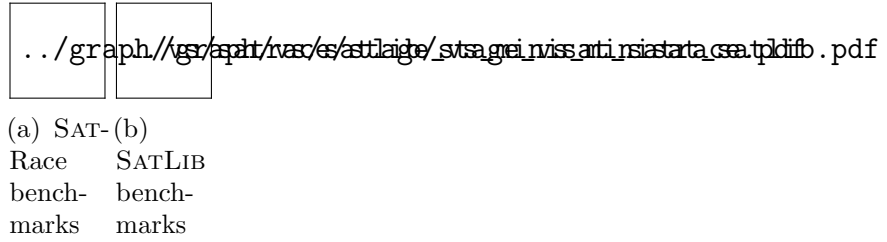


Figure 4.1: Comparison between *StaGe* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

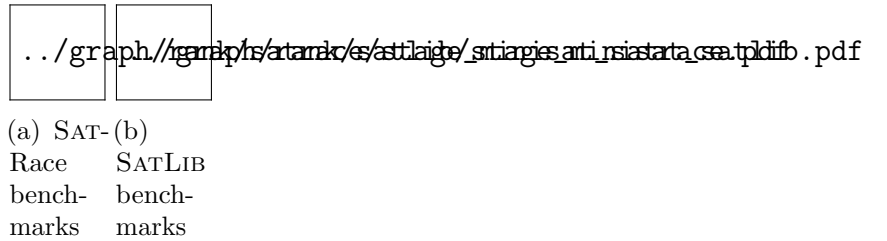


Figure 4.2: Comparison between *StaGe* and *MiniSat* on (a) SAT-Race and (b) SATLIB benchmarks.

This development is the starting point. Its results are comparable to other modern SAT-solvers like *MiniSat*, *zChaff*, *Siege*, and *BerkMin*. In the next sections different techniques are described, introduced and tested in order to individuate the possible improvements as regards the CPU-time required, number of solved instances, decisions made, and conflicts found.

4.2 Implicates

The most important innovative feature introduced in *StaGe* is called *Implicates*. It remembers the name of Implication used in *conflict-driven clause learning* as described in Chapter 3. This technique is focused only on the binary clauses while the N -ary clauses are not considered.

Each binary clause is a logic implication. For example, the clause $[A, B]$ is the formula: $A \vee B$. By the De Morgan's laws¹, it is possible to obtain the implications $\neg A \rightarrow B$ and $\neg B \rightarrow A$ that are equivalent to the binary clause. Note that these two implications are one the negation of the other. Considering all the binary clauses of a SAT-instance, the implications are used to build a graph. To avoid misunderstands this graph is called *implicate graph*. For each set of binary clauses there are many different *implicate graphs*.

Clause	Implications	
$\neg A, B$	$A \rightarrow B$	$\neg B \rightarrow \neg A$
$\neg B, D$	$B \rightarrow D$	$\neg D \rightarrow \neg B$
$\neg D, A$	$D \rightarrow A$	$\neg A \rightarrow \neg D$
C, D	$\neg C \rightarrow D$	$\neg D \rightarrow C$
$\neg B, C$	$B \rightarrow C$	$\neg C \rightarrow \neg B$
$\neg B, \neg A$	$B \rightarrow \neg A$	$A \rightarrow \neg B$
D, C	$\neg D \rightarrow C$	$\neg C \rightarrow D$
$\neg E, C$	$E \rightarrow C$	$\neg C \rightarrow \neg E$
$\neg E, \neg A$	$E \rightarrow \neg A$	$A \rightarrow \neg E$

Table 4.3: Binary clauses and implications example.

In Table 4.3 it is shown an example of a set of binary clauses and their implications. With the implications found, the *implicate graphs* are built.

The implicate graph is a directed graph and each vertex represents a literal. The incident edges of each vertex are the variable assignments that implicate the assignment of the vertex/literal itself. For each variable there are at most two vertices, one for literal. If there is an edge from the vertex a to b then the vertex a is the *father* of b . Instead if there is a path from the vertex a to b then a is *parent* of b .

¹For a short description see http://en.wikipedia.org/wiki/De_Morgan_duality.

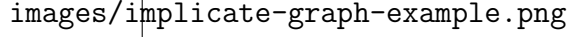


Figure 4.3: Example of graph of Implicate.

This consideration introduces the concept of *dominant*. A vertex a *dominates* b , if the vertex a is a parent of b . For example, in Table 4.3, E dominates $-A$ and C . It means that if the vertex a is assigned then all the vertices which it dominates are assigned by BCP. We order the dominant according to the distance. The distance is evaluated as the number of edge from a vertex to its dominant. So the father literal is a dominant of order 1.

It is possible to have cycles in the graph. The literals in a loop are equivalent, to be more precise every vertex (literal) of a cycle dominates the other ones. If one of these literals is assigned, the other literals in the loop are implied and assigned too. In the example, in Figure 4.3, there is a cycle in A , B , and D . If the literal A is assigned then the other literals are propagated by the BCP procedure.

With this property it is possible to find the *permanent* assignments. If a vertex a dominates its negation then it is necessary assigning the literal $-a$. In Figure 4.3, the literal A dominates $-A$. So it is assigned $-A$ at decision level 0. If a vertex a dominates its negation and vice versa then the formula is unsatisfiable. If a cycle includes a vertex a and its negation the formula is unsatisfiable too. Instead if a vertex a dominates a vertex b and its negation, it is assigned the negation of literal a because the assignment of a will make a contradiction.

With the implicate graph it is possible to find if there is a conflict that involves only the binary clauses and assign variables at decision level 0. The objective of this feature is to increase the number of propagations during BCP. To do this, for each literal is computed the *dominant with the highest order* (DHO) that represents the dominant literal in the heuristic procedure. Like this, the number of variables assigned by BCP (excluding the dominant variable) is equal or higher than the order of dominant. Selecting the dominant with the highest order increases the number of propagated variables.

Why do we want to increase the number of variable propagated by the BCP procedure? The implicate technique is performed only on binary clauses.

Therefore the reason of these assignments are the binary clauses. With many assignments the possibility to find a conflict in a binary clause increases. In this way the size of the conflict clause is kept down. In *StaGe* the conflict clauses with only two literals are not deleted. To have short clauses eases the search. This approach is exactly the opposite of the *MiniSat* strategy. For the author of *MiniSat*, to have big clauses seems to increase the performance. In our experiments this is not confirmed. This means that it does not exist a solution absolutely better than another. It depends on the type of benchmarks, instances and algorithm implementation.

4.2.1 Different implementations

For each literal a dominant is computed and stored in the *Database* class. Different implementations were developed in our solver *StaGe*. The basic

Algorithm 4.1 visit() procedure.

Require: literal l

Require: distance d

Require: queue q

- 1: **for all** binary clause c where $l \in c$ **do**
 - 2: $lit \leftarrow$ other literal in c
 - 3: $dom \leftarrow \neg lit$
 - 4: $q \leftarrow (dom, distance + 1)$
 - 5: **end for**
-

implicate procedure use a queue q to save the dominants for each literal l . It is an expensive implementation. Indeed it is computed the same procedure for each literal and they are not used the saved dominants. This implementation cause a bottleneck in a modern SAT-solver. So it is necessary performing a better procedure with a different algorithm. If a is a dominant of b then it is also the dominant of literals of which b is dominant. To compute the dominants for each literal it is better to evaluate what literals are dominated. For each literal it is stored a dominant vector used to save the dominant literals found. In addition, for each dominant there is a vector that contains the dominated literals. The literal l , if not evaluated, is added to the *buffer*. Then for each literal lit in *buffer* it is checked the other literals *other_lit* presented in the binary clauses where $\neg lit$ is presented. If the distance of *other_lit* is lower than the distance of lit then *other_lit* is updated. The

Algorithm 4.2 Implicate basic function.

Require: queue q to save the visited literals

Require: distance d

```

1: for all literal  $l$  do
2:   clear  $q$ 
3:    $q \leftarrow (l, 0)$ 
4:   while  $q$  is not empty do
5:      $q \Rightarrow (lit, d)$ 
6:     VISIT( $lit, d, q$ )
7:   end while
8:    $dom[l] \leftarrow$  get lit with highest  $d$  from  $q$ 
9: end for

```

Algorithm 4.3 Implicate function, second version.

Require: temp *buffer* vector

```

1: for all literal  $l$  do
2:   if  $l$  is not evaluated then
3:      $buffer \leftarrow l$ 
4:     while  $buffer$  is not empty do
5:        $buffer \Rightarrow lit$ 
6:       for all binary clause  $c$  where  $\neg lit \in c$  do
7:          $other\_lit \leftarrow$  other literal in  $c$ 
8:         if  $distance[other\_lit] \leq distance[lit]$  then
9:           update  $distance[other\_lit]$ 
10:           $dominant[other\_lit] \leftarrow l$ 
11:           $dominants[lit] \leftarrow other\_lit$ 
12:           $buffer \leftarrow other\_lit$ 
13:        end if
14:      end for
15:    end while
16:  end if
17: end for

```

dominant vector stores the dominant literals, and the dominants array the dominated literals. This last vector is used to update a part of the implicate graph. Indeed, if a group of literals is set with a specific dominant, it is not sure that it is the DHO because it could be dominated by another literal. It is necessary updating each literal dominated by the dominant literal in the case that this dominant literal is not the DHO and it is in turn dominated.

StaGe Implementation

In *StaGe*, the *Implicate* class follows the improvements described in the previous section. The entry point is obviously the *run()* method that calls the *reset()* and *search()* procedures. In this algorithm, first of all, the *distance* literal is set zero for each literal. When for a literal is found a dominant or the literal is a dominant itself the distance is updated. Like this, it is not possible to check the literal more times than one. Each literal is analysed as dominant at most only one time.

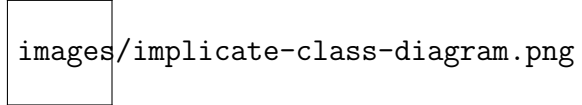


Figure 4.4: Diagram of Implicate class.

The *run()* method calls the *reset* method to set the initial value for the *Implicate* data members. Then for each literal it is called the *search* if the literal is **FREE** and if it was not already evaluated.

Algorithm 4.4 *Implicate*, *run()* method

```

1: RESET()
2: for all literal l do
3:   if l is Free and distance[l] = 0 then
4:     SEARCH(l)
5:   end if
6: end for

```

With this implementation of the *run()* method, the *search* is called not for each literal but for only a part that is sufficient to set all the literals. For example, if a formula has 10 literals and 5 binary clauses, it is not necessary computing the *search* for each 10 literals, but for a number that compute

uint parent	stores the parent vertex for each literal.
bool is_dom	indicates if the literal is a DHO.
uint distance	indicates the distance between the literal and its dominant.
vector<uint> buffer	visits the literals during the search
vector<uint> dominant	contains the dominated literal for each dominant.

Table 4.4: Implicate data members.

the dominants for each literal. This increases the performances of the search. The worst case is when each binary clause has not a common variable with at least another binary clause. In this situation, the implicate graph has only dominants of order 1 and the number of literals checked during every search is low.

Algorithm 4.5 Implicate, reset() method.

```

1: for all literal  $l$  do
2:    $db.dominant[l] \leftarrow l$  {cancels the dominant}
3:    $dominant[l].CLEAR()$  {empties the dominant vector}
4:    $is\_dom[l] \leftarrow \text{false}$  { $l$  is not a DHO}
5:    $distance[l] \leftarrow 0$  { $l$  is not evaluated}
6:    $parent[l] \leftarrow db.num\_literals$  { $l$  has not a parent}
7: end for

```

In Algorithm 4.6, the literal l is a DHO of lit if this one is FREE, the distance between lit and its dominant is lower than the distance between l and its dominant, and lit is not a parent of l . Specifically, lit has not be a dominant of l . Indeed if lit dominates l we are in the presence of a cycle. In this way the cycle is ignored. To do this, the *is_super_parent()* method is executed. In this procedure the *parent* data member is recursively visited to check if lit dominates l . If these conditions are verified l is a DHO of lit . Then it is updated the dominant vector adding lit . The *db.dominant* data member stores the DHO for each literal. If lit was previously evaluated, it means that it is a DHO of some literals. Then each literal of which lit is the DHO is set with l as new DHO. On the contrary, if it is not still evaluated then it is added to the *buffer*.

The Implicate procedure is called in the *run()* method of the Solver class.

Algorithm 4.6 Implicate, search() method.

Require: literal dom to set as temporary DHO.

```

1:  $buffer \leftarrow dom$ 
2: while  $buffer$  is not empty do
3:    $buffer \Rightarrow l$ 
4:   for all binary clauses  $c$  with  $\neg l \in c$  do
5:      $lit \leftarrow$  other literal of  $c$ 
6:     if  $lit$  is Free and is  $\neg dom$  and  $distance[lit] \leq distance[l]$  and  $lit$  is
       not parent of  $l$  then
7:        $parent[lit] \leftarrow l$ 
8:       move  $lit$  from old to new dominant
9:        $db.dominant[lit] \leftarrow dom$ 
10:      if  $lit = dom$  then
11:        update every literal dominated by  $lit$ 
12:      else
13:         $buffer \leftarrow lit$ 
14:      end if
15:       $INC(distance[lit])$ 
16:    end if
17:  end for
18: end while
19: clear  $buffer$ 

```

Algorithm 4.7 Implicate, is_super_parent method.

```

1: {check if  $par\_l$  dominates  $l$ }
2: repeat
3:   if  $parent[l] = par\_l$  then
4:     return true
5:   end if
6:    $l \leftarrow parent[l]$ 
7: until  $l$  has no parent
8: return false

```

Before the search begins, it is built the implicate graph with the code shown in Algorithm 4.8. It calls the `init()` and `run()` implicate procedures and prints the statistics, like, for example, the used memory and the CPU-time required. To manage the implicate feature there is the policy management described in

Algorithm 4.8 Solver, implicate initialization.

```

1: if IMPLICATE enabled then
2:   implicate.INIT()
3:   implicate.RUN()
4:   implicate.PRINT_STATS()
5: end if

```

the Database section. By default, the implicate function is not executed every time. When the BCP finishes without conflicts it is executed the implicate policy. In Database there is a flag: *implicate_enabled*, which indicates that the implicate function is enabled or not. Periodically this option is switched. To execute the *run()* method it is necessary that the number of conflicts from the last execution of *implicate.run()* is greater than the specific limit. Then the number of conflicts is zeroed and the limit is increased with *implicate_inc*. The

Algorithm 4.9 Solver, implicate policy.

```

1: if IMPLICATE enabled and db.num_conflicts_from_implicate  $\geq$ 
   db.implicate_limit then
2:   implicate.RUN()
3:   db.num_conflicts_from_implicate  $\leftarrow$  0
4:   db.implicate_limit  $\leftarrow$  db.implicate_limit  $\times$  db.implicate_inc
5: end if

```

value of *implicate_limit* and *implicate_inc* parameters are set after different experiments and tests. The better performances are with these parameters: 150 for *implicate_limit* and 1.75 for *implicate_inc*. Obviously, it is a better solution for each kind of instances. If you want to execute a specific kind of instances, the optimal values can be different. In Heuristics, if the implicate feature is enabled, it is selected the *dominant_lit* instead of the literal *next_lit* chosen by the standard method. The *next_var* variable, if not chosen, it is again inserted into the *variable_order* heap.

Algorithm 4.10 Heuristics, implicate management.

Require: variable *next_var* chosen by standard heuristic

Require: literal *next_lit* chosen by standard heuristic

```

1: if IMPLICATE enabled then
2:   dominant_lit  $\leftarrow$  db.dominant[next_lit]
3:   if dominant_lit is Free then
4:     db.variable_order  $\leftarrow$  next_var
5:     return dominant_lit
6:   else
7:     return next_lit
8:   end if
9: end if

```

4.2.2 Benchmarking

In *StaGe*, the implicate technique is executed with the last implementation described. So it is periodically computed with the specific parameters. We did several experiments with the Implicate feature. The first one was to execute the technique only at beginning and not during the search. We call this implementation *implicate static*. Another experiment was done with the implication feature executed when a *unit clause* is found. This approach is called *implicate unit*. These two approaches have not improvement the performances. They are comparable with *zChaff* and make worse the results of our solver *StaGe*.

In SAT-Race benchmarks in Tables 4.5 and 4.6, the two implementations of implicate work as *zChaff*. The implicate static method seems to have better performances.

In conclusion, the implicate static approach does not work fine with the industrial benchmarks as the *velev* and *manol-pipe* instance families. Same results for the implicate unit solution. In some cases, implicate static gets better results, but, in other ones, is the unit paradigm that performs better.

Obviously it is not guaranteed that the implicate feature works fine with all kind of benchmarks. In addition, it is difficult to prove for what reason a feature works fine with some benchmarks and not with other ones.

The SATLIB benchmarks in Tables 4.7 and 4.8 are solved by these two implementations with reasonable, but not excellent, results. Considering the results of other solvers, they require more time to solve the instances.

In practice, the implicate feature with these two approaches does not

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IS	<i>StaGe</i>	+IS	<i>StaGe</i>	+IS
aloul (1/1)	65.0	86.9	8154.8	9716.4	7083.3	8579.2
een (3/3)	14.1	41.3	49.2	73.2	12.2	25.3
goldb (2/4)	1788.3	2102.3	1467.1	1400.6	269.5	479.6
grieu (3/3)	577.9	337.0	445.9	209.3	327.4	153.8
hoons (3/4)	1354.6	2044.3	63.0	111.7	49.8	94.9
manol-pipe (25/34)	7747.7	10723.4	552.2	376.5	152.6	130.3
mizh (0/3)	2695.8	2700.0	2063.0	-	723.3	-
narain (1/3)	1297.3	2489.6	139.9	11.9	79.4	3.8
schup (2/3)	1678.3	2449.8	529.5	530.3	368.5	392.6
simon (2/4)	1714.2	2627.9	1668.3	2323.0	1174.2	1863.7
stric-bmc (1/1)	6.7	9.6	26.2	29.7	10.8	14.7
vange (1/1)	11.4	27.3	367.6	476.4	60.4	102.1
velev (18/34)	13857.3	16933.4	665.2	231.9	110.7	80.9
TOTAL (62/98)	32808.5	42572.7	716.2	540.5	278.4	317.9

Table 4.5: *StaGe* + Implicate-Static (IS) on SAT-Race benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IU	<i>StaGe</i>	+IU	<i>StaGe</i>	+IU
aloul (1/1)	65.0	89.9	8154.8	9917.0	7083.3	8801.3
een (3/3)	14.1	53.9	49.2	54.7	12.2	16.9
goldb (2/4)	1788.3	2120.0	1467.1	1307.1	269.5	434.6
grieu (3/3)	577.9	114.5	445.9	151.7	327.4	109.7
hoons (3/4)	1354.6	2428.1	63.0	104.7	49.8	93.4
manol-pipe (26/34)	7747.7	10543.9	552.2	450.4	152.6	153.9
mizh (0/3)	2695.8	2700.0	2063.0	-	723.3	-
narain (0/3)	1297.3	2700.0	139.9	-	79.4	-
schup (1/3)	1678.3	2417.9	529.5	468.5	368.5	346.1
simon (2/4)	1714.2	2442.8	1668.3	1658.4	1174.2	1296.9
stric-bmc (1/1)	6.7	17.4	26.2	34.4	10.8	17.2
vange (1/1)	11.4	15.0	367.6	404.5	60.4	68.8
velev (17/34)	13857.3	18427.4	665.2	331.4	110.7	117.2
TOTAL (60/98)	32808.5	44070.8	716.2	583.9	278.4	322.5

Table 4.6: *StaGe* + Implicate-Unit (IU) on SAT-Race benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IS	<i>StaGe</i>	+IS	<i>StaGe</i>	+IS
AIS (4/4)	0.1	2.0	1.6	21.6	1.2	18.7
BF (4/4)	0.0	0.0	0.4	0.3	0.1	0.1
blocksworld (7/7)	1.0	3.8	1.4	3.7	0.9	2.9
BMC (13/13)	18.5	45.2	11.7	17.0	4.0	7.8
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	6.0	63.1	66.0	42.6	48.9
ind-inference (41/41)	0.3	1.7	0.5	1.6	0.1	0.9
logistics (4/4)	0.1	0.1	1.5	1.4	0.3	0.3
parity (20/30)	9003.1	9003.7	5.5	6.2	5.1	5.8
phole (5/5)	84.3	93.3	2648.3	2650.7	2250.5	2291.9
pret (8/8)	0.0	0.0	1.5	1.6	0.6	0.6
QG (22/22)	11.7	13.2	6.0	5.9	5.2	5.3
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.1
SW100 (100/100)	0.0	0.2	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12769.4	40.9	41.6	34.4	35.6

Table 4.7: *StaGe* + Implicate-Static (IS) on SATLIB benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IU	<i>StaGe</i>	+IU	<i>StaGe</i>	+IU
AIS (4/4)	0.1	2.0	1.6	21.6	1.2	18.7
BF (4/4)	0.0	0.1	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	3.5	1.4	2.5	0.9	1.9
BMC (13/13)	18.5	80.2	11.7	19.1	4.0	9.7
flat (100/100)	0.0	0.1	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	7.6	63.1	71.5	42.6	52.3
ind-inference (41/41)	0.3	2.7	0.5	2.4	0.1	1.5
logistics (4/4)	0.1	0.2	1.5	2.0	0.3	0.5
parity (20/30)	9003.1	9004.0	5.5	6.4	5.1	6.1
phole (5/5)	84.3	105.7	2648.3	2899.5	2250.5	2525.2
pret (8/8)	0.0	0.0	1.5	1.6	0.6	0.6
QG (22/22)	11.7	11.8	6.0	6.0	5.2	5.4
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.2	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12818.1	40.9	45.5	34.4	39.2

Table 4.8: *StaGe* + Implicate-Unit (IU) on SATLIB benchmarks.

increase the performances of our solver *StaGe*. Even in some cases, the results are worsen. This is the reason because the final release of *StaGe* presents only the implicate executed periodically when the BCP ends correctly. This method is called implicate *online* to indicate the execution during the search.

Unlike the *static* and *unit* approaches, the behaviour of implicate on-line with the velev instance family is almost the same of that one of the standard version of *StaGe*. Unfortunately, the problems remain with the manol-pipe benchmark. This impacts negatively on the performances which are lower than the standard version of *StaGe*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IO	<i>StaGe</i>	+IO	<i>StaGe</i>	+IO
aloul (1/1)	65.0	74.3	8154.8	8296.1	7083.3	7222.3
een (3/3)	14.1	33.4	49.2	69.7	12.2	20.7
goldb (2/4)	1788.3	1956.4	1467.1	986.6	269.5	204.9
grieu (3/3)	577.9	489.5	445.9	334.7	327.4	246.0
hoons (3/4)	1354.6	1717.3	63.0	64.6	49.8	53.9
manol-pipe (27/34)	7747.7	9319.4	552.2	430.6	152.6	122.5
mizh (1/3)	2695.8	2165.3	2063.0	1201.2	723.3	411.5
narain (0/3)	1297.3	2700.0	139.9	-	79.4	-
schup (2/3)	1678.3	2057.7	529.5	414.3	368.5	299.5
simon (3/4)	1714.2	1386.4	1668.3	995.9	1174.2	738.3
stric-bmc (1/1)	6.7	10.2	26.2	31.3	10.8	14.0
vange (1/1)	11.4	13.2	367.6	399.7	60.4	55.7
velev (22/34)	13857.3	14979.3	665.2	412.7	110.7	77.4
TOTAL (69/98)	32808.5	36902.5	716.2	548.2	278.4	244.9

Table 4.9: *StaGe* + Implicate Online (IO) on SAT-Race benchmarks.

The SATLIB benchmarks in Table 4.10 are solved by *on-line* approach as the other implementations. Adopting the on-line approach, the statistics are slightly lower than those ones of *StaGe*.

Comparison

From the experiments described the implicate techniques worsen the performances of our solver *StaGe*. The idea of this techniques to promote the conflict in short clauses is not a winning solution. The graphs in tables 4.5, 4.6, 4.7, and 4.8 show the evolution of the various versions of the Implicate feature compared with *StaGe*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IO	<i>StaGe</i>	+IO	<i>StaGe</i>	+IO
AIS (4/4)	0.1	0.5	1.6	6.3	1.2	5.4
BF (4/4)	0.0	0.0	0.4	0.3	0.1	0.1
blocksworld (7/7)	1.0	1.2	1.4	1.5	0.9	1.0
BMC (13/13)	18.5	28.4	11.7	14.4	4.0	5.7
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	4.6	63.1	59.3	42.6	41.6
ind-inference (41/41)	0.3	0.8	0.5	0.9	0.1	0.4
logistics (4/4)	0.1	0.1	1.5	2.1	0.3	0.5
parity (20/30)	9003.1	9001.9	5.5	3.4	5.1	3.1
phole (5/5)	84.3	92.0	2648.3	2639.8	2250.5	2259.6
pret (8/8)	0.0	0.0	1.5	1.6	0.6	0.6
QG (22/22)	11.7	17.1	6.0	7.6	5.2	6.7
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.1
SW100 (100/100)	0.0	0.2	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12746.8	40.9	41.0	34.4	34.7

Table 4.10: *StaGe* + Implicate Online (IO) on SATLIB benchmarks.

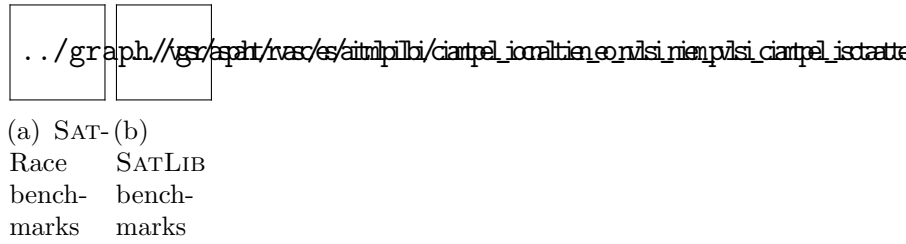


Figure 4.5: Comparison between *StaGe* + Implicate Online and *StaGe* + Implicate-Static on (a) SAT-Race and (b) SATLIB benchmarks.

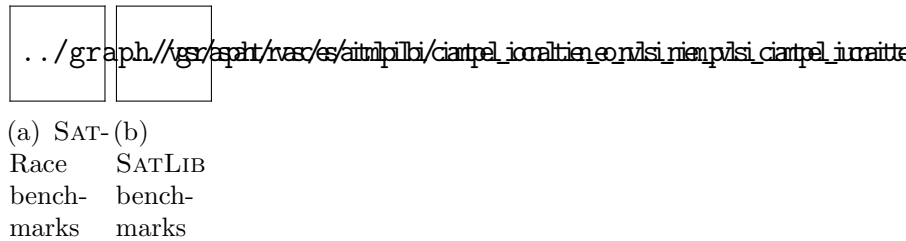


Figure 4.6: Comparison between *StaGe* + Implicate Online and *StaGe* + Implicate-Unit on (a) SAT-Race and (b) SATLIB benchmarks.

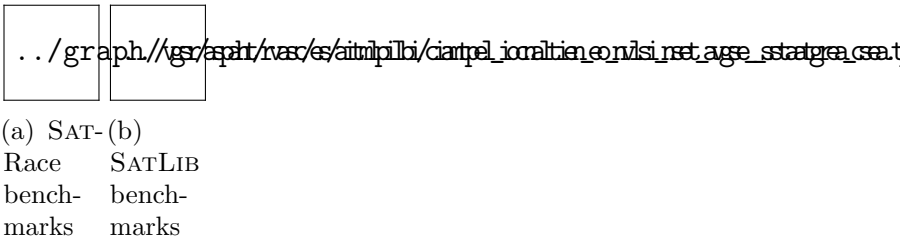


Figure 4.7: Comparison between *StaGe* + Implicate Online and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

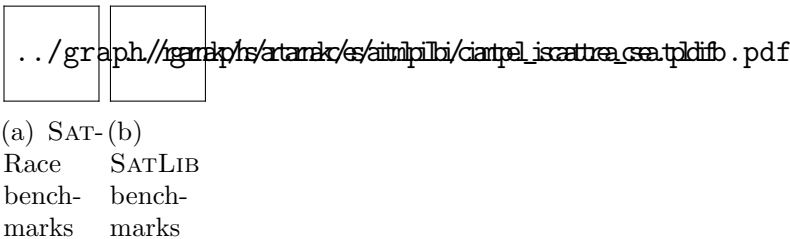


Figure 4.8: Comparison between *StaGe* and the different versions of *StaGe* + Implicate on (a) SAT-Race and (b) SATLIB benchmarks.

4.3 Monotone Literal Fixing

Monotone literal fixing (MLF) and learning are well-known look-ahead mechanisms in propositional satisfiability (SAT). *StaGe* introduces an implementation derived from the work of [11]. The interaction between MLF and learning is far from being obvious, and it poses non-trivial questions about both the detection and the propagation of monotone literals. The complications arise from the presence of learned clauses, and are related to the question about whether learned clauses have to be considered or not during the detection and/or propagation of monotone literals.

4.3.1 Techniques description

In the original DPLL algorithms the pure literal detection feature is included unlike the most modern SAT solvers that do not perform this technique. In many experiments it seems that this technique does not outperform the efficiency of the resolution. In some cases, seems that the results are worse.

MLF is the technique to detect and manage the *pure literals* in SAT instances. A literal is pure when its negation is not present. So a variable is present only with one type of literal. This means that it is possible to assign this literal without problems of conflicts. Indeed, to individuate a conflict it is necessary that the assigned literal is involved in the creation of an empty clause. This is not possible because the assigned literal has not its negation in the formula. Considering this aspect, the technique is not so difficult to implement. At the reality, there are some questions that represent the problems to adopting the MLF to solve SAT instances.

First of all, the present of learning determines that the input instance is augmented by the clauses stored while backtracking. The first question is: what is the definition of pure literal in the presence of learned clauses?

As described in [11] for the QBF² formula, in theory it is necessary detecting monotone literals in all clauses including learnt ones. In this situation it is obviously to keep track of the deleted learned clauses for each pure literal assigned. Thus, introducing MLF is bound to increase the time needed to assign a literal l , and the increase is proportional to the number of clauses that contain l . Moreover, even using a lazy data structure to detect pure literals it is necessary to maintain, for each variable v , an index to all clauses

²*Quantified Boolean Formula*, for further information see <http://www.qbflib.org/>.

that contain v and its negation. The index must be updated every time a new clause is learned, scanned each time it is searched a new clauses to watch, and scanned when it is deleted a learned clause. Therefore, this MLF algorithm suffers an overhead depending on the number of learned clauses and, in theory, it may became exponential bigger. For this reason the detection of monotone literals is performed for only the *static* clauses (i.e. the clauses of the original instance). This is correct because the learned clauses can be safely discarded without altering the satisfactory of the original SAT instance.

The second question is connected to fact that if the learned clauses are not considered, the sets of pure literals detected by MLF are the same ones? The learning strategy adds new clauses to the database and it is obviously that if a literal is pure in the static clauses, it is not guaranteed that this literal is still pure considering the learned clauses too. But, as said before, it is not necessary considering the learned clauses.

The most important question is if it is possible that a reason obtained from a conflict containing a monotone literals. The answer is the consequence of the previous question. A pure literal can be appear as not pure in learned clauses. The strategy to adopt to manage the monotone literals appearing in reasons and/or conflict clauses is not to delete it from these clauses. Indeed deleting (or ignoring) a literal from a clause changes the solver instance itself and it would cause the solver to report an incorrect solution. The right procedure is to ignore the conflicts caused by the learned clauses where a pure literal appears. Whit this strategy the overhead required to detect the monotone literal is kept down and it is not compromise the correctness of the instance.

4.3.2 Implementation

In *StaGe*, the MLF feature is developed with the idea of the *watching structure*. It's an evolution of the idea of the two watching literals scheme adopted to perform the boolean constraint propagation.

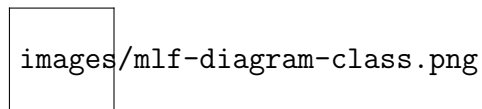


Figure 4.9: Diagram of MLF class.

As the other classes, the entry point of the MLF one is the *run()* method. This is called by *Solver* to compute the MLF policy when the BCP procedure ends without conflicts. In addition, it is also computed during the *unit* propagation at decision level 0. Two watching structures are used: one for binaries (*binary_watched*), and one for *N*-aries (*nary_watched*). With these structures, for each literal, it is maintained the trace of what clauses they watch. It is not used a queue to store temporary the literal to propagate. Like in the BCP and *Unit* implementations, an index (*db.index_4_mlf*) is utilized and it is scanned the stack vector without removing the elements from it. For each literal three methods are executed. The *nary_watched*

Algorithm 4.11 MLF, *run()* method.

```

1: repeat
2:    $stack \Rightarrow_{index\_4\_mlf} l$ 
3:   SUBSUME(l)
4:   SEARCH_BINARIES(l)
5:   SEARCH_NARIES()
6: until db.index_4_mlf  $\neq$  EOF stack
```

vector contains the index of the watched instances. A watched is a container that point a specific clause. It contains the literals that satisfied this specific clause. In the *subsume()* method, it is set the literal *lit* to satisfy all that

Algorithm 4.12 Watched struct.

```

1: id, index
2: subsume, lit
3: pointer, Clause
```

clauses watched by *lit* itself that are not satisfied yet. If a clause is satisfied by *lit* the index (*id*) of the watched instance is added to the *nary_buffer* to perform the detection of new pure literals. If this method is called in *Unit* at decision level 0 all the references to this watched index are removed and the clause, trivially satisfied, is deleted from the memory. The detection of monotone literals is computed with the methods *search_binaries()* and *search_naries()*. In *search_binaries()*, the *binary_watched* structure is scanned. This structure contains the literals of the binary clauses watched by *lit*. If one of these literals is FREE, it is moved to the *binary_watched* structure of the first non-TRUE literal of *binary_watched[lit]*. If this literal is not found

Algorithm 4.13 MLF, subsume() method.

Require: literal l to propagate

```

1: for all watched index  $w$  of  $l$  do
2:   if  $w.subsume$  is  $\neg$  true then
3:      $w.subsume \leftarrow l$ 
4:      $nary\_buffer \leftarrow w.id$ 
5:     if UNIT propagation then
6:        $c \leftarrow w.pointer$ 
7:       remove  $w$  from every literal  $l \in c$ 
8:       delete  $c$  from memory
9:     end if
10:  end if
11: end for
12: if UNIT propagation then
13:   delete from memory binary watched index vector of  $l$ 
14: end if

```

Algorithm 4.14 MLF, search_binaries() method.

Require: literal lit to propagate

```

1: for all literal  $l$  in  $binary\_watched[lit]$  do
2:   if  $l$  is Free then
3:     search literal  $s \neg$  true in  $binary\_watched[l]$ 
4:     if  $s$  is found then
5:       move  $l$  to  $binary\_watched[s]$ 
6:     else
7:       search nary watched clause  $c$  not satisfied
8:       if  $c$  is found then
9:         move  $l$  to  $nary\_watched[db.naries[c.id]]$ 
10:      else
11:         $db.SET\_MLF(not(l), is\_with\_unit)$ 
12:      end if
13:    end if
14:  end if
15: end for
16: if UNIT propagation then
17:   delete from memory binary watched index vector of  $lit$ 
18: end if

```

the procedure looks the *nary_watched* structure in order to find a clause already non-satisfied. If there is at least one clause the literal l is moved to the *nary_watched* of this clause. If there is not a non-FREE literal to watch the literal l it means that l is a pure literal and it is assigned in the stack with the Database method *set_mlf()*. The *search_naries()* procedure computes

Algorithm 4.15 MLF, *search_naries()*, method.

```

1: while nary_buffer is not empty do
2:   nary_buffer  $\leftarrow id$ 
3:   for all literal  $l$  of nary_watched[ $id$ ] do
4:     if  $l$  is Free then
5:       repeat
6:         do nothing
7:       until literal  $s \neg$  true in binary_watched[ $l$ ]
8:       if  $s$  is found then
9:         move  $l$  from binary_watched[ $s$ ]
10:      else
11:        repeat
12:          do nothing
13:        until nary_watched clause  $c$  not satisfied
14:        if  $c$  is found then
15:          moves  $l$  to nary_watched[db.naries[ $c.id$ ]]
16:        else
17:          db.SET_MLF( $\neg l$ )
18:        end if
19:      end if
20:    end if
21:  end for
22:  if UNIT propagation then
23:    nary_watched[ $id$ ].DEALLOC()
24:  end if
25: end while
26: clear nary_buffer

```

the same strategy for the *search_binaries()* structure. The *nary_buffer* vector contains the clauses that are satisfied by the method *subsume()*. So a literal of these clauses can be pure. In *nary_buffer* is stored the index of a watched structure. For each watched index are performed exactly the same operations of *search_binaries()*.

The MLF approach requires substantial modifications of the *StaGe* code.

The policy section of Solver checks the value of the MLF parameters. The

Algorithm 4.16 Database, MLF policy.

```

1: if MLF enabled and  $db.num\_conflicts\_from\_mlf \geq db.mlf\_limit$  then
2:    $mlf.RUN()$ 
3:    $db.num\_conflicts\_from\_mlf \leftarrow 0$ 
4:    $db.mlf\_limit \leftarrow db.mlf\_limit \times db.implicate\_inc$ 
5: end if

```

limit is set considering different experiments. The default value imposed is 100, and for the increase limit is 1,1. Due to implicate feature, MLF is not periodically disabled, but it is executed only every n number of conflicts.

It is necessary initializing the watched structure. For this reason it is developed the *init()* method that it is called, if MLF is enabled, in the beginning of the *run()* Solver method. With this method, every literal is

Algorithm 4.17 MLF, *init()* method.

```

1: {setting MLF data members}
2: for all clause  $c$  do
3:   for all literal  $l \in c$  do
4:     if  $l$  is not inserted then
5:       insert  $l$  in nary_watched of  $c$ 
6:     end if
7:   end for
8: end for
9: for all literal  $l$  do
10:  for all literal  $s$  in  $db.binaries[l]$  do
11:    if  $s$  is not inserted then
12:      insert  $s$  in binary_watched of  $l$ 
13:    end if
14:  end for
15: end for

```

inserted in only one arbitrary watched clause. It is not important what clause is watched by the literal.

The most important modifications about *StaGe* to insert MLF feature regard BCP and Backtracking. In BCP it is necessary to ignore the empty clauses that contain at least one pure literal. This problem regards only the learned clauses. For this reason the binary clauses are not considered in these

Algorithm 4.18 BCP MLF management code of propagate().

Require: current clause c

```

1: if MLF enabled and  $c.\text{Is\_LEARNT}()$  then
2:   check if a literal is pure
3: end if

```

modifications. To detect if a literal is pure there is, in Database, a boolean variable called *is_mlf_var* that indicates if a variable is pure or not. This is set on *set_mlf()* procedure. If there is a monotone literal the clause is ignored with the boolean variable *has_mlf*. Like this, it is simple to find a reason with a MLF literal.

In Backjumping, it is only necessary ignoring the assigned pure literals. The code shown in Algorithm 4.19 gets the next assigned literal from the

Algorithm 4.19 Backjumping, MLF management.

```

1: repeat
2:   repeat
3:      $\text{db.stack\_assign} \Rightarrow_{\text{index\_stack}} l$ 
4:      $\text{DEC}(\text{index\_stack})$ 
5:   until  $\text{db.seen}[\text{VAR}(l)]$  is true
6:    $\text{db.seen}[\text{VAR}(l)] \leftarrow \text{false}$ 
7: until  $\text{db.is\_mlf\_var}[\text{VAR}(l)]$  is false

```

stack to perform the conflict driven clause learning scheme. The monotone literals are ignored with the *is_mlf_var* data member.

4.3.3 Benchmarking

The MLF feature is tested with two different approaches: one with the detection of MLF literal every time the BCP ends without conflicts (called *MLF-Always*), and one with the policy previously described (called *MLF-Online*).

The results with SAT-Race benchmarks in Tables 4.11 and 4.12 show that the MLF approach requires more execution time for the detection of pure literals and this affects the performances.

The real differences between the two approaches compare with the velev and manol-pipe instance families. In this case the statistics of MLF-Online

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+MA	<i>StaGe</i>	+MA	<i>StaGe</i>	+MA
aloul (1/1)	65.0	93.5	8154.8	8446.2	7083.3	7548.3
een (3/3)	14.1	27.3	49.2	52.3	12.2	13.5
goldb (2/4)	1788.3	2005.3	1467.1	916.2	269.5	130.5
grieu (2/3)	577.9	1292.4	445.9	331.4	327.4	241.4
hoons (3/4)	1354.6	1798.8	63.0	63.0	49.8	49.8
manol-pipe (28/34)	7747.7	10246.4	552.2	499.4	152.6	127.5
mizh (0/3)	2695.8	2700.0	2063.0	-	723.3	-
narain (2/3)	1297.3	2379.9	139.9	9.6	79.4	2.5
schup (2/3)	1678.3	2432.9	529.5	447.2	368.5	308.8
simon (3/4)	1714.2	2303.9	1668.3	1668.3	1174.2	1174.2
stric-bmc (1/1)	6.7	11.5	26.2	27.6	10.8	11.2
vange (1/1)	11.4	34.3	367.6	400.5	60.4	75.8
velev (16/34)	13857.3	18199.1	665.2	191.8	110.7	57.8
TOTAL (64/98)	32808.5	43525.2	716.2	541.9	278.4	268.9

Table 4.11: *StaGe* + MLF-Always (MA) on SAT-Race benchmarks.

are significantly better, and it is comparable to the results of *BerkMin*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+MO	<i>StaGe</i>	+MO	<i>StaGe</i>	+MO
aloul (1/1)	65.0	87.5	8154.8	8168.5	7083.3	7094.4
een (3/3)	14.1	14.9	49.2	43.6	12.2	10.3
goldb (3/4)	1788.3	1881.8	1467.1	1467.1	269.5	269.5
grieu (3/3)	577.9	988.0	445.9	445.9	327.4	327.4
hoons (3/4)	1354.6	1727.4	63.0	63.0	49.8	49.8
manol-pipe (28/34)	7747.7	8854.6	552.2	527.7	152.6	139.3
mizh (0/3)	2695.8	2700.0	2063.0	-	723.3	-
narain (2/3)	1297.3	2054.5	139.9	9.6	79.4	2.5
schup (2/3)	1678.3	2551.8	529.5	601.6	368.5	422.0
simon (3/4)	1714.2	2094.7	1668.3	1668.3	1174.2	1174.2
stric-bmc (1/1)	6.7	7.8	26.2	26.5	10.8	11.3
vange (1/1)	11.4	13.7	367.6	367.6	60.4	60.4
velev (20/34)	13857.3	15827.9	665.2	368.0	110.7	78.1
TOTAL (70/98)	32808.5	38804.5	716.2	614.1	278.4	271.0

Table 4.12: *StaGe* + MLF-Online (MO) on SAT-Race benchmarks.

The MLF-Always approach require more time to determine the pure literals and this affects both on CPU-time required that the number of instances resolved. Consequently, the approach MLF-Online behaves better because the search for pure literal does not happen every time but only at regular intervals.

The results with the SATLIB benchmarks in Tables 4.13 and 4.14 show no significant changes compared to the standard version of *StaGe*. However, it is presented a slightly worsening about the CPU-time. Also in this case the MLF-Online approach has better statistics than the MLF-Always one.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+MA	<i>StaGe</i>	+MA	<i>StaGe</i>	+MA
AIS (4/4)	0.1	0.1	1.6	1.6	1.2	1.2
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	1.8	1.4	1.4	0.9	0.9
BMC (13/13)	18.5	40.0	11.7	15.4	4.0	5.5
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	7.4	63.1	73.9	42.6	50.3
ind-inference (41/41)	0.3	0.5	0.5	0.3	0.1	0.2
logistics (4/4)	0.1	0.1	1.5	1.5	0.3	0.3
parity (20/30)	9003.1	9003.7	5.5	5.5	5.1	5.1
phole (5/5)	84.3	69.2	2648.3	2027.8	2250.5	1784.7
pret (8/8)	0.0	0.0	1.5	1.5	0.6	0.6
QG (22/22)	11.7	24.3	6.0	6.0	5.2	5.2
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.1
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12747.4	40.9	31.9	34.4	27.6

Table 4.13: *StaGe* + MLF-Always (MA) on SATLIB benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+MO	<i>StaGe</i>	+MO	<i>StaGe</i>	+MO
AIS (4/4)	0.1	0.1	1.6	1.6	1.2	1.2
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	1.1	1.4	1.4	0.9	0.9
BMC (13/13)	18.5	27.7	11.7	14.3	4.0	5.5
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	5.9	63.1	63.1	42.6	42.6
ind-inference (41/41)	0.3	0.3	0.5	0.5	0.1	0.1
logistics (4/4)	0.1	0.1	1.5	1.5	0.3	0.3
parity (20/30)	9003.1	9003.2	5.5	5.5	5.1	5.1
phole (5/5)	84.3	91.3	2648.3	2636.3	2250.5	2240.2
pret (8/8)	0.0	0.0	1.5	1.5	0.6	0.6
QG (22/22)	11.7	13.7	6.0	6.0	5.2	5.2
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12743.6	40.9	40.8	34.4	34.3

Table 4.14: *StaGe* + MLF-Online (MO) on SATLIB benchmarks.

Like *Implicate* technique, the MLF approach seems to have no significantly impact on solving SAT instances. In many problems, it increases only the total time required to search the solution without improving the statistics. Indeed in many cases, the statistics between the two approaches are very similar.

Comparison

Figures 4.10, 4.11, 4.12 show the comparison between the two MLF approaches and the standard version of *StoGe*. The MLF-Online version has better results in terms of CPU-time than the Always version. In the charts in Figure 4.12 are compared together the MLF-Online, MLF-Always, and *StoGe* version. As previously discussed, MLF does not improve the performance of our solver *StoGe* and it is unsuitable for the use in the Boolean Satisfiability problems. As future work, it is possible to try to increase the performances with different algorithms that do not ignore the clauses with pure literals but adopt different learning strategies.

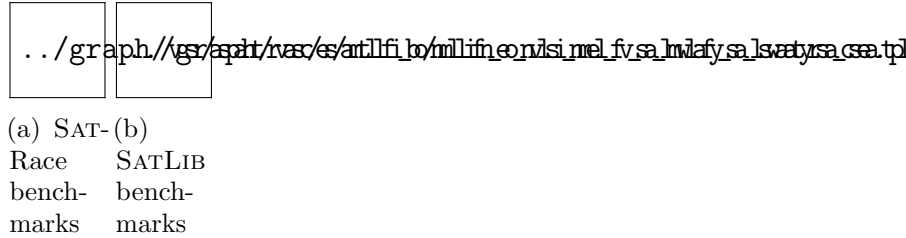


Figure 4.10: Comparison between *StoGe* + MLF Online and *StoGe* + MLF Always on (a) SAT-Race and (b) SATLIB benchmarks.

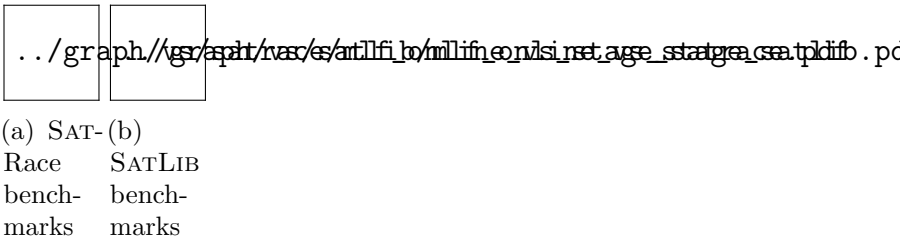


Figure 4.11: Comparison between *StaGe* + MLF Online and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

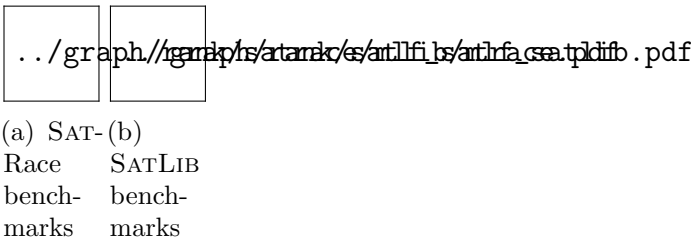


Figure 4.12: Comparison between *StaGe* and the different version of *StaGe* + MLF on (a) SAT-Race and (b) SATLIB benchmarks.

4.4 Score Initialization

In the state-of-art SAT-solvers as *MiniSat* and *RSat*, the variables are ordered by their activities in a specific structure (*heap*) used by heuristic to select the new literal to assign. Initially the variable activities are zeroed and they are inserted in the head without a specific ordering. Thus, the variables are inserted by the natural ordering. The first variable chosen is, always, 1 or -1 , depending on specific implementation.

As described in Chapter 2, *MiniMarch* introduces the concept of score initialization inspired by the work developed in the *Actin* solver [9]. This strategy is known as *Activity* or *Score initialization*.

The activity initialization used in *Actin*, computes, for each variable v , a raw initial activity $a_{raw}(v)$:

$$a_{raw}(v) = \sum_{\substack{\text{clauses } C, \\ v \in C \\ \vee \neg v \in C}} \sum_{\substack{l \in C, \\ l \neq v \\ \wedge l \neq \neg v}} \text{varcount}(l) \quad (4.1)$$

where $\text{varcount}(l)$ is the number of occurrences of the literal l in the CNF. The $a_{raw}(v)$ are normalized by finding the largest raw initial activity $a_{raw,max}$ and dividing all $a_{raw}(v)$ by it, yielding the actual initial activity $a_0(v)$ of each variable v :

$$a_0(v) = \frac{a_{raw}(v)}{a_{raw,max}} \rightarrow a_0(v) \in [0, 1] \quad (4.2)$$

In *StaGe* this algorithm is implemented with the Database method *si()* (Score Initialization). It is called during the SAT-instance reading. The *var_score* member represents the activity variable. The variables are ordered with this parameter through the *filter()* method. Instead, *var_count* member is computed during the CNF reading. In the procedure *read()*, for each non-unary clause is updated the score of every presented literal.

4.4.1 Benchmarking

This technique has performances comparable with *BerkMin*. The results of the SAT-Race benchmarks in Tables 4.15 show that there is a significant deterioration in terms of number of instances resolved and execution time required than the standard version of *StaGe*. The greatest difficulties of this

Algorithm 4.20 Database, si() method.

```

1:  $max \leftarrow 0$ 
2: for all clause  $c$  do
3:    $sum \leftarrow 0$ 
4:   for all  $l \in c$  do
5:      $sum \leftarrow sum + var\_count[VAR(l)]$ 
6:   end for
7:   for all  $l \in c$  do
8:      $var\_score[VAR(l)] \leftarrow var\_score[VAR(l)] + sum - var\_count[VAR(l)]$ 

9:   if  $var\_score[VAR(l)] > max$  then
10:     $max \leftarrow var\_score[VAR(l)]$ 
11:   end if
12: end for
13: end for
14: for all variable  $v$  do
15:    $var\_score[v] \leftarrow var\_score[v] / max$ 
16: end for
17:  $variable\_order.FILTER()$ 

```

Algorithm 4.21 Database, activity initialization.

Require: vector $lits$ containing the binary clauses read

```

1: if SI enabled then
2:    $INC(var\_count[VAR(lits[0])])$ 
3:    $INC(var\_count[VAR(lits[1])])$ 
4: end if
5: ...

```

Require: vector $lits$ containing the nary clauses read

```

6: if SI enabled then
7:   for all literal  $l$  in  $lits$  do
8:      $INC(var\_count[VAR(l)])$ 
9:   end for
10: end if

```

feature are with the manol-pipe and velev family instances. In particular the majority of instances not resolved occurs with this last instance family. These results lead to the view that the score initialization feature is not suitable to solve industrial benchmarks, in particular those instances derived from the pipe-lined machine verification (as the manol-pipe and velev family instances).

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+SI	<i>StaGe</i>	+SI	<i>StaGe</i>	+SI
aloul (1/1)	65.0	74.8	8154.8	8154.8	7083.3	7083.3
een (3/3)	14.1	24.4	49.2	65.3	12.2	18.6
goldb (3/4)	1788.3	1917.3	1467.1	1472.3	269.5	263.0
grieu (3/3)	577.9	937.4	445.9	448.0	327.4	324.8
hoons (3/4)	1354.6	1466.2	63.0	65.4	49.8	52.5
manol-pipe (28/34)	7747.7	7984.7	552.2	437.7	152.6	109.5
mizh (2/3)	2695.8	2142.9	2063.0	1768.1	723.3	482.6
narain (2/3)	1297.3	1907.0	139.9	11.5	79.4	2.8
schup (2/3)	1678.3	1901.9	529.5	381.9	368.5	262.7
simon (2/4)	1714.2	2352.5	1668.3	866.3	1174.2	309.1
stric-bmc (1/1)	6.7	7.8	26.2	30.5	10.8	12.0
vange (1/1)	11.4	5.6	367.6	149.1	60.4	16.8
velev (19/34)	13857.3	16247.4	665.2	320.5	110.7	50.8
TOTAL (70/98)	32808.5	36969.9	716.2	555.5	278.4	217.7

Table 4.15: *StaGe* + SI on SAT-Race benchmarks.

In Table 4.16 are shown the results with SATLIB benchmarks. Our solver *StaGe* with the SI feature solves the same number of instances of the other solvers tested. Also in this case there is a worsening about the CPU-time required.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+SI	<i>StaGe</i>	+SI	<i>StaGe</i>	+SI
AIS (4/4)	0.1	0.1	1.6	1.5	1.2	1.3
BF (4/4)	0.0	0.0	0.4	0.3	0.1	0.1
blocksworld (7/7)	1.0	0.7	1.4	1.1	0.9	0.6
BMC (13/13)	18.5	16.3	11.7	10.0	4.0	3.4
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	2.6	63.1	40.8	42.6	26.1
ind-inference (41/41)	0.3	1.2	0.5	1.0	0.1	0.6
logistics (4/4)	0.1	0.1	1.5	2.0	0.3	0.3
parity (20/30)	9003.1	9001.9	5.5	3.3	5.1	3.1
phole (5/5)	84.3	94.1	2648.3	2648.3	2250.5	2250.5
pret (8/8)	0.0	0.0	1.5	1.5	0.6	0.6
QG (22/22)	11.7	14.2	6.0	6.6	5.2	5.7
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.1
SW100 (100/100)	0.0	0.2	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12731.6	40.9	40.7	34.4	34.2

Table 4.16: *StaGe* + SI on SATLIB benchmarks.

Ultimately, the use of score initialization (SI) feature does not lead to increase in benefits than the basic version of *StaGe*. Especially with the instances selected by SAT-Race 2006, it is more significant the worsening of performances.

Comparison

The graphs in Figures 4.13 and 4.14 summarize the progress of *StaGe* with the score initialization enabled compared to the basic version. With the industrial instances of SAT-Race 2006 it is evident a worsening performance in terms of CPU-time while those ones selected by SATLIB the differences are less pronounced.

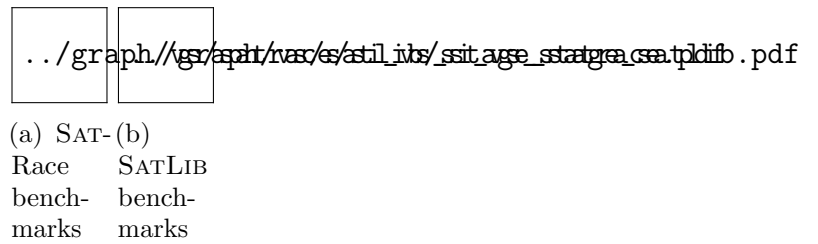


Figure 4.13: Comparison between *StaGe* + SI and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

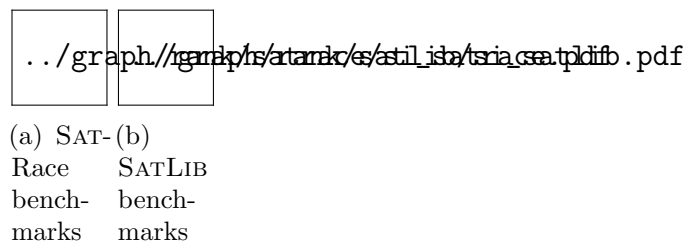


Figure 4.14: Comparison between *StaGe* and *StaGe* + SI on (a) SAT-Race and (b) SATLIB benchmarks.

4.5 Expensive Conflict Clause Minimization

The expensive conflict clause minimization is one of the most important new techniques adopted in the modern SAT solvers. In particular, this feature is used in the last version of *MiniSat*, and *RSat*. *MiniSat* does not natively implement this version and it is necessary to compile the source code with specific parameters. Instead, in *RSat*, this feature is enabled by default.

The standard method to remove redundant literals from the new learned conflict clauses is described in Chapter 3 in the Backjumping section. Instead this method adopts a new approach to check if it is possible to remove a useless literal from the conflict clause.

In [16] it is described the main technique to perform the conflict clause minimization. It refers to the concept of *self-subsumption*. A clause c is self-subsumed by a clause c_1 if the resolution on a common variable v shows that: $c \times_v c_1 \subseteq c$. In the conflict clause it consists to check if at least one reason of such literal is self-subsumed by the conflict clause itself.

The expensive technique requires the computation of a hashing code for each literal. Then the conflict clause is represented with a combination of the hashing code of every literals that contains. In theory, it is not possible for two clauses to have the same code because they have not all the same literal, but in the *MiniSat* implementation this is not guaranteed.

It is possible to remove a literal if it is not a decision literal and if all the variables of its reason are already visited during the BCP procedure. This means that the reason of this literal is self-subsumed by the conflict clause and thus the literal is redundant and it is possible to remove it.

As described in Chapter 2, this feature increases the performance of the *MiniSat* standard version. In *StaGe*, the results are not the same ones. It seems to decrease the performances of our solver. Maybe, this is derived from the fact that this implementation it was developed specifically for *MiniSat*, and *StaGe* uses additional and different features that can negatively influence the results.

4.5.1 Implementation

The *StaGe* implementation follows that one adopted in *MiniSat*. The procedure to compute the hashing code for each variable is called *abstract_level(var)*.

In practice, each literal has a code that indicates the own decision level.

Algorithm 4.22 Backjumping, `abstract_level()` variable method.

Require: variable v to compute the abstract level

1: **return** $1 \gg (\text{db.level}[v] \& 31)$

Algorithm 4.23 Backjumping, `abstract_level()` clause method.

1: `abstract` $\leftarrow 0$
2: **for all** variable v in `CONFLICT CLAUSE` **do**
3: `abstract` $\leftarrow \text{abstract} \mid \text{abstract_level}(v)$
4: **end for**
5: **return** `abstract`

Only the decision levels lower than 31 are considered while the ones greater than this number are cut. For example, if a variable has decision level 33, only the first 5 bits are considered. Therefore, 33 is equal to 1, 34 to 2, etc. . . Then this decoded number indicates the position of the bit 1. The variable with the decision level 5 has an abstract level equal to $2^5 = 32$, 20 to $2^{20} = 1048576$, etc. . . . It is obvious the reason to consider only the 5 least significant bits of the decision level. The abstract level is stored in an integer number of 32 bits and at most 31 shifts are possible.

The `abstract_level` of the conflict clause is computed as *bitwise or* from the abstract level of each variable which appears in the conflict clause itself. The computed number represents an index where each bit equals to 1 indicates that there is at least one variable/literal of the correspondent decision level. To compute if a literal is redundant or not, it is used the *analyze_stack* vector. The considered literal is inserted into this stack. So, for each literal presented in the stack, it is scanned the correspondent reason. If every variables of the reason are already “seen” then the literal *lit* is removable. Instead if at least one variable of the reason is not seen, is checked, in turn, its reason. Calling this variable r , if its reason is not NULL it is performed the comparison between the abstract level of r and the one of the conflict clause. The operation used is a *bitwise logic and*. In this way if the decision level of r is comprised in the conflict clause, r is added to the *analyzed_stack* to continue the check. Obviously, there is an approximation owing to the bit shrinking above described.

Algorithm 4.24 Backjumping, removable() method.

Require: literal l to check if it is redundant

Require: abstract, abstract level of CONFLICT CLAUSE

Require: temp *analyze_stack* vector to save the literals to check

```

1: analyze_stack  $\leftarrow l$ 
2: while analyze_stack is not empty do
3:   analyze_stack  $\Rightarrow l$ 
4:    $r \leftarrow$  reason of  $\text{VAR}(l)$ 
5:   for all literal  $s$  in  $r$  do
6:     if  $\text{VAR}(r)$  is  $\neg$  SEEN then
7:       if  $\text{reason}[\text{VAR}(r)] \neq \text{NULL}$  and  $\text{ABSTRACT\_LEVEL}(\text{VAR}(r)) \&$ 
         abstract then
8:         set  $\text{VAR}(s)$  as SEEN
9:         analyze_stack  $\leftarrow s$ 
10:      else
11:        return false
12:      end if
13:    end if
14:  end for
15: end while
16: return true

```

4.5.2 Benchmarking

In *StaGe*, the results of the benchmarks with the expensive conflict clause minimization (ECCMIN) enabled does not show substantial improvements. In Table 4.17, the results of the SAT-Race benchmarks are higher than those ones obtained with the score initialization (SI) feature in the previous section. In addition, in this case there is an improvement in terms of number of instances resolved and execution time required. We are still far from the performance of the standard version of *StaGe*. Even with this feature, it is difficult to resolve industrial instances derived from the pipe-lined machine verification. Indeed the major problems derive from the velev instance family.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+ECCMIN	<i>StaGe</i>	+ECCMIN	<i>StaGe</i>	+ECCMIN
aloul (1/1)	65.0	109.7	8154.8	9369.4	7083.3	8153.8
een (3/3)	14.1	16.2	49.2	53.7	12.2	12.1
goldb (2/4)	1788.3	1911.3	1467.1	976.0	269.5	143.6
grieu (3/3)	577.9	857.9	445.9	550.9	327.4	405.6
hoons (3/4)	1354.6	1468.3	63.0	40.0	49.8	32.3
manol-pipe (29/34)	7747.7	8149.5	552.2	473.4	152.6	120.5
mizh (1/3)	2695.8	2019.8	2063.0	1030.9	723.3	269.5
narain (3/3)	1297.3	1732.4	139.9	100.8	79.4	51.6
schup (2/3)	1678.3	1800.3	529.5	358.6	368.5	243.6
simon (3/4)	1714.2	2177.7	1668.3	1952.0	1174.2	1444.2
stric-bmc (1/1)	6.7	5.1	26.2	20.4	10.8	8.0
vange (1/1)	11.4	14.9	367.6	444.0	60.4	66.3
velev (20/34)	13857.3	16401.5	665.2	362.0	110.7	72.7
TOTAL (72/98)	32808.5	36664.5	716.2	591.6	278.4	278.6

Table 4.17: *StaGe* + ECCMIN on SAT-Race benchmarks.

As usual, the SATLIB benchmarks is solved with no problems. Table 4.18 shows more or less the same statistics of the standard conflict clause minimization.

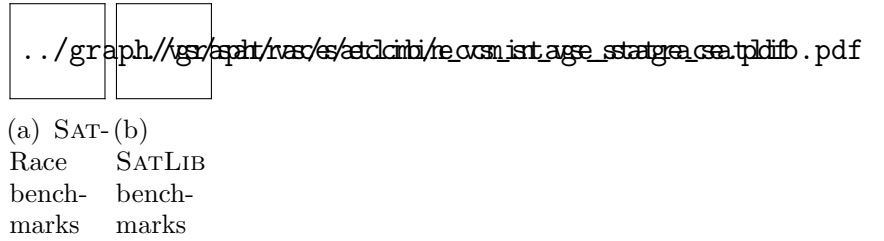
The ECCMIN feature seems do not influence the efficiency of the *StaGe* solver. In *MiniSat*, this technique increases the number of solved instances requiring lower execution time. Maybe, the structures in *StaGe* are not adapt to this feature. Indeed it is important to remember that the structure used is inspired by the one used in *Siege* that does not perform any minimization, even the standard minimization. A future work could be to analyse the possible case adopting a different implementation or different algorithm. In practice, for *StaGe*, ECCMIN is not a good solution. Therefore, it is reasonable to remain with the standard CCMIN adopted in the original *StaGe* version.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+ECCMIN	<i>StaGe</i>	+ECCMIN	<i>StaGe</i>	+ECCMIN
AIS (4/4)	0.1	0.0	1.6	0.8	1.2	0.6
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	0.7	1.4	1.1	0.9	0.6
BMC (13/13)	18.5	17.8	11.7	11.5	4.0	3.9
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	3.4	63.1	44.1	42.6	28.8
ind-inference (41/41)	0.3	0.9	0.5	0.9	0.1	0.4
logistics (4/4)	0.1	0.1	1.5	1.9	0.3	0.4
parity (20/30)	9003.1	9002.1	5.5	3.3	5.1	3.1
phole (5/5)	84.3	96.5	2648.3	2178.0	2250.5	1846.4
pret (8/8)	0.0	0.0	1.5	1.5	0.6	0.5
QG (22/22)	11.7	16.9	6.0	6.7	5.2	5.8
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12738.5	40.9	33.8	34.4	28.3

Table 4.18: *StaGe* + ECCMIN on SATLIB benchmarks.

Comparison

In Figures 4.15 and 4.16 are shown the comparisons between *StaGe* with the ECCMIN feature enabled and *StaGe* with the standard clause conflict minimization. The graphs show more or less the same situation of the SI (score initialization) feature previously discussed. With the instances selected by SAT-Race 2006, the differences arise mainly for instances that require more time for the resolution. Instead with the instances selected by SATLIB the trend is more or less the same one without ECCMIN enabled.

Figure 4.15: Comparison between *StaGe* + ECCMIN and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

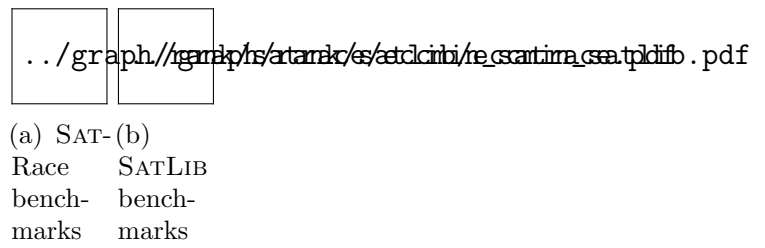


Figure 4.16: Comparison between *StaGe* and *StaGe* + ECCMIN on (a) SAT-Race and (b) SATLIB benchmarks.

4.6 Progress Saving

The *progress saving* technique was introduced in *RSat* by the work of [10]. It consists to memorize for each variable the last assigned literal. In this way the next time a variable is selected from heuristic, the literal is not chosen with a specific strategy (for example, randomly) but it is used the last saved sign of the variable.

In the opinion of the authors of [10] and the ones of *RSat*, this strategy reducing the work repetition caused by non-chronological backtracking commonly practised by DPLL-based SAT-solvers. In particular, it seems that this technique reveals significant improvements on a broad range of industrial instances.

At the reality, it is not possible to compute the real improvements derived from the utilization of progress saving. In *RSat*, during the search, this technique is periodically disabled to avoid the risks to search in the same space. This is the main point: how to set the value to enable and disable periodically this feature? The answer at this question consists on an empirical setting of the value based on different experiments performed by the author of *RSat*. As other empirical results, it is not guaranteed that this feature performs good performances with every structure and each other possible strategy. Indeed, in *StaGe*, this feature works better in combination with score initialization strategy than alone. The best our experiments with this feature are performed with the original parameters set by *RSat*. In addition, it is important to remember that *RSat* utilizes a different approach to perform the BCP procedure. It is based on ordered propagation where the literals are not propagated with the traditional method but according to their activities.

4.6.1 Implementations

To implement this feature it used the same algorithm developed in *RSat*. Even the parameters are the same. As above described, it seems that these parameters perform the best results. In Database it is stored the *ps_saved* vector that contains, for each variable, the last assigned literal.

The other parameters are the same used in *RSat*, and are described in the Database section of Chapter 3. The most important parameters are: *ps_limit_on* and *ps_limit_off* that are respectively set 100 and 400. The assigned literal is saved in the *cancel()* method during the loop to unassign

the variables until to the new current decision level. The cancel method is described in the Database section of Chapter 3 too.

In Solver, when the BCP procedure ends without conflicts is executed the Ps Policy. In Heuristics, the next decision literal is chosen based on what stored on *ps_saved* member. Instead, if the variable is not already assigned, it is executed the strategy described in Chapter 3.

Algorithm 4.25 Solver, PS policy.

```

1: if Ps enabled and  $\text{db.num\_conflicts\_from\_ps} \geq \text{db.ps\_limit}$  then
2:   if  $\text{db.ps\_on} = \text{true}$  then
3:      $\text{db.ps\_limit} \leftarrow \text{db.ps\_limit\_on}$ 
4:   else
5:      $\text{db.ps\_limit} \leftarrow \text{db.ps\_limit\_off}$ 
6:   end if
7:    $\text{db.ps\_on} \leftarrow \neg \text{db.ps\_on}$ 
8:    $\text{db.num\_conflicts\_from\_ps} \leftarrow 0$ 
9: end if

```

4.6.2 Benchmarking

With the SAT-Race benchmarks in Table 4.19, Ps enabled version of *StaGe* has performances better the the two previous features analyzed. Compared to other feature, the progress saving technique suffers with the instances derived from the pipe-lined machine verification, like the manol-pipe and velev instance families. However, this technique does not improve the performance of our solver *StaGe*. These results below those one obtained with the standard versions of *StaGe* are comparable with *Siege*. Obviously, also in this case the worsening concerns the number of instances resolved and the CPU-time required.

The Ps feature solves the same number of the SATLIB instances of the other solvers. As described in Table 4.20, it requires more time to solve the instances than the standard version of *StaGe*.

By experiments, the Ps technique does not seem to improve the performance of the solver *StaGe*. However, this does not mean that the feature can be useful in some situations. Indeed in Chapter 5 we will describe the combination of the Ps and Si techniques that outperform the results of *StaGe*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+PS	<i>StaGe</i>	+PS	<i>StaGe</i>	+PS
aloul (1/1)	65.0	189.8	8154.8	19941.8	7083.3	16279.3
een (3/3)	14.1	8.7	49.2	34.6	12.2	7.2
goldb (2/4)	1788.3	1915.1	1467.1	1743.8	269.5	160.1
grieu (3/3)	577.9	214.6	445.9	256.1	327.4	148.8
hoons (3/4)	1354.6	1628.5	63.0	76.7	49.8	57.9
manol-pipe (29/34)	7747.7	7236.9	552.2	362.9	152.6	109.2
mizh (1/3)	2695.8	2194.7	2063.0	2514.0	723.3	482.2
narain (2/3)	1297.3	1868.7	139.9	19.3	79.4	2.2
schup (1/3)	1678.3	2411.9	529.5	525.7	368.5	319.2
simon (3/4)	1714.2	1892.5	1668.3	2050.7	1174.2	1515.6
stric-bmc (1/1)	6.7	3.9	26.2	26.3	10.8	7.1
vange (1/1)	11.4	3.2	367.6	137.7	60.4	9.7
velev (22/34)	13857.3	13182.6	665.2	346.5	110.7	63.7
TOTAL (72/98)	32808.5	32751.1	716.2	723.3	278.4	377.5

Table 4.19: *StaGe* + PS on SAT-Race benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+PS	<i>StaGe</i>	+PS	<i>StaGe</i>	+PS
AIS (4/4)	0.1	0.1	1.6	1.1	1.2	0.8
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	0.8	1.4	1.5	0.9	0.8
BMC (13/13)	18.5	19.9	11.7	16.1	4.0	4.6
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	4.5	63.1	61.4	42.6	39.0
ind-inference (41/41)	0.3	2.5	0.5	2.8	0.1	1.2
logistics (4/4)	0.1	0.1	1.5	1.6	0.3	0.2
parity (20/30)	9003.1	9003.2	5.5	5.5	5.1	5.0
phole (5/5)	84.3	197.5	2648.3	5729.1	2250.5	4530.7
pret (8/8)	0.0	0.0	1.5	1.4	0.6	0.5
QG (22/22)	11.7	10.4	6.0	5.2	5.2	4.4
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.2	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12839.3	40.9	86.9	34.4	68.2

Table 4.20: *StaGe* + PS on SATLIB benchmarks.

Comparison

The technique PS is not able to solve the same number of instances of the standard version of *StaGe*. However, as shown in Figures 4.17 and 4.18, with the instances selected by SAT-Race 2006 it appears clearly visible the improvements in terms of execution time. In the other instances selected by SATLIB, the results are very similar to the other features, a further confirmation of the relative ease with which they resolve this type of instances. To remember, of course, that in the SATLIB bechmarks, no solver is able to solve at least one instances of the GCP family.

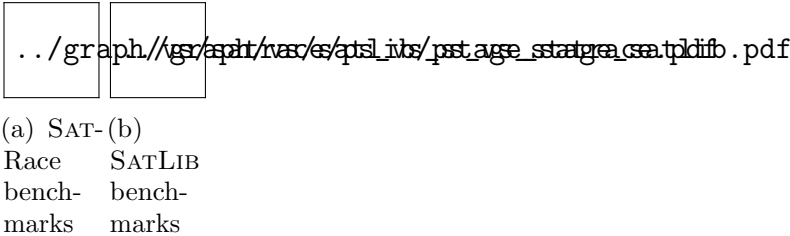


Figure 4.17: Comparison between *StaGe* + PS and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

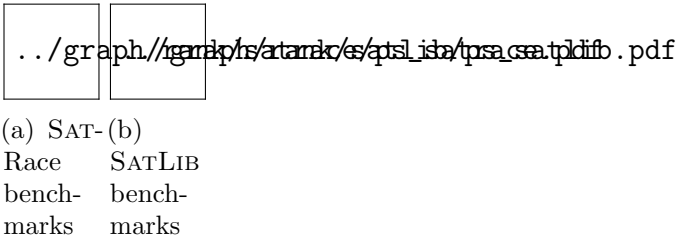


Figure 4.18: Comparison between *StaGe* and *StaGe* + PS on (a) SAT-Race and (b) SATLIB benchmarks.

4.7 Look-ahead Technique

As described in Chapter 2, the *look-ahead* mechanism is introduced by *MiniMarch*. This technique assigns the best literal of a chosen variable. To compute the most appropriate literal, it is checked if there is a conflict or not and the number of propagations. The heuristic chooses the next variable and consequently the next literal. Then the solver assigns it. If there is a conflict the look-ahead technique is not executed and the search continues normally. Instead, if there are not conflicts, it is erased the current decision level and it is assigned the negation of the last chosen literal. Now, if there is a conflict, the search continues with the backjumping procedure. If a conflict is not found they are compared the number of propagations done by the two literals (that are connected to the variable chosen by heuristic). It is reassigned the literal with the greater number of propagations.

This method consents to assign an higher number of variables in order to increase the search resolution. Unfortunately, it consists an expensive technique that requires, when used, to assign at least one literal that does not influence the search. Indeed, to compute the best literal for the chosen variable, it is necessary to test both ones.

4.7.1 FLD

A different approach, introduced in our solver *StaGe*, it is to make less expensive the look-ahead mechanism. For this reason it is developed the *flip last decision* (FLD) feature. The main problem of LHT is that if the first assigned literal is the best (considering the number of propagations), it is necessary to assign its negation and then reassigning the literal.

FLD does not require to reassign the first literal. It is flipped the chosen literal only in the case in which there are not conflicts or propagations. So there are two different solutions. In *StaGe*, it is possible to set the unit of measurements more appropriate to the specific instance. The two different options are: *conflict* (*conf*) or *propagation* (*prop*).

In our implementation these two different approaches share the same structures and parameters. Obviously, it is not possible to enable the two features (FLD and LHT) at the same time.

4.7.2 Implementation

As usual, the parameters used are saved in the Database class.

<i>lht_enabled</i>	bool
<i>lht_decide</i>	bool
<i>lht_ref</i>	uint
<i>lht_value</i>	uint

Table 4.21: Database, LHT parameters.

The *fld_on* parameter indicates if the FLD or LHT techniques are active. Each technique is periodically activated with the same policy used for the other features.

<i>fld_enabled</i>	bool
<i>fld_on</i>	bool
<i>fld_param</i>	bool pointer
<i>fld_limit</i>	uint
<i>fld_limit_off</i>	uint
<i>fld_limit_on</i>	uint
<i>num_conflicts_from_fld</i>	uint

Table 4.22: Database, FLD parameters.

LHT considers only the number of propagations as parameter. Instead FLD is set to count as parameter what it is chosen from the command line. The *fld_param* member is a pointer to the selected parameter (*has_propagated* or *has_conflict*). At the start, it is necessary to choose the next literal with the standard procedure. The chosen literal is propagated with the BCP function. If there is a conflict the variable *db.has_conflict* is set TRUE. If at least one variable is assigned during the propagation the member *db.has_propagated* is set TRUE. If FLD is enabled the variable *fld_param* points to one of these two parameters. So, if there are no conflicts or no propagations, it is executed the third conditional block (from line 7 to 9 of Algorithm 4.26). Then it is performed the *fld()* function that cancels that last decision level and assigns the negation of the last chosen literal.

Instead if LHT is enabled it is necessary counting the number of propagations in order to compare the two literals. It is used the *fld()* method to assign the

Algorithm 4.26 Solver, decision procedure with FLD and LHT features.

```

1: if db.lht_decide = true and db.has_conflict  $\neg$  false and
   db.stack_assign.SIZE() - db.lht_ref < db.lht_value then
2:   db.FLD()
3:   db.lht_decide  $\leftarrow$  false
4: else if db.decision_level = 0 or db.fld_on = false or db.has_flipped =
   true or *db.fld_param = true then
5:   standard decision
6: else
7:   db.lht_value  $\leftarrow$  db.stack_assign.SIZE() - db.lht_ref
8:   db.FLD()
9:   db.lht_decide  $\leftarrow$  db.lht_enabled
10: end if

```

other literal at the same mode of FLD feature. In addition, it is used an index (*lht_ref*) and a counter (*lht_value*) to compute the number of propagations. It is obvious that the propagations are saved in the stacks. So with an index that points to the position of the last assigned literal by heuristic it is possible to count the number of assigned variables through BCP and this value is saved in the *lht_value* data member. When it is necessary choosing the best literal this information is compared to the one of the other literal (first condition, Algorithm 4.26).

The *fld()* method cancels the last decision level and reset all the parameters. The *set_index()* method updates the index to perform the BCP procedure. These methods are expensive to compute. For this reason they are executed

Algorithm 4.27 Database, fld() method.

```

1: DEC(decision_level)
2: l  $\leftarrow$  CANCEL()
3: INC(decision_level)
4: has_conflict  $\leftarrow$  false
5: has_propagated  $\leftarrow$  false
6: has_flipped  $\leftarrow$  true
7: SET_INDEX()
8: SET( $\neg$ l)

```

with a specific policy. This is the same used for the other features. The value of the parameters are set with different empirical experiments. The procedure is exactly the same described for the Ps feature. The values of the parameters

fld_limit_on and fld_limit_off are, respectively, 200 and 100.

Algorithm 4.28 Database, FLD/LHT policy.

```

1: if FLD enabled and  $db.num\_conflicts\_from\_fld \geq db.fld\_limit$  then
2:   if  $db.fld\_on = \mathbf{true}$  then
3:      $db.fld\_limit \leftarrow db.fld\_limit\_on$ 
4:   else
5:      $db.fld\_limit \leftarrow db.fld\_limit\_off$ 
6:   end if
7:    $db.fld\_on \leftarrow \neg db.fld\_on$ 
8:    $db.num\_conflicts\_from\_fld \leftarrow 0$ 
9: end if

```

4.7.3 LHT benchmarking

We tested this technique in *StaGe*, to improve the search in order to reduce the amount of time spent in useless search spaces. The results are not satisfactory especially for the instances derived from the pipe-lined machine verification.

The results in Table 4.23 show that in this case there are not improvements compared to the results obtained with the standard version of *StaGe*. Unlike the PS feature, the most problems for the LHT feature are with the velev and manol-pipe benchmarks. In addition, the results are comparable to those ones obtained with the SI feature enabled.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+LHT	<i>StaGe</i>	+LHT	<i>StaGe</i>	+LHT
aloul (1/1)	65.0	82.5	8154.8	8009.9	7083.3	7502.7
een (3/3)	14.1	17.9	49.2	39.6	12.2	11.1
goldb (3/4)	1788.3	1845.8	1467.1	1227.3	269.5	276.1
grieu (3/3)	577.9	616.1	445.9	313.1	327.4	275.4
hoons (3/4)	1354.6	1620.1	63.0	50.9	49.8	43.1
manol-pipe (28/34)	7747.7	9402.9	552.2	462.0	152.6	127.6
mizh (2/3)	2695.8	1842.0	2063.0	1441.4	723.3	478.9
narain (3/3)	1297.3	1322.3	139.9	77.0	79.4	49.7
schup (2/3)	1678.3	2553.9	529.5	452.3	368.5	354.7
simon (3/4)	1714.2	1985.5	1668.3	1425.6	1174.2	1053.8
stric-bmc (1/1)	6.7	7.5	26.2	23.4	10.8	10.0
vange (1/1)	11.4	20.6	367.6	477.5	60.4	74.5
velev (19/34)	13857.3	16555.3	665.2	229.7	110.7	62.9
TOTAL (72/98)	32808.5	37872.4	716.2	541.7	278.4	265.9

Table 4.23: *StaGe* + LHT on SAT-Race benchmarks.

Also the LHT feature solves the same instances of the SATLIB benchmarks in Table 4.24. The GCP and a part of parity are very difficult to solve by all the solvers.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+LHT	<i>StaGe</i>	+LHT	<i>StaGe</i>	+LHT
AIS (4/4)	0.1	12.2	1.6	95.2	1.2	89.5
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	1.4	1.4	1.6	0.9	1.1
BMC (13/13)	18.5	17.8	11.7	9.0	4.0	3.1
flat (100/100)	0.0	0.1	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	4.3	63.1	45.9	42.6	34.5
ind-inference (41/41)	0.3	0.4	0.5	0.5	0.1	0.1
logistics (4/4)	0.1	0.1	1.5	1.3	0.3	0.3
parity (20/30)	9003.1	9003.4	5.5	5.5	5.1	5.2
phole (5/5)	84.3	70.1	2648.3	1692.4	2250.5	1568.3
pret (8/8)	0.0	0.0	1.5	1.3	0.6	0.5
QG (22/22)	11.7	14.2	6.0	5.8	5.2	5.5
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12724.2	40.9	27.7	34.4	25.3

Table 4.24: *StaGe* + LHT on SATLIB benchmarks.

Our experiments report that the LHT feature is not a solution to increase significantly the performances of the *StaGe* SAT-solver. For this reason it is performed the FLD feature that increases the efficiency of the search.

4.7.4 FLD benchmarking

The FLD feature is developed in *StaGe* with two different approaches. Both techniques, CONF and PROP, are tested in several experiments. In this section, it will be described the best results of every solution. The best results obtained are with the PROP solution that increases the performances of *StaGe* with the LHT feature.

CONF-type benchmarking

The technique FLD-Conf worsens considerably the performances of our solver *StaGe*. Table 4.25 shows this consideration. Only *zChaff* resolves a minor number of instances. In particular, the problems occur with the velev benchmark. This means that the FLD-Conf technique is not suitable for solving industrial instances derived from the pipe-lined machine verification.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+FC	<i>StaGe</i>	+FC	<i>StaGe</i>	+FC
aloul (1/1)	65.0	77.3	8154.8	8454.6	7083.3	7790.0
een (3/3)	14.1	39.7	49.2	78.3	12.2	23.7
goldb (2/4)	1788.3	1955.5	1467.1	909.7	269.5	137.5
grieu (3/3)	577.9	413.4	445.9	186.9	327.4	157.2
hoons (3/4)	1354.6	1594.1	63.0	76.6	49.8	61.5
manol-pipe (29/34)	7747.7	9417.1	552.2	522.8	152.6	132.9
mizh (0/3)	2695.8	2700.0	2063.0	-	723.3	-
narain (2/3)	1297.3	2164.9	139.9	9.0	79.4	2.6
schup (0/3)	1678.3	2700.0	529.5	-	368.5	-
simon (3/4)	1714.2	1982.6	1668.3	1517.7	1174.2	1077.0
stric-bmc (1/1)	6.7	9.0	26.2	27.7	10.8	12.7
vange (1/1)	11.4	19.5	367.6	467.2	60.4	67.8
velev (17/34)	13857.3	17473.0	665.2	244.8	110.7	70.6
TOTAL (65/98)	32808.5	40546.3	716.2	549.0	278.4	264.0

Table 4.25: *StaGe* + FLD-Conf (FC) on SAT-Race benchmarks.

With the SATLIB instances, the results are the same obtained with the LHT feature. There are slight improvements in terms of execution time.

The FLD feature with the CONF option worsens the results obtained with the LHT technique, above described. This probably means that the use of the number of conflicts as unit of measurements is not a good solution. Indeed with the PROP option, there is a substantial improvement of the performances. In the next section, the different results are described. The substantial improvements of the FLD feature in comparison of LHT concern the number of instances solved and the execution time required.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+FC	<i>StaGe</i>	+FC	<i>StaGe</i>	+FC
AIS (4/4)	0.1	0.1	1.6	0.9	1.2	0.7
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	1.0	1.4	1.1	0.9	0.8
BMC (13/13)	18.5	21.7	11.7	12.2	4.0	4.3
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	4.0	63.1	47.3	42.6	33.2
ind-inference (41/41)	0.3	0.7	0.5	0.8	0.1	0.3
logistics (4/4)	0.1	0.1	1.5	1.8	0.3	0.4
parity (20/30)	9003.1	9002.6	5.5	4.5	5.1	4.3
phole (5/5)	84.3	71.5	2648.3	2016.1	2250.5	1833.9
pret (8/8)	0.0	0.0	1.5	1.4	0.6	0.5
QG (22/22)	11.7	15.2	6.0	6.7	5.2	6.2
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12717.2	40.9	31.5	34.4	28.2

Table 4.26: *StaGe* + FLD-Conf (FC) on SATLIB benchmarks.

PROP-type benchmarking

The results with the instances selected by SAT-Race 2006 are shown in Table 4.27. They indicate an improvement compared to the FLD-Conf and LHT versions. Despite these improvements, the benefits are not comparable to the standard version of *StaGe*.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+FP	<i>StaGe</i>	+FP	<i>StaGe</i>	+FP
aloul (1/1)	65.0	67.1	8154.8	7323.3	7083.3	6804.4
een (3/3)	14.1	25.7	49.2	74.9	12.2	18.7
goldb (3/4)	1788.3	1836.5	1467.1	1350.9	269.5	265.3
grieu (3/3)	577.9	24.5	445.9	56.8	327.4	40.7
hoons (3/4)	1354.6	1622.9	63.0	86.3	49.8	68.6
manol-pipe (29/34)	7747.7	8456.2	552.2	509.1	152.6	131.2
mizh (1/3)	2695.8	2065.9	2063.0	1357.2	723.3	349.7
narain (2/3)	1297.3	1988.0	139.9	10.8	79.4	3.1
schup (1/3)	1678.3	2469.4	529.5	497.6	368.5	355.4
simon (3/4)	1714.2	2605.0	1668.3	2631.8	1174.2	2059.2
stric-bmc (1/1)	6.7	8.0	26.2	29.1	10.8	11.9
vange (1/1)	11.4	11.7	367.6	360.0	60.4	61.0
velev (22/34)	13857.3	16536.7	665.2	744.3	110.7	213.8
TOTAL (73/98)	32808.5	37717.6	716.2	730.5	278.4	321.3

Table 4.27: *StaGe* + FLD-Prop (FP) on SAT-Race benchmarks.

The SATLIB benchmarks are solved by FLD-Prop with a little worsening about the CPU-time. Also in this case the GCP and a part of parity instance family are not solved.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+FP	<i>StaGe</i>	+FP	<i>StaGe</i>	+FP
AIS (4/4)	0.1	1.9	1.6	19.6	1.2	18.2
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	0.7	1.4	1.2	0.9	0.7
BMC (13/13)	18.5	26.9	11.7	14.5	4.0	5.3
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	4.3	63.1	54.6	42.6	38.2
ind-inference (41/41)	0.3	0.5	0.5	0.6	0.1	0.2
logistics (4/4)	0.1	0.1	1.5	1.3	0.3	0.3
parity (20/30)	9003.1	9003.6	5.5	6.1	5.1	5.7
phole (5/5)	84.3	72.1	2648.3	2058.8	2250.5	1893.0
pret (8/8)	0.0	0.0	1.5	1.4	0.6	0.5
QG (22/22)	11.7	11.2	6.0	5.4	5.2	4.8
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12721.5	40.9	32.5	34.4	29.4

Table 4.28: *StaGe* + FLD-Prop (FP) on SATLIB benchmarks.

The results in Tables 4.27 and 4.28 show the difficulties encountered by the LHT and FLD techniques in two different versions: CONF and PROP. Using as a comparison the standard version of *StaGe*, the results are not good from the point of view of the number of instances solved and the CPU-time required.

Comparison

In Tables 4.19, 4.20, 4.21, 4.22 are shown the results of comparing the best version of this technique: FLD-Prop respectively with LHT, FLD-Conf and *StaGe*. For the instances selected by the SAT-Race 2006, FLD-Prop has better performance in terms of CPU-time compared to the version of FLD-Conf. Instead the performances are worse compared to the LHT feature and the standard version of *StaGe*. In Figure 4.21 are compared all the different variations of this technique with *StaGe*. As already described, none of these features increase the performance of the standard version of *StaGe*.

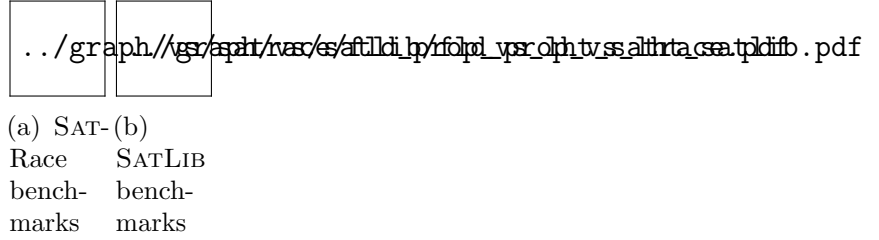


Figure 4.19: Comparison between *StaGe* + FLD-Prop. and *StaGe* + LHT on (a) SAT-Race and (b) SATLIB benchmarks.

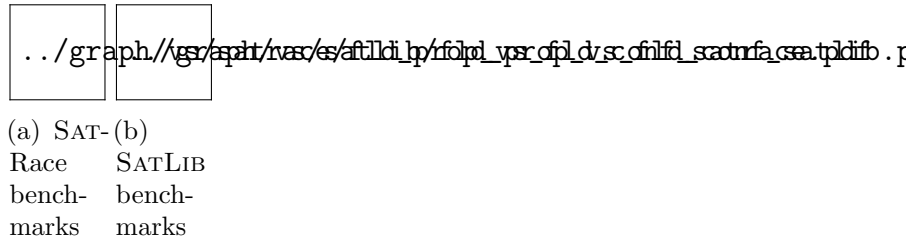


Figure 4.20: Comparison between *StaGe* + FLD-Prop. and *StaGe* + FLD-Conf. on (a) SAT-Race and (b) SATLIB benchmarks.

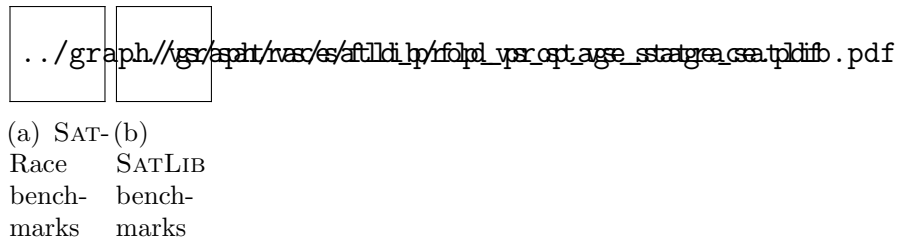


Figure 4.21: Comparison between *StaGe* + FLD-Prop. and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

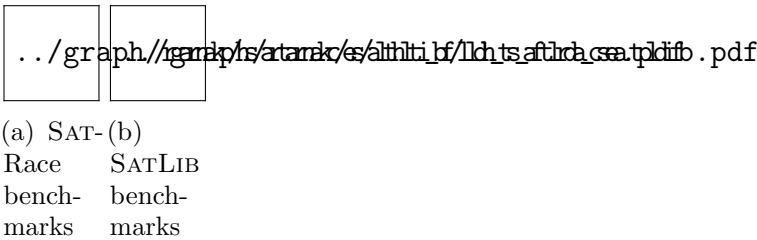


Figure 4.22: Comparison between *StaGe*, *StaGe* + LHT and the different versions of *StaGe* + FLD on (a) SAT-Race and (b) SATLIB benchmarks.

4.8 Priority Propagation

The last feature described is focused on the implementation of a different method to perform the BCP propagation. The standard method for the BCP procedure is to watch, for each literal, the clauses where its negation appears. In *StaGe*, the clauses are in two different structures: one for binary clauses, and one for the other N -ary clauses. So the standard BCP visits first the binary clauses and then the N -ary ones. The *Priority Propagation* (PP) technique consists in the assignment of a priority for the binary clauses. The propagation begins with standard breath search for only the binary clauses. Then, if there are not conflicts, a new breath search is performed for the N -ary clauses. This procedure is repeated until a conflict is found or all the literals are been propagated.

This solution is developed with the objective to find a conflict, if it exists, in the binary clauses in order to obtain conflict clauses of reduced size. This would be increase the performances of the search.

4.8.1 Implementation

In the *run()* method of the BCP class, there are two different strategies that are alternatively executed. The member *db.pp_on* indicates if the feature is

Algorithm 4.29 BCP, Priority Propagation code.

```

1: if db.pp_on = true then
2:   repeat
3:     perform BCP only on binaries
4:     perform BCP only on  $N$ -aries
5:   until exist at least 1 literal to propagate
6: else
7:   standard BCP
8: end if

```

active or not. If it is enabled, two breath-first search are executed. Each search is similar to the one described in Chapter 3 about BCP. The loop is necessary, because it is possible to assign a variable in N -ary BCP and then this variable has to be propagated in every two type of clauses, included the binary ones.

As the other feature, there is a specific policy to manage its use. The parameter values are 250 and 100, respectively, for *pp_limit_off* and *pp_limit_on*.

Algorithm 4.30 Solver, PP policy.

```

1: if PP enabled and  $\text{db.num\_conflicts\_from\_pp} \geq \text{db.pp\_limit}$  then
2:   if  $\text{db.pp\_on} = \text{true}$  then
3:      $\text{db.pp\_limit} \leftarrow \text{db.pp\_limit\_on}$ 
4:   else
5:      $\text{db.pp\_limit} \leftarrow \text{db.pp\_limit\_off}$ 
6:   end if
7:    $\text{db.pp\_on} \leftarrow \neg \text{db.pp\_on}$ 
8:    $\text{db.num\_conflicts\_from\_pp} \leftarrow 0$ 
9: end if

```

4.8.2 Benchmarking

The results report that the PP feature does not increase the performance of the search. In Table 4.29 are shown the results of the PP feature with the SAT-Race 2006 instances. Even with this technique it is evident the deterioration in terms of the number of instances solved and CPU-time required. The results are comparable to those obtained from the version with the feature SI enabled.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+PP	<i>StaGe</i>	+PP	<i>StaGe</i>	+PP
aloul (1/1)	65.0	79.5	8154.8	8041.6	7083.3	6994.9
een (3/3)	14.1	18.7	49.2	53.0	12.2	13.8
goldb (2/4)	1788.3	1949.0	1467.1	1051.7	269.5	175.9
grieu (2/3)	577.9	931.2	445.9	82.3	327.4	58.3
hoons (3/4)	1354.6	1576.2	63.0	85.5	49.8	65.7
manol-pipe (29/34)	7747.7	9570.4	552.2	690.5	152.6	167.7
mizh (1/3)	2695.8	2188.3	2063.0	2625.5	723.3	556.3
narain (2/3)	1297.3	2017.6	139.9	13.4	79.4	3.4
schup (0/3)	1678.3	2700.0	529.5	-	368.5	-
simon (2/4)	1714.2	2904.0	1668.3	2238.6	1174.2	1473.2
stric-bmc (1/1)	6.7	7.6	26.2	24.3	10.8	10.5
vange (1/1)	11.4	15.5	367.6	399.6	60.4	68.7
velev (22/34)	13857.3	14538.4	665.2	452.6	110.7	85.6
TOTAL (69/98)	32808.5	38496.4	716.2	699.4	278.4	261.4

Table 4.29: *StaGe* + PP on SAT-Race benchmarks.

In the SATLIB benchmarks (Table 4.30), the PP feature solves the usual instances with a worsening about the CPU-Time compared to the standard

StaGe case.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+PP	<i>StaGe</i>	+PP	<i>StaGe</i>	+PP
AIS (4/4)	0.1	0.4	1.6	4.9	1.2	4.1
BF (4/4)	0.0	0.0	0.4	0.4	0.1	0.1
blocksworld (7/7)	1.0	0.9	1.4	1.2	0.9	0.7
BMC (13/13)	18.5	23.5	11.7	14.3	4.0	4.8
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	5.1	63.1	52.5	42.6	35.7
ind-inference (41/41)	0.3	0.8	0.5	0.9	0.1	0.4
logistics (4/4)	0.1	0.1	1.5	1.5	0.3	0.3
parity (20/30)	9003.1	9004.2	5.5	6.5	5.1	6.0
phole (5/5)	84.3	93.1	2648.3	2510.0	2250.5	2132.5
pret (8/8)	0.0	0.0	1.5	1.5	0.6	0.6
QG (22/22)	11.7	22.6	6.0	7.6	5.2	6.7
SSA (8/8)	0.0	0.0	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12750.8	40.9	39.2	34.4	32.9

Table 4.30: *StaGe* + PP on SATLIB benchmarks.

In practice, the PP feature does not let to increase the efficiency of the *StaGe* solver.

Comparison

The graphs in Figures 4.23 and 4.24 show the non-potential of the PP feature with this type of instances. Unlike the Ps feature that gets better results with the instances of SAT-Race than *StaGe* from the point of view of the execution time, with the PP technique this is not possible. Instead the considerations for the instances of SATLIB follow the same ones of the other previous features.

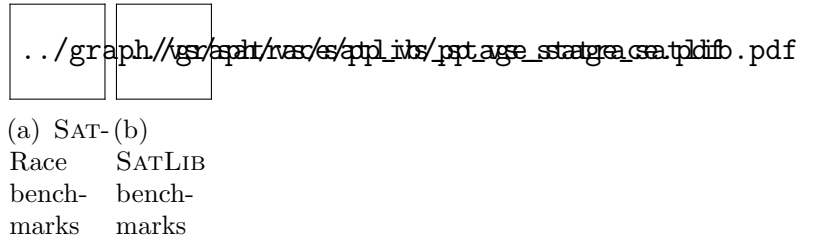


Figure 4.23: Comparison between *StaGe* + PP and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

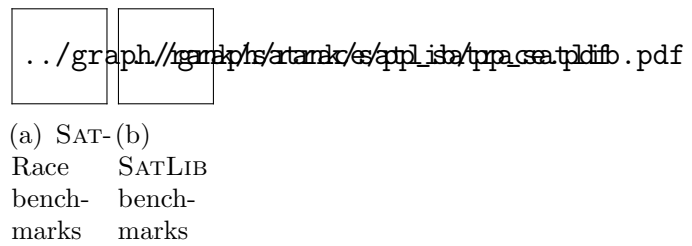


Figure 4.24: Comparison between *StaGe* and *StaGe + PP* on (a) SAT-Race and (b) SATLIB benchmarks.

Chapter 5

Conclusions

The different techniques described in Chapter 4 was tested with the benchmarks selected from SAT Race and SATLIB. The best results are obtained by using the PS and SI techniques. The various tests show that to achieve performance improvements it is necessary to implement specific policies that will enable and disable the different techniques during the search.

5.1 Technique combination

In several experiments made the best performances are obtained by the use of the Progress Saving and Score Initialization techniques. As described in Chapter 4, PS increases the performances solving as regard the standard *StaGe* version from the point of view of CPU-time. With the combination of these two techniques, it is further possible to improve this aspect. Moreover, the number of instances solved is greater than the versions with the single features enabled. These considerations are witnessed by the results of the SAT-Race benchmarks shown in Table 5.1. The most significant improvements occur with the manol-pipe instance family.

The results with the SATLIB benchmarks in Table 5.2 are the same obtained with the other solvers and the standard version of *StaGe*. Compared to this version there is a slightly worsening about the execution time required. Also in this case it is not possible to solve at least one instance of the GCP family.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+PS+SI	<i>StaGe</i>	+PS+SI	<i>StaGe</i>	+PS+SI
aloul (1/1)	65.0	188.6	8154.8	19941.8	7083.3	16279.3
een (3/3)	14.1	10.1	49.2	37.4	12.2	8.2
goldb (2/4)	1788.3	1910.4	1467.1	1662.9	269.5	142.4
grieu (3/3)	577.9	321.8	445.9	351.7	327.4	202.5
hoons (3/4)	1354.6	1748.0	63.0	117.8	49.8	87.8
manol-pipe (31/34)	7747.7	7008.3	552.2	493.7	152.6	161.5
mizh (2/3)	2695.8	1793.6	2063.0	2201.1	723.3	458.5
narain (3/3)	1297.3	1151.5	139.9	63.0	79.4	12.7
schup (2/3)	1678.3	2051.4	529.5	529.9	368.5	311.6
simon (3/4)	1714.2	2243.1	1668.3	2576.1	1174.2	2002.7
stric-bmc (1/1)	6.7	4.8	26.2	32.0	10.8	7.6
vange (1/1)	11.4	2.9	367.6	90.4	60.4	2.9
velev (22/34)	13857.3	12570.6	665.2	281.5	110.7	64.6
TOTAL (77/98)	32808.5	31005.2	716.2	776.5	278.4	408.9

Inst. family (#,S)	CPU-Time (s)			Decisions ($\times 10^3$)			Conflicts ($\times 10^3$)		
	+PS	+SI	+PS+SI	+PS	+SI	+PS+SI	+PS	+SI	+PS+SI
aloul (1/1)	189.8	74.8	188.6	19942	8155	19942	16279	7083	16279
een (3/3)	8.7	24.4	10.1	35	65	37	7	19	8
goldb (2/4)	1915.1	1917.3	1910.4	1744	1472	1663	160	263	142
grieu (3/3)	214.6	937.4	321.8	256	448	352	149	325	202
hoons (3/4)	1628.5	1466.2	1748.0	77	65	118	58	53	88
manol-pipe (31/34)	7236.9	7984.7	7008.3	363	438	494	109	110	162
mizh (2/3)	2194.7	2142.9	1793.6	2514	1768	2201	482	483	458
narain (3/3)	1868.7	1907.0	1151.5	19	12	63	2	3	13
schup (2/3)	2411.9	1901.9	2051.4	526	382	530	319	263	312
simon (3/4)	1892.5	2352.5	2243.1	2051	866	2576	1516	309	2003
stric-bmc (1/1)	3.9	7.8	4.8	26	31	32	7	12	8
vange (1/1)	3.2	5.6	2.9	138	149	90	10	17	3
velev (22/34)	13182.6	16247.4	12570.6	347	321	281	64	51	65
TOTAL (77/98)	32751.1	36969.9	31005.2	723	556	776	377	218	409

Table 5.1: *StaGe* + Ps + Si on SAT-Race benchmarks.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+PS+SI	<i>StaGe</i>	+PS+SI	<i>StaGe</i>	+PS+SI
AIS (4/4)	0.1	0.1	1.6	2.2	1.2	1.7
BF (4/4)	0.0	0.0	0.4	0.3	0.1	0.1
blocksworld (7/7)	1.0	0.3	1.4	0.5	0.9	0.3
BMC (13/13)	18.5	21.7	11.7	16.4	4.0	4.8
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	3.2	63.1	52.2	42.6	33.8
ind-inference (41/41)	0.3	0.6	0.5	0.8	0.1	0.3
logistics (4/4)	0.1	0.1	1.5	1.9	0.3	0.2
parity (20/30)	9003.1	9003.1	5.5	5.9	5.1	5.5
phole (5/5)	84.3	202.0	2648.3	5729.1	2250.5	4530.7
pret (8/8)	0.0	0.0	1.5	1.4	0.6	0.5
QG (22/22)	11.7	12.7	6.0	6.8	5.2	5.8
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.0
SW100 (100/100)	0.0	0.1	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12844.0	40.9	86.7	34.4	68.2

Inst. family (#,S)	CPU-Time (s)			Decisions ($\times 10^3$)			Conflicts ($\times 10^3$)		
	+PS	+SI	PS+SI	+PS	+SI	PS+SI	+PS	+SI	PS+SI
AIS (4/4)	0.1	0.1	0.1	1	2	2	1	1	2
BF (4/4)	0.0	0.0	0.0	0	0	0	0	0	0
blocksworld (7/7)	0.8	0.7	0.3	2	1	1	1	1	0
BMC (13/13)	19.9	16.3	21.7	16	10	16	5	3	5
flat (100/100)	0.0	0.0	0.0	0	0	0	0	0	0
GCP (0/4)	3600.0	3600.0	3600.0	-	-	-	-	-	-
hanoi (2/2)	4.5	2.6	3.2	61	41	52	39	26	34
ind-inference (41/41)	2.5	1.2	0.6	3	1	1	1	1	0
logistics (4/4)	0.1	0.1	0.1	2	2	2	0	0	0
parity (20/30)	9003.2	9001.9	9003.1	5	3	6	5	3	5
phole (5/5)	197.5	94.1	202.0	5729	2648	5729	4531	2250	4531
pret (8/8)	0.0	0.0	0.0	1	2	1	1	1	1
QG (22/22)	10.4	14.2	12.7	5	7	7	4	6	6
SSA (8/8)	0.0	0.0	0.1	0	0	0	0	0	0
SW100 (100/100)	0.2	0.2	0.1	0	0	0	0	0	0
TOTAL (338/352)	12839.3	12731.6	12844.0	87	41	87	68	34	68

Table 5.2: *StaGe* + PS + SI on SATLIB benchmarks.

Comparison

The charts in Tables 5.1, 5.2, 5.3 and 5.4 confirm that the combination of these two techniques is significantly better than the versions with each feature enabled and the standard one.

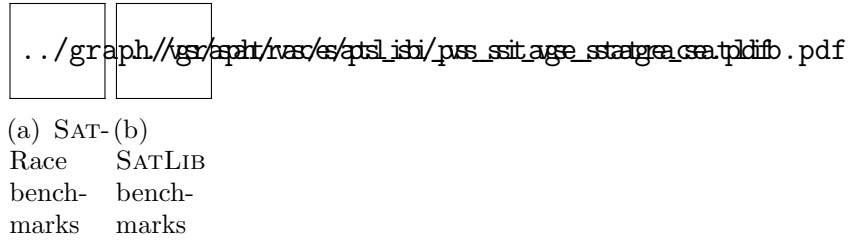


Figure 5.1: Comparison between *StaGe* + PS + SI and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

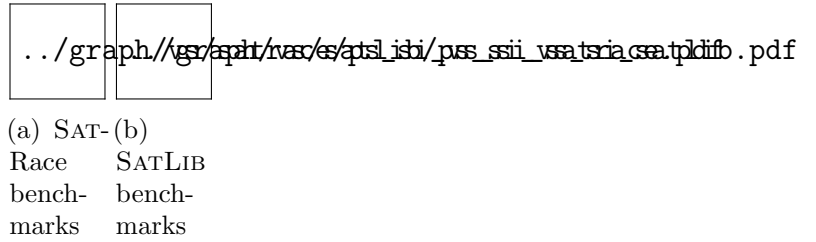


Figure 5.2: Comparison between *StaGe* + PS + SI and *StaGe* + SI on (a) SAT-Race and (b) SATLIB benchmarks.

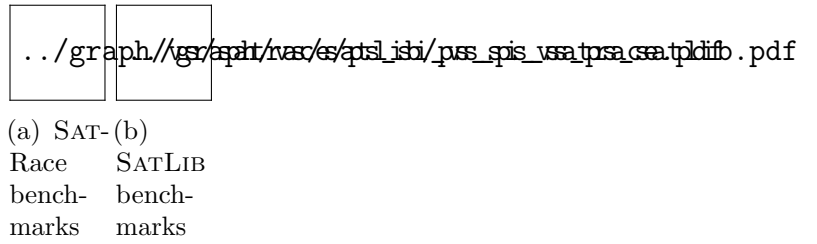


Figure 5.3: Comparison between *StaGe* + PS + SI and *StaGe* + PS on (a) SAT-Race and (b) SATLIB benchmarks.

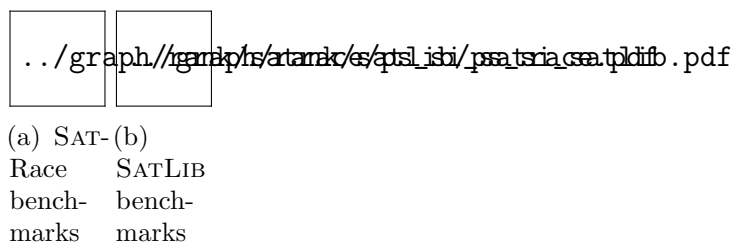


Figure 5.4: Comparison between *StaGe*, *StaGe* + PS, *StaGe* + SI, and *StaGe* + PS + SI on (a) SAT-Race and (b) SATLIB benchmarks.

Other important experiments were done to test the possibility of the combination of the Implicate and Ps features. Obviously, it was used the Implicate-Online version.

In the SAT-Race benchmarks (Table 5.3), the results of this combination are unsatisfying. The statistics are better than in the case with the single features Implicate and Ps enabled but this combination requires more time to solve the instances compared to the version with only Ps enabled.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IO+PS	<i>StaGe</i>	+IO+PS	<i>StaGe</i>	+IO+PS
aloul (1/1)	65.0	390.3	8154.8	47497.1	7083.3	36378.1
een (3/3)	14.1	226.6	49.2	1163.0	12.2	381.7
goldb (2/4)	1788.3	2008.7	1467.1	1808.6	269.5	302.8
grieu (2/3)	577.9	1459.5	445.9	526.6	327.4	314.6
hoons (3/4)	1354.6	1681.5	63.0	74.2	49.8	61.2
manol-pipe (30/34)	7747.7	7114.6	552.2	441.1	152.6	138.1
mizh (1/3)	2695.8	2259.8	2063.0	1288.9	723.3	456.6
narain (2/3)	1297.3	2116.3	139.9	16.2	79.4	3.2
schup (2/3)	1678.3	1932.6	529.5	456.5	368.5	280.0
simon (3/4)	1714.2	1628.2	1668.3	1544.4	1174.2	1154.7
stric-bmc (1/1)	6.7	9.3	26.2	35.1	10.8	13.4
vange (1/1)	11.4	3.8	367.6	148.1	60.4	11.0
velev (22/34)	13857.3	13719.9	665.2	479.3	110.7	131.3
TOTAL (73/98)	32808.5	34551.2	716.2	1187.8	278.4	691.6

Inst. family (#,S)	CPU-Time (s)			Decisions ($\times 10^3$)			Conflicts ($\times 10^3$)		
	+IO	+PS	+IO+PS	+IO	+PS	+IO+PS	+IO	+PS	+IO+PS
aloul (1/1)	74.3	189.8	390.3	8296	19942	47497	7222	16279	36378
een (3/3)	33.4	8.7	226.6	70	35	1163	21	7	382
goldb (2/4)	1956.4	1915.1	2008.7	987	1744	1809	205	160	303
grieu (2/3)	489.5	214.6	1459.5	335	256	527	246	149	315
hoons (3/4)	1717.3	1628.5	1681.5	65	77	74	54	58	61
manol-pipe (30/34)	9319.4	7236.9	7114.6	431	363	441	123	109	138
mizh (1/3)	2165.3	2194.7	2259.8	1201	2514	1289	412	482	457
narain (2/3)	2700.0	1868.7	2116.3	-	19	16	-	2	3
schup (2/3)	2057.7	2411.9	1932.6	414	526	456	300	319	280
simon (3/4)	1386.4	1892.5	1628.2	996	2051	1544	738	1516	1155
stric-bmc (1/1)	10.2	3.9	9.3	31	26	35	14	7	13
vange (1/1)	13.2	3.2	3.8	400	138	148	56	10	11
velev (22/34)	14979.3	13182.6	13719.9	413	347	479	77	64	131
TOTAL (73/98)	36902.5	32751.1	34551.2	548	723	1188	245	377	692

Table 5.3: *StaGe* + Implicate-Online (IO) + Ps on SAT-Race benchmarks.

The results with the SATLIB benchmarks (Table 5.4) follows the consideration derived from the previous experiments. The instances not solved are the same as above. Respect to the case with only the Implicate feature enabled there is a deterioration of performances about the execution time required.

Compared to the case with only Ps enabled there is a small improvement in terms of CPU-time instead.

Inst. family (#,S)	CPU-Time (s)		Decisions ($\times 10^3$)		Conflicts ($\times 10^3$)	
	<i>StaGe</i>	+IO+PS	<i>StaGe</i>	+IO+PS	<i>StaGe</i>	+IO+PS
AIS (4/4)	0.1	0.3	1.6	4.5	1.2	3.6
BF (4/4)	0.0	0.0	0.4	0.3	0.1	0.1
blocksworld (7/7)	1.0	1.0	1.4	1.5	0.9	0.9
BMC (13/13)	18.5	24.9	11.7	15.0	4.0	5.4
flat (100/100)	0.0	0.0	0.0	0.0	0.0	0.0
GCP (0/4)	3600.0	3600.0	-	-	-	-
hanoi (2/2)	5.3	2.3	63.1	40.2	42.6	25.0
ind-inference (41/41)	0.3	1.3	0.5	1.6	0.1	0.7
logistics (4/4)	0.1	0.1	1.5	2.0	0.3	0.4
parity (20/30)	9003.1	9003.9	5.5	6.8	5.1	6.4
phole (5/5)	84.3	145.2	2648.3	4472.0	2250.5	3590.8
pret (8/8)	0.0	0.0	1.5	1.5	0.6	0.6
QG (22/22)	11.7	20.6	6.0	8.6	5.2	7.5
SSA (8/8)	0.0	0.1	0.2	0.2	0.0	0.1
SW100 (100/100)	0.0	0.3	0.1	0.1	0.0	0.0
TOTAL (338/352)	12724.6	12800.1	40.9	68.3	34.4	54.5

Inst. family (#,S)	CPU-Time (s)			Decisions ($\times 10^3$)			Conflicts ($\times 10^3$)		
	+IO	+PS	+IO+PS	+IO	+PS	+IO+PS	+IO	+PS	+IO+PS
AIS (4/4)	0.5	0.1	0.3	6	1	4	5	1	4
BF (4/4)	0.0	0.0	0.0	0	0	0	0	0	0
blocksworld (7/7)	1.2	0.8	1.0	1	2	2	1	1	1
BMC (13/13)	28.4	19.9	24.9	14	16	15	6	5	5
flat (100/100)	0.0	0.0	0.0	0	0	0	0	0	0
GCP (0/4)	3600.0	3600.0	3600.0	-	-	-	-	-	-
hanoi (2/2)	4.6	4.5	2.3	59	61	40	42	39	25
ind-inference (41/41)	0.8	2.5	1.3	1	3	2	0	1	1
logistics (4/4)	0.1	0.1	0.1	2	2	2	0	0	0
parity (20/30)	9001.9	9003.2	9003.9	3	5	7	3	5	6
phole (5/5)	92.0	197.5	145.2	2640	5729	4472	2260	4531	3591
pret (8/8)	0.0	0.0	0.0	2	1	2	1	1	1
QG (22/22)	17.1	10.4	20.6	8	5	9	7	4	7
SSA (8/8)	0.1	0.0	0.1	0	0	0	0	0	0
SW100 (100/100)	0.2	0.2	0.3	0	0	0	0	0	0
TOTAL (338/352)	12746.8	12839.3	12800.1	41	87	68	35	68	55

Table 5.4: *StaGe* + Implicate-Online (IO) + PS on SATLIB benchmarks.

Comparison

The charts in Figures 5.5, 5.6, 5.7, 5.7 show the comparison between the combination of the Implicate and PS features, each feature enabled and the standard version of *StaGe*. This combination improves the performances of *StaGe* with only Implicate enabled. Compared to the PS feature, there is a slightly worsening in terms of CPU-time. As already mentioned, compared to

Conclusions

StaGe, it is needed more time to resolve the various instances which influences negatively the performances.

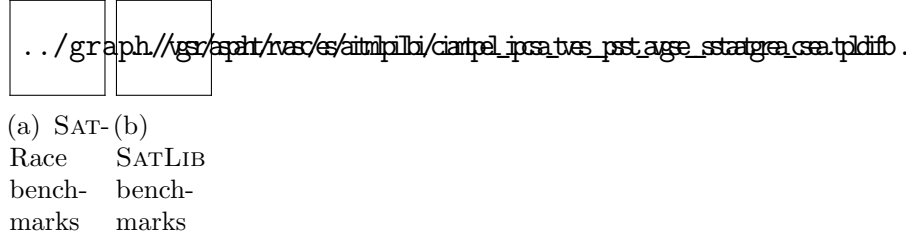


Figure 5.5: Comparison between *StaGe* + Implicate + Ps and *StaGe* on (a) SAT-Race and (b) SATLIB benchmarks.

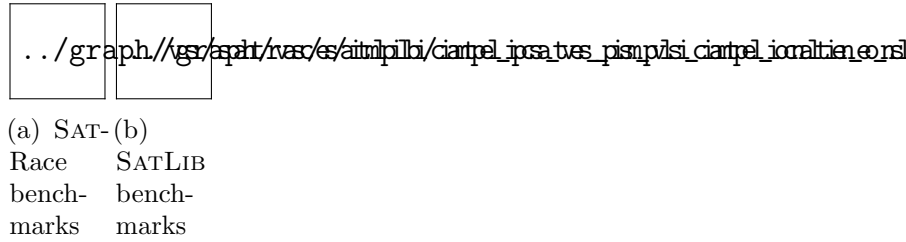


Figure 5.6: Comparison between *StaGe* + Implicate + Ps and *StaGe* + Implicate on (a) SAT-Race and (b) SATLIB benchmarks.

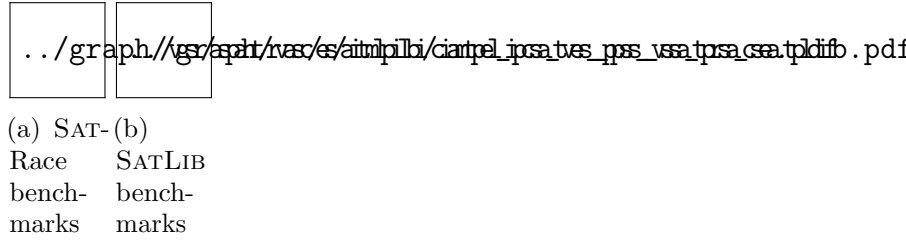


Figure 5.7: Comparison between *StaGe* + Implicate + Ps and *StaGe* + Ps on (a) SAT-Race and (b) SATLIB benchmarks.

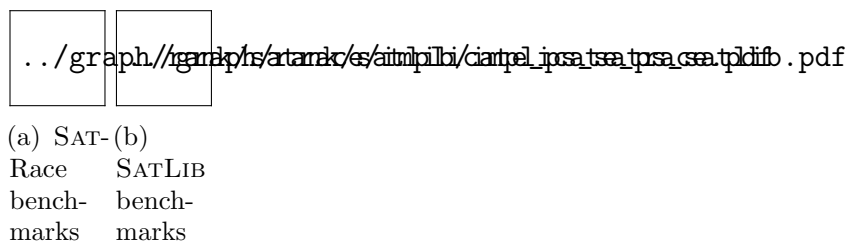


Figure 5.8: Comparison between *StaGe*, *StaGe* + Implicate, *StaGe* + Ps, and *StaGe* + Implicate + Ps on (a) SAT-Race and (b) SATLIB benchmarks.

The other tested experiments report that it is difficult mixing different features without optimizing the parameters for each specific technique. Note that the chosen parameters for the features are not optimized to a specific set of instances. They are set to solve many instances as possible and, secondly, to reduce the CPU-time spent. With these considerations it becomes complex to set the parameters to combine the techniques in order to solve the highest number of instances.

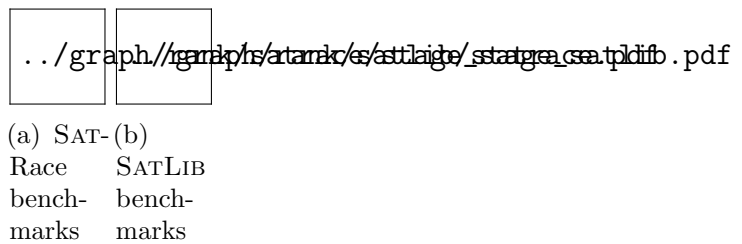


Figure 5.9: Comparison between the different version *StaGe* tested in this thesis on (a) SAT-Race and (b) SATLIB benchmarks.

In Table 5.5 are shown the results of the different SAT-solvers on SAT-Race benchmarks. *RSat* solves the greatest number of instances. Instead the combination of the PS and SI techniques solves the same number of instances of the standard version of *StaGe*. As previously described, there is an improvements about the total CPU-time required. The other techniques do not increase the performance of *StaGe*. Indeed Table 5.5 show a worsening in terms of number of instances solved.

The results on SATLIB Benchmarks do not reports meaningful improvements and, for this reason, they are not shown.

Solver	# Instances solved
<i>RSat</i>	79
<i>StaGe</i> + Ps + SI	77
<i>StaGe</i>	77
<i>MiniSat</i> -Simp.	77
<i>MiniMarch</i>	75
<i>MiniSat</i>	74
<i>StaGe</i> + FLD-Prop.	73
<i>StaGe</i> + Implicate + Ps	73
<i>StaGe</i> + Ps	72
<i>StaGe</i> + LHT	72
<i>StaGe</i> + ECCMIN	72
<i>BerkMin</i>	71
<i>StaGe</i> + SI	70
<i>StaGe</i> + MLF-Online	70
<i>StaGe</i> + Implicate-Online	69
<i>StaGe</i> + PP	69
<i>StaGe</i> + FLD-Conf.	65
<i>StaGe</i> + MLF-Always	64
<i>StaGe</i> + Implicate-Static	62
<i>StaGe</i> + Implicate-Unit	60
<i>zChaff</i>	60

Table 5.5: Number of SAT-Race instances solved by each solver tested.

5.2 Future work

In this thesis different techniques have been presented and tested in order to improve the performance of modern SAT-solvers. Moreover, the variation of performance with the combinations of the techniques above mentioned have been analyzed too. To this purpose a new SAT-solver called *StaGe* has been developed. It is comparable with the state-of-art SAT-solvers.

As future work it may be interesting to introduce an auto-tuning (AT) technique that lets to optimize the parameters in order to solve specific benchmarks or mixing different techniques. In this way, it would be possible to focus on the techniques that do not work well together and the ones that can improve the performance if working together. In addition, it would be possible to adapt the parameters during the search, changing it dynamically. Another important development would be the Reassignments feature. In theory, this technique consists to reassign a variable (obviously with the other sign) in order to subsume a clause that is currently empty. It is a complex feature because requires to modify the whole concept of stack assignment. Usually, if assigned, a variable occurs in the stack assignment at most once; by using this technique a variable may be assigned several times in the stack. As consequence, it is necessary to rewrite at least the entire procedure of backtracking.

*“Computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity,
and especially because it produces objects of beauty.”*

Donald E. Knuth

Acknowledgements

We would like to express our deepest gratitude to all the people who gave us the possibility to complete this thesis. We would like to thank Prof. Enrico Giunchiglia for giving us the chance to develop a specific and custom work and for his fundamental supervision during these months. We are especially grateful to him for his patience and his numerous interventions and clarifications.

We thank *Star-Lab* for the organization and the possibility to improve our knowledge. In particular, it is necessary to remember the help of Dr. Paolo Marin and Dr. Massimo Narizzano with every kind of problem.

Finally we want to thank our parents and friends for their help and support, which allowed us to have this experience and realize this thesis.

Bibliography

- [1] Edmund Clarke, Daniel Kroening, and Karen Yorav. Behavioral consistency of c and verilog programs using bounded model checking. Technical Report CMU-CS-03-126, Carnegie Mellon University, School of Computer Science, 2003.
- [2] Alessandro Armando, David A. Basin, Yohan Boichut, Yannick Chevalier, Luca Compagna, Jorge Cuéllar, Paul Hankes Drielsma, Pierre-Cyrille Héam, Olga Kouchnarenko, Jacopo Mantovani, Sebastian Mödersheim, David von Oheimb, Michaël Rusinowitch, Judson Santiago, Mathieu Turuani, Luca Viganò, and Laurent Vigneron. The avispa tool for the automated validation of internet security protocols and applications. In Etesami and Rajamani [3], pages 281–285.
- [3] Kousha Etesami and Sriram K. Rajamani, editors. *Computer Aided Verification, 17th International Conference, CAV 2005, Edinburgh, Scotland, UK, July 6-10, 2005, Proceedings*, volume 3576 of *Lecture Notes in Computer Science*. Springer, 2005.
- [4] Gerard J. Holzmann. Book review: Design and validation of computer protocols (prentice hall, 1991). *SIGCOMM Comput. Commun. Rev.*, 21(2):14, 1991. Reviewer-Lars-Ake Fredlund.
- [5] Armin Biere, Edmund M. Clarke, Richard Raimi, and Yunshan Zhu. Verifying safety properties of a power pc microprocessor using symbolic model checking without bdds. In *CAV*, pages 60–71, 1999.
- [6] Christoph Kern and Mark R. Greenstreet. Formal verification in hardware design: a survey. *ACM Transactions on Design Automation of Electronic Systems.*, 4(2):123–193, 1999.

- [7] Fady Copt, Limor Fix, Ranan Fraer, Enrico Giunchiglia, Gila Kamhi, Armando Tacchella, and Moshe Y. Vardi. Benefits of bounded model checking at an industrial setting. In *CAV*, pages 436–453, 2001.
- [8] Ofer Strichman. Accelerating bounded model checking of safety properties. *Formal Methods in System Design*, 24(1):5–24, 2004.
- [9] R.H. Kibria. Actin solver description. submitted to SAT-Race 2006.
- [10] K. Pipatsrisawat and A. Darwiche. A lightweight component caching technique for satisfiability solvers. Paper submitted to SAT’07, 2007.
- [11] E. Giunchiglia, M. Narizzano, and A. Tacchella. Monotone literals and learning in qbf reasoning. In *Principles and Practice of Constraint Programming (CP2004)*, 2004.
- [12] S. Cook. The complexity of theorem proving procedures. In Association for Computing Machinery, editor, *Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [13] L. Levin. Universal search problems. *Problems of Information Transmission*, 9:265–266, 1973.
- [14] L. Levin. A survey of russian approaches to perebor (brute-force searches) algorithms. *Annals of the History of Computing*, 6:384–400, 1984.
- [15] S.A. Cook and D.G. Mitchell. Finding hard instances of the satisfiability problem: A survey. *Dimacs Series in Discrete Mathematics and Theoretical Computer Science*, 35:1–17, 1997.
- [16] N. Een and N. Sörensson. Minisat - a sat solver with conflict-clause minimization. poster for SAT 2005, 2005.
- [17] Knot Pipatsrisawat and Adnan Darwiche. Rsat 2.0: Sat solver description. Technical Report D-153, Automated Reasoning Group, Computer Science Department, UCLA, 2007.
- [18] Results of sat-competition 2007. <http://www.cril.univ-artois.fr/SAT07/>.
- [19] Results of sat-race 2006. <http://fmv.jku.at/sat-race-2006/results.html>.
- [20] L. Ryan. Efficient algorithms for clause-learning sat solvers. Master’s thesis, Simon Fraser University, 2003.

- [21] Zhaohui Fu, Yogesh Marhajan, and Sharad Malik. zchaff sat solver. Technical report, Department of Electrical Engineering, Princeton University, NJ 08544, USA, 2004.
- [22] Siert Wieringa, Hans van Maaren, and Marijn Heule. Minimarch. Website, 2008. <http://www.st.ewi.tudelft.nl/sat/minimarch.php>.
- [23] L. Zhang, C.F. Madigan, M.H. Moskewicz, and S. Malik. Efficient conflict driven learning in a boolean satisfiability solver. In *International Conference on Computer- Aided Design (ICCAD'01)*, pages 279–285, November 2001.
- [24] M. Davis and H. Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7:201–215, 1960.
- [25] M. Davis, G. Logemann, and D. Loveland. A machine program for theorem-proving,. *Communications of the ACM*, 5:394–397, 1962.
- [26] W. Freeman. *Improvements to Propositional Satisfiability Search Algorithms*. PhD thesis, Department of computer and Information science, University of Pennsylvania, 1995.
- [27] C.M. Li and Anbulagan. Heuristics based on unit propagation for satisfiability problems. In *IJCAI97 [41]*, pages 366–371, 1997.
- [28] O. Dubois and G. Dequen. A backbone-search heuristic for efficient solving on hard 3-sat formulae. In *IJCAI*, pages 248–253, 2001.
- [29] E. Hirsch and A. Kojevnikov. Unitwalk: A new sat solver that uses local search guided by unit clause elimination. Technical report, Steklov Institute of Mathematics at St. Petersburg, 2001.
- [30] J.P. Marques-Silva and K.A. Sakallah. Grasp – a new search algorithm for satisfiability. In *IEEE/ACM International Conference on Computer-Aided Design*, pages 220–227, November 1996.
- [31] R.J.J. Bayardo and R.C. Schrag. Using csp look-back techniques to solve real-world sat instances. In *Fourteenth National Conference on Artificial Intelligence (AAAI '97)*, pages 203–208, Providence, Rhode Island, 1997.

- [32] H. Zhang. Sato: an efficient propositional prover. In *International Conference on Automated Deduction (CADE '97)*, volume 1249, pages 272–275, 1997.
- [33] M.W. Moskewicz, C.F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: Engineering an efficient sat solver. In *38th DAC*, 2001.
- [34] E. Goldberg and Y. Novikov. Berkmin: a fast and robust sat solver. In *DATE*, 2002.
- [35] A. Nadel. The jerusat sat solver. Master’s thesis, Hebrew University of Jerusalem, 2002.
- [36] M.D.T. Lewis, T. Schubert, and B.W. Becker. Early conflict detection based bcp for sat solving. In *The International Conference on Theory and Applications of Satisfiability Testing*, 2004.
- [37] P. Beame, H. Kautz, and A. Sabharwal. Understanding the power of clause learning. In *Eighteenth International Joint Conference on Artificial Intelligence (IJCAI-2003)*, 2003.
- [38] J. Huang. The effect of restarts on the efficiency of clause learning. In *AAAI-06 Workshop on Learning for Search*, 2006.
- [39] M. Luby, A. Sinclair, and D. Zuckerman. Optimal speed-up of las vegas algorithms. *Israel Symposium on Theory of Computing Systems*, pages 128–133, 1993.
- [40] H. Zhang and M.E. Stickel. An efficient algorithm for unit propagation. In *Fourth International Symposium on Artificial Intelligence and Mathematics (AI-MATH '96)*, Fort Lauderdale (Florida, USA), 1996.
- [41] *Fifteenth International Joint Conference on Artificial Intelligence (IJCAI97)*, Nagaya, Japan, August 1997.